

Contents

IoT Exam Notes — Key Concepts	5
1. IoT Fundamentals	5
2. Elixir & BEAM	6
3. Metadata, RDF & Neo4j	6
4. C Programming & Memory	7
5. ADC & Sampling	8
6. Radio & ESP32 WiFi	8
7. Radio Standards	8
8. Stream Processing & MQTT	9
9. Performance Evaluation	10
10. Budget Management	11
11. Annotating Identity, Time and Space	11
Quick-Reference: Common Gotchas	12
Introduction to Internet of Things (IoT) & Big Data	13
Course Overview & Meta	13
Prerequisites Checklist	13
The 6 Domains of IoT Systems	13
Core Definitions: Cutting Through the Hype	13
Lecture 2: Drinking the Elixir	14
Part 1: Introduction to the BEAM Ecosystem	14
Part 2: Functional Programming Foundations	15
Part 3: Language Walkthrough & Syntax Primitives	15
Part 4: The Actor Model & GenServer Architecture	16
Part 5: Fault Tolerance & Supervision Trees	17
Part 6: Distributed Edge Mesh with Nerves	18
Lecture 3: Metadata and Information Modeling	19
Part 0: Introduction & Motivation	19
Part 1: RDF Ontologies	20
RDF: Resource Description Framework	20
SPARQL & Query Complexity	21
Ontology Construction (OWL & RDFS)	22
World Assumptions: Open vs. Closed	23
Part 2: Property Graphs with Neo4j	23
Querying with Cypher	24
Part 3: Semantic Web (RDF) vs. Property Graphs (Neo4j)	25
Lecture 4: To C or Not to C	26
Part 1: Introduction & The Embedded Case for C	26
Part 2: Hardware & Processor Execution Model	26
Part 3: Compilation Pipelines	28
Part 4: Basic Language Abstractions	28
Part 5: Logic and Control Flow Constructs	29
Part 6: Data Constructs	30
Part 7: Pointer Mechanics & Function Indirection	31
Part 8: Precompiler Directives	32
Part 9: The Ten Commandments of C	32

Lecture 5: Building a Toolchain	34
Part 1: Physical Signals & Circuit Fundamentals	34
Part 2: Thermistors & Voltage Dividers	34
Part 3: Input & Output (I/O) Regimes	35
Part 4: Serial Communication & RS232	36
Part 5: Parsing & Dispatching Incoming Data	36
Lecture 6: Sampling Physical Phenomena	38
Part 1: High-Level Overview & Pipelines	38
Part 2: The Temporal Aspect of Sampling	38
Part 3: Analog to Digital Conversion (ADC)	39
Part 4: Signal Imperfections & Noise	40
Part 5: Bandwidth & Offloading Demands	41
Practical Bandwidth Math Application	41
Lecture 7: Radio Communication	43
Part 1: High-Level Overview & Frame Formats	43
Part 2: The Rules of the Game & EM Spectrum	44
Part 3: The Radio Performance Space	44
Part 4: Noise & Environmental Imperfections	45
Part 5: Network Topologies & ACKs	45
Part 6: The OSI Network Model	46
Part 7: TinyOS CC2420 Radio Stack (TEP 126)	47
Lecture 8: Radio Standards	48
Spread Spectrum & Frequency Hopping	48
Beamforming & Backscatter	48
Software Defined Radio (SDR)	48
Low-Power Listening (LPL) & Clear Channel Assessment (CCA)	49
Mesh Networking & Routing	49
Eventual Consistency & Vector Clocks	50
OTA (Over-The-Air) Updates	50
NTP (Network Time Protocol)	50
WiFi (802.11)	51
ESP-WIFI-MESH & ESP-NOW	51
Bluetooth Low Energy (BLE)	51
IEEE 802.15.4 & 6LoWPAN	52
Zigbee & Z-Wave	52
Thread & Matter	52
LoRa & LoRaWAN	53
Sigfox	53
NB-IoT (Narrowband IoT)	53
Lecture 9: Stream Processing	54
Part 0: Anatomy of an IoT Cloud	54
Part 1: Stream Processing Graphs & Nodes	54
Part 2: Concurrency Concerns & remedies	56
Part 3: Message Queues vs. Event Busses	56
Part 4: The Publish-Subscribe Pattern	57
Part 5: MQTT Protocol Specification	57
Part 6: Distributed Concerns & System Failures	58

Lecture 10: Performance Evaluation	60
Part 0: Introduction & Systems Overview	60
Part 1: Experimental Computer Science	60
Part 2: Performance Evaluation Methodology	61
Part 3: The Experimental Testing Pipeline	62
Part 4: Data Visualization	63
Lecture 11: Budget Management	65
Part 1: The Core Philosophy of Budgets	65
Part 2: Energy Calculations & Power Profiles	66
Part 3: Power Throttling & Hardware Optimization	68
Part 4: Protocol Throttling: Low-Power Listening (LPL)	69
Part 5: Algorithms for Active Budget Management	70
Part 6: Resource Trade-off Matrix	71
Lecture 12: Annotating Identity, Time and Space	72
Part 1: Naming	72
Part 2: Identity	72
Part 3: Time	73
Part 4: Space	74
Summary: The Three Annotation Axes	75
Solutions — Exercises 1: Introduction	76
Q1: IoT Definition	76
Q2: The 6 Domains — Smart Building Example	76
Q3: Big Data Consequences	77
Q4: Data → Information → Knowledge → Wisdom	77
Q5: Computation Tier Trade-offs	78
Q6: GDPR Implications	78
Q7: Programming Language Choices	78
Q8: Intelligent vs. Smart	79
Solutions — Exercises 2: Drinking the Elixir	80
Quest 0: Install Livebook	80
Quest 1: Kickstart	80
Quest 2: Getting Acquainted	85
Quest 3: Understanding the LiveMD Format	86
Quest 4: Sources of Information	86
Quest 5: Experimenting with Pipelines	86
Quest 6: Working with Functions	87
Quest 7: Servers Everywhere!	88
Quest 8: World of Zuul — Fixing say	88
Solutions — Exercises 3: Metadata — Adventures in Semantic Madness	90
Quest 1: Exploring the Lecture Example	90
Quest 2: Exploring the Source	90
Quest 3: A New Query	90
Quest 4: Universe Expansion	91
Quest 5: Temperature Actuator (Thermostat)	92
Quest 6: Schema Validation	94

Solutions — Exercises 4: C — Into the Muck	96
Quest 1: Making That First Compile	96
Quest 2: Offsetting the Index	96
Quest 3: In Support of a Hero — TSP	97
Quest 4: The Final Test — Memory Layout Analysis	99
Solutions — Exercises 5: Toolchain — A Device Takes Its First Step	101
Quest 1: Early Steps — Conceptual Notes	101
Quest 2: Blinking Away	101
Quest 3: Serial Count	102
Quest 4: Reporting Changes (Polling)	104
Quest 5: Command and Control	105
Quest 6: Bridging Devices	108
Solutions — Exercises 6: Sampling Physical Phenomena	110
Quest 1: Seeing Potential	110
Quest 2: Guessing Game	111
Quest 3: Getting Real — LMT86 Temperature Conversion	112
Quest 4: Cracks are Forming	113
Quest 5: Root-Causing Noise	115
Quest 6: Laying Pipes — FreeRTOS Pipeline	116
Solutions — Exercises 7: Radio — That Old Wireless Magic	118
Quest 1: Establish an Access Point	118
Quest 2: Obtaining an IP Address	118
Quest 3: Uplink	121
Quest 4: Temperature Uplink	122
Solutions — Exercises 8: Radio Technology	124
Q1: Technology Comparison	124
Q2: Technology Selection	124
Q3: Duty Cycle Regulations	125
Q4: Spreading Factor (LoRa)	125
Q5: Mesh vs. Star Topology	126
Q6: Frequency Hopping	126
Q7: Security Considerations	127
Q8: Gateway Density	127
Solutions — Exercises 9: Stream Processing — Pushing and Transforming Data	128
Quest 1: Installing an MQTT Broker	128
Quest 2: Signal Generator	128
Quest 3: Moving Averages	129
Quest 4: Achieving Absolute Humidity	130
Quest 5: Topic Structure and Latency	131
Quest 6: Data Flows	132
Quest 7: Chatty Room	133
Solutions — Exercises 10: Performance Evaluation — Streaming Humidity	135
Quest 1: Test Harness	135
Quest 2: Initial Setup	135
Quest 3: End-to-End Latency	136

Solutions — Exercises 11: Budget Management — Getting the Most for Your Money	138
Quest 1: Running Average	138
Quest 2: Thresholding	140
Solutions — Exercises 12: Annotating Identity, Time and Space	142
Q1: The Three Annotation Axes	142
Q2: Identity Sources	142
Q3: Identity Assignment Points	143
Q4: Timestamping	143
Q5: Time Sources	144
Q6: Space — Position Sources	145
Q7: Precision vs. Accuracy in Space	145
Q8: Full Annotation Pipeline — Agricultural Soil Moisture System	146
Re-exam 2025 — Model Answers	148
Question 1 (short) — What is a C “union”?	148
Question 2 (short) — FreeRTOS task interface	148
Question 3 (short) — What goes into a C header file?	148
Question 4 (short) — Adding a .c/.h file combination to a FreeRTOS project	149
Question 5 (detailed) — Elixir actor communication: call vs. cast	149
Question 6 (short) — ADC_ATTEN_DB_11 practical implications	149
Question 7 (detailed) — Noise, adaptive sampling, and radio budget	150
Question 8 (short) — Benefit of persistent sessions in MQTT	150
Question 9 (detailed) — Beamforming: benefits and mechanism	150
Question 10 (short) — Why benchmarking in public cloud is hard	151
Question 11 (short) — Routing table for multihop routing	151
Question 12 (short) — When to choose LoRa over LoRaWAN?	151
Question 13 (short) — External ADC	152
Question 14 — Clever Cup System Design	152

IoT Exam Notes — Key Concepts

1. IoT Fundamentals

The 6 Domains (every IoT system spans all six): 1. Data Collection → 2. Data Transportation → 3. Information/Metadata Management → 4. Data Processing → 5. Decision Support → 6. Actuation

The 3 Tiers: - **Edge** — the physical device; constrained, real-time, offline-capable - **Fog** — regional aggregator/gateway; intermediate compute - **Cloud** — unlimited compute/storage; not real-time

Technical IoT definition: A cyber-physical system of individually addressable devices with significant logic running elsewhere (not the buzzword “everything with WiFi”).

Data → Information → Knowledge → Wisdom (DIKW): - Data: raw value (0.73) - Information: data + context (73% RH, room U181, 14:32) - Knowledge: information + pat-

tern/rule (>70% RH promotes mould) - Wisdom: knowledge + judgement/action (increase ventilation now, schedule maintenance in 48h)

Big Data problem: 10k buildings × 200 sensors × 1 Hz × 8 bytes = **16 MB/s** — store raw for 24h, aggregate to 5-min averages for 90 days, daily stats long-term.

GDPR: Occupancy data becomes personal data if it can identify a person (even without name). Requires lawful basis, data minimisation, purpose limitation, retention policy.

Why Java is bad at the Edge: JVM needs 50–200 MB RAM (MCU has 256–520 KB), GC stop-the-world pauses violate real-time, slow cold-start. Fine in the cloud.

Smart vs. Intelligent: Smart = reacts to predefined rules. Intelligent = reasons, adapts, learns autonomously without reprogramming.

2. Elixir & BEAM

Pattern matching: = is a match operator, not assignment. {a, b} = {1, 2} binds a=1, b=2. Pin operator ^a matches against existing binding.

Key data types: Atoms (:ok, :error) — named constants, not garbage collected. Tuples — fixed-size. Lists — linked list, [head | tail]. Maps — %{key => val}.

Anonymous functions need .() to call: f.(x). Named functions use Module.fun(x).

Pipe operator |> passes result of left as first arg to right. dbg() lets you inspect and toggle individual pipeline stages interactively in Livebook.

GenServer: Separates interface functions (client-side API) from callback functions (server-side logic). - call — synchronous, waits for reply. Use for reads/queries. - cast — asynchronous, fire-and-forget. Use for updates/events (avoids deadlock). - handle_call/3 returns {:reply, response, new_state} - handle_cast/2 returns {:noreply, new_state}

Registry: Lets processes register under a name and be looked up without knowing their PID. Supports unique (one process per key) and duplicate (many) patterns.

Supervision tree: Supervisors restart crashed children automatically. Strategies: :one_for_one (restart only failed child), :one_for_all (restart all), :rest_for_one (restart failed + those started after it).

Concurrency model (BEAM): Lightweight processes (not OS threads), no shared memory, message passing only. Millions of processes possible. BEAM schedules them cooperatively → no GC stop-the-world, no OS context-switch overhead.

Fork-join: spawn N worker processes → each sends result back → collect with receive. Pattern: spawn → work → send result → receive all.

3. Metadata, RDF & Neo4j

Why metadata? Raw sensor readings (stream 17 = 0.73) are meaningless without context — what does 17 mean? Where is it? What unit?

RDF Triples: Everything is {subject, predicate, object}. Resources identified by URIs. Schema-free — anyone can add any triple.

SPARQL query pattern:

```
SELECT ?room WHERE {  
  ?room a schema:Room .  
  ?room schema:contains ?sensor .  
  ?sensor schema:provides ?stream .  
  ?stream schema:hasUnit unit:Kelvin .  
}
```

Brick Schema: Standard ontology for building systems. Key relationships: hasPart, is-locatedIn, provides, hasUnit.

SHACL: Schema validation layer for RDF — defines rules like “every sensor must have a unit.” Without it, anything can be added (as discovered by Giles, who burned a small building).

Neo4j / Cypher: Nodes have labels (:Room), relationships have types ([:CONTAINS]). Cypher reads like graph diagrams:

```
MATCH (b:Building)-[:CONTAINS]->(r:Room)-[:HAS_STREAM]->(s:Stream {unit: "Kelvin"})  
RETURN r.name
```

OPTIONAL MATCH: Allows the query to return rows even when the optional pattern is absent — essential for thermostat queries where some rooms may lack sensors.

Key insight: The same building model can be queried for thermostat control, HVAC optimisation, GDPR compliance — the ontology separates data from application logic.

4. C Programming & Memory

Pointers: `int *p = &x` — `p` holds the address of `x`. `*p` dereferences it. Pointer arithmetic `p + 1` moves by `sizeof(*p)` bytes.

Memory layout: Stack (local vars, grows down), Heap (malloc, grows up), BSS (uninitialised globals), Data (initialised globals), Text (code). Casting `int*` to `long*` changes stride — adjacent `int` array elements will not align with `long` reads.

FreeRTOS task creation:

```
xTaskCreate(TaskFunction, "Name", stack_size, pvParameters, priority, &handle);
```

`pvParameters` is `void*` — cast to your actual type inside the task.

Queue pattern:

```
QueueHandle_t q = xQueueCreate(size, sizeof(int));  
xQueueSendToBack(q, &value, timeout); // producer  
xQueueReceive(q, &value, timeout);    // consumer
```

Pipeline pattern (FreeRTOS): `TaskSample` → `[tx_queue]` → `TaskTransmit`. Tasks are decoupled; queue buffers differences in rate.

Dispatch table (C): Array of {name, function_pointer} structs. dispatch(string) splits input, looks up name, calls matching function. Enables extensible CLI without if-chains.

5. ADC & Sampling

Nyquist-Shannon: Sample at $\geq 2 \times$ the **highest frequency** of the signal. Sampling below this causes **aliasing** (high-freq content folds back as false low-freq signal).

ADC types: - **SAR (Successive Approximation):** Binary search from MSB down. n -bit result needs n clock cycles. Common in MCUs (ESP32). Low power. - **Flash ADC:** One comparator per level ($2^n - 1$). Extremely fast (single cycle), but hardware scales exponentially — impractical for high bit depths.

ESP32 ADC: 12-bit (0-4095), reference ~ 3.3 V. Raw value $\rightarrow$ voltage $\rightarrow$ temperature with `adc_oneshot_read()`.

LMT86 temperature conversion:

1. Raw ADC $\rightarrow$ voltage: ``V = raw * V_ref / (2^12 - 1)``
2. Voltage $\rightarrow$ temperature (datasheet Section 8.3.1): ``T = (V - offset) / scale``
3. Do integer arithmetic as long as possible before final truncation to `int8_t`.

Noise: Wall power introduces 50 Hz mains noise into ADC readings (visible as periodic oscillation). Battery power is cleaner. Source: ground loops and capacitive coupling.

6. Radio & ESP32 WiFi

WiFi 2.4 GHz: 13 channels (EU), 22 MHz wide, only channels **1, 6, 11** are **non-overlapping**.

WPA2 4-way handshake: ANonce (AP) $\rightarrow$ SNonce + MIC (client) $\rightarrow$ GTK + MIC (AP) $\rightarrow$ ACK. Derives PTK from both nonces + PMK + MACs.

TCP socket flow: `socket()` $\rightarrow$ `connect()` $\rightarrow$ `send()` / `recv()` $\rightarrow$ `close()`. `htons()` converts host $\rightarrow$ network byte order for port numbers.

WiFi callback pattern (ESP32): Register a callback for `IP_EVENT_STA_GOT_IP`. Inside callback: create a FreeRTOS task that opens a TCP socket. This decouples networking from application logic.

ESP-NOW: Connectionless, peer-to-peer, no router, up to 250-byte payload, ~ 1 ms latency. Uses MAC addresses as peer identifiers.

7. Radio Standards

Spread spectrum: Widens signal across more bandwidth than needed $\rightarrow$ resistant to interference and interception. - **FHSS** (Frequency Hopping): Jumps between channels pseudo-randomly. Bluetooth uses Adaptive FH (79 channels, avoids occupied ones). - **DSSS** (Direct

Sequence): Multiplies data with pseudo-noise chip sequence. Used in WiFi 802.11b and GPS.

LoRa (physical layer): Chirp Spread Spectrum. Parameters: Spreading Factor (SF7-SF12), Bandwidth, Coding Rate. Higher SF = longer range, slower data rate, more time-on-air.

SF	~Data Rate	~Range (rural)
7	5.5 kbps	2 km
9	1.8 kbps	5 km
12	0.3 kbps	15 km

LoRaWAN (MAC layer): Star-of-stars. Device → Gateway → Network Server → App Server.
 - Class A: uplink-triggered, two short RX windows after each uplink (lowest power) - Class B: scheduled downlink slots via beacon - Class C: always receiving (highest power) - OTAA (join via JoinRequest/JoinAccept) vs ABP (hardcoded session keys — less secure) - EU 868 MHz ISM band: **1% duty cycle limit** - AES-128 for encryption. Frame counter prevents replay attacks.

BLE: - **GAP:** Advertiser (broadcasts on channels 37/38/39) vs. Scanner/Initiator - **GATT:** Profile → Service (UUID) → Characteristic (value + properties) - Notify = server pushes, no ACK. Indicate = server pushes, requires ACK.

Zigbee vs Z-Wave: - Zigbee: 2.4 GHz, up to 65k nodes, overlaps with WiFi channels - Z-Wave: sub-GHz (868 MHz EU), max 232 nodes, better wall penetration, proprietary

Thread/Matter: - Thread: IPv6 mesh over 802.15.4, RPL routing, no single point of failure - Matter: application-layer interoperability (Apple/Google/Amazon), runs over Thread/WiFi/Ethernet

OTA updates (A/B dual-bank): Download to inactive bank → verify checksum/signature → set boot flag → reboot → app calls mark_valid or bootloader rolls back.

NTP stratum: Stratum 0 = atomic/GPS, Stratum 1 = server connected to stratum 0, etc.
 Offset = $\frac{(T_2 - T_1) + (T_3 - T_4)}{2}$. Accuracy ~1-10 ms internet, <1 ms LAN. Use PTP (IEEE 1588) for s precision.

Mesh routing: - Distance Vector (Bellman-Ford): each node advertises its table to neighbours. Counting-to-infinity problem fixed by split horizon / poison reverse / hold-down timers. - Link State (Dijkstra / OSPF): flood full topology, run shortest path. Faster convergence, higher overhead. - RPL: designed for lossy networks, builds DODAG rooted at border router.

8. Stream Processing & MQTT

MQTT QoS: - QoS 0 — at most once (fire and forget) - QoS 1 — at least once (retransmit until ACK, may duplicate) - QoS 2 — exactly once (4-step handshake: PUBLISH → PUBREC → PUBREL → PUBCOMP)

MQTT wildcards: + = one level. # = rest of path (must be last). Topics are case-sensitive UTF-8.

Last Will and Testament (LWT): Client registers a backup message on connect. Broker publishes it automatically if client disconnects unexpectedly.

Broker intelligence spectrum:

Dumb Broker/Smart Client $\longleftrightarrow$ Smart Broker/Dumb Client
Kafka (exact topic) MQTT (wildcards) sMAP Data Portal (ontologies)

COP Theorem: Under overload, a stream processor can guarantee **Completeness** (keep all data) OR **Timeliness** (process immediately), never both.

Message Queue vs. Event Bus: - Message Queue: every message processed reliably, TTL + ACK loop, requeues on failure. Use for billing, audit. - Event Bus: low-latency, stale data dropped. Use for real-time control, clock sync.

Stream node properties to consider: - Overload policy (buffer vs. drop) - Trigger condition (barrier sync vs. latest values) - Batch size (throughput $\uparrow$ latency $\uparrow$) - Fault tolerance / state recovery

Flow vs. Enum vs. Stream (Elixir): - Enum — eager, entire collection in memory - Stream — lazy, one element at a time, avoids intermediate collections - Flow (GenStage) — concurrent partitioned processing across cores

BEAM for distributed IoT: Kubernetes for heterogeneous/low-granularity deployments. BEAM (Erlang/Elixir) for homogeneous/high-granularity (millions of lightweight processes, built-in supervision, native clustering).

Performance constant (disk vs. register): Disk is ~ 25 million $\times$ slower than a CPU register. Keep hot state in memory.

9. Performance Evaluation

Good hypothesis: falsifiable, isolates one variable. Bad: “show system A is faster than B” (biased), or “the internet increases productivity” (unmeasurable).

Metrics: - **Latency:** $t_{\text{render}} - t_{\text{event}}$ - **Throughput:** operations per second - **Memory footprint:** RAM used by a process

Parameters vs. Factors: Parameters = held constant. Factors = varied to study effect on metrics. Full-factorial = 2^k combinations for k binary factors — use hill-climbing when too expensive.

Workload types: Synthetic (generated) vs. Replayed (from production traces).

Clock alignment problem: Monotonic clocks safe on single machine. Across machines: clocks drift due to thermal variance — must sync (NTP/PTP) or bound uncertainty mathematically.

Plot types: | Plot | Use for | |—|—| | Scatter | Time series, correlations | | Histogram | Distribution of raw samples | | KDE | Smooth density estimate | | CDF | Reading exact percentiles (e.g. 95th) | | Violin | Comparing distributions across factors |

Plot requirements: Must be self-contained, every axis labelled with unit, y-axis starts at 0 unless log scale or epoch timestamps, save as vector format (PDF/SVG).

10. Budget Management

The four budgets: Radio, Memory, Processing, **Energy** (binding — if exhausted, all others drop to zero).

Energy math:

$$P = V \cdot I \quad E = P \cdot t = V \cdot I \cdot t$$

Duty cycle model:

$$P_{\text{avg}} = D \cdot P_{\text{active}} + (1 - D) \cdot P_{\text{sleep}} \quad D = \frac{t_{\text{active}}}{t_{\text{active}} + t_{\text{sleep}}}$$

Battery lifetime: $t_{\text{lifetime}} = \frac{\text{Capacity (mAh)}}{I_{\text{avg}} \text{ (mA)}}$

Sleep states: Active (10-50 mA) → Idle/Sleep (1-5 mA, s wake) → Deep Sleep (1-50 A, ms wake). Lower state = more latency to wake up.

DMA: Peripheral writes directly to RAM without waking the CPU — main core stays asleep while data streams in from sensors.

LPL (Low-Power Listening): Receiver sleeps, wakes briefly to check for RF energy. Transmitter prepends a preamble $\geq$ receiver's sleep period. Transfers energy cost from receiver to transmitter. Optimal when RX is battery-powered and transmissions are rare.

Relative energy costs: - CPU instruction: $1\times$ - Flash read: $10\times$ - Flash write: $100\text{--}1000\times$ - Sensor sampling: $1000\times$ - Radio transmission: **$100,000\text{--}1,000,000\times$** ← compress locally, don't send raw

Budget allocation algorithm (5 steps): Track B_{rem} , track T_{rem} , split into N chunks, allocate $B_{\text{chunk}} = B_{\text{rem}}/N$, evaluate each event against remaining chunk budget.

Flash endurance: 10,000-100,000 write/erase cycles. Erase unit = full sector (expensive). Use RAM ring buffers to minimise write frequency.

11. Annotating Identity, Time and Space

Why annotation matters: Raw data from a sensor is meaningless without knowing *which device*, *when*, and *where*. These three axes transform raw data into information.

Identity

Sources of identity: 1. Unique device number (serial from chip, MCU, or radio module) 2. User-provided (entered via app during installation) 3. Physical location-derived (GPS coordinate hardcoded at deployment) 4. Network-derived (DNS name, network topology enumeration) 5. Context-derived (combination of what/where/what-happens-around-it)

Where identity lives: - On the node (self-aware, works offline, zero-touch enrollment) - Cloud (central registry, large managed fleets) - Private / user-held (privacy-preserving, zero-knowledge)

A single device may carry **multiple identities** at different exposure levels (serial for provisioning, UUID for application, human label for dashboard).

Time

Sources of time on-device: - Offset (counter since boot — no wall clock) - GNSS (accurate, power-hungry) - USB injection at flash time - Downstream push from gateway - NTP

Why multiple GNSS systems? Selective Availability — governments can degrade GPS (e.g., US degraded civilian GPS during Kargil War 1999). Galileo (EU), GLONASS (RU), BeiDou (CN) reduce geopolitical dependency.

Timestamping points in the pipeline:

Device	Gateway	Cloud
1. At sampling	3. At reception	5. At reception
2. At transmission	4. At push	6. At delivery

Earlier = more accurate but requires synchronised device clock. Later = simpler but adds variable network delay.

Space

Direct positioning: Hardcoded firmware, GPS/GNSS, downstream push, USB injection.

Indirect (triangulation): RSSI from known neighbours, WiFi/BLE fingerprinting. Less accurate indoors due to multipath reflections — best for room-level presence, not precise coordinates.

Triangulation math: Given distances to ≥ 3 known positions, solve for unknown position. Exact if distances are precise; least-squares optimisation if noisy.

Quick-Reference: Common Gotchas

- cast vs call in GenServer: always use cast for fire-and-forget events (slider updates, notifications) — call can deadlock if both processes call each other synchronously.
- Zigbee and WiFi both use 2.4 GHz — Zigbee channels 11-14 overlap with WiFi channels 1/6/11.
- ABP (LoRaWAN) hardcodes session keys — device can be cloned, no per-session key derivation.
- OPTIONAL MATCH in Cypher / OPTIONAL { } in SPARQL — use when some rooms may be missing sensors; otherwise the query returns nothing for incomplete rooms.
- Nyquist: sample rate must be $\geq 2 \times$ **highest frequency**, not just higher than it.
- htonl() — host to network byte order (big-endian). Always use for port numbers and IP addresses in socket structs.
- Flash erase granularity: you must erase a full sector (often 4 KB) before rewriting a single byte — never write to flash in a tight loop.
- COP theorem applies at the node/stream level, not just the system level — an individual overloaded GenServer must choose to buffer or drop.

Introduction to Internet of Things (IoT) & Big Data

Course Overview & Meta

- **Lecturer:** Aslak Johansen (asjo@mmmi.sdu.dk)
- **Target Audience:** 78 MSc Software Engineering students
- **Prerequisites:** Equivalent of a Bachelor's degree in Software Engineering

INFO: Weekly Schedule & Expectations * **Tuesdays (4h):** Lecture introducing new material + Exercises with physical availability of lecturer. *Preparation generally needed.* * **Thursdays (4h):** Lab follow-up space in **Room U42**. Lecturer is available *remotely* over Discord. * **Self-Study:** Requires **13.5 hours per week** outside of class for prep, exercises, and exam studying.

Prerequisites Checklist

The course expects familiarity with or will superficially touch upon: - **Computer Systems:** bits, bytes, bit shifting, memory, bus architectures - **Concurrency:** threads, race conditions - **Operating Systems:** process management - **Statistics:** basic foundations - **Electronics:** basic circuits - **Electromagnetism:** superficial overview for radio contexts

The 6 Domains of IoT Systems

An IoT system bridges the gap from raw physical phenomena to actions via six distinct dimensions: 1. **Data Collection:** Sensing at the device layer. 2. **Data Transportation:** Moving data across networks. 3. **Information/Metadata Management:** Organizing definitions and schemas. 4. **Data Processing:** Ingesting and transforming datastreams. 5. **Decision Support:** Evaluating processing data to make choices. 6. **Actuation:** Realizing a decision physically.

Core Definitions: Cutting Through the Hype

Internet of Things (IoT)

- **The Buzzword Definition:** Consumer gadgets controlled from a single app.
- **The Technical Course Definition:** > “A cyber-physical system consisting of a large number of individually and globally addressable devices which have a significant portion of their logic running elsewhere”

The many competing players (vendors, telcos, enterprises) each pursue their own agenda, which leads to three systemic problems:

1. **Siloed access** — every vendor wants “the whole cake” (exclusive control of their ecosystem).
2. ****Extern**

Lecture 2: Drinking the Elixir

Part 1: Introduction to the BEAM Ecosystem

The Origin of Erlang

- Developed in the 1980s by Ericsson (pioneered by figures like Joe Armstrong) to address the stringent requirements of telephone exchange systems.
- Engineered from the ground up for specific runtime properties:
 - **Distribution:** Inherent multi-node coordination.
 - **Soft Real-Time Properties:** Predictable execution timelines.
 - **Zero Downtime:** Continuous operations remaining online even during active software upgrades and hardware faults.

Scale & Industry Adoption

The BEAM ecosystem runs highly resilient, massive-scale systems worldwide:

- **Discord:** Utilizes Elixir to orchestrate over 5 million concurrent users.
- **WhatsApp & Facebook Chat:** Relies on Erlang infrastructure to manage messaging servers processing millions of active channels per node.
- **Cisco & Ericsson:** Powers 40% of cellular data infrastructure (GPRS, 3G, 4G, 5G); historical routing analysis showed up to 90% of global internet traffic crossing Erlang-based nodes.

The Core Paradigm: Option of Uniformity

Instead of deploying a highly fragmented, brittle software stack, the BEAM runtime unifies diverse infrastructure requirements into a single platform:

Operational Need	Traditional Stack Candidates	The BEAM Ecosystem Approach
HTTP Server / Request Processing	Nginx / Phusion Passenger / Ruby on Rails	Erlang / Elixir Native
Long-running Requests	Go	Erlang / Elixir Native
Server-wide State	Redis	Erlang / Elixir Native
Background Jobs / Persistence	Redis / MongoDB / Cron / Bash / Ruby	Erlang / Elixir Native
Service Crash Recovery	Upstart / Systemd	Erlang / Elixir Native (Supervisors)

QUOTE: Saša Jurić (*Elixir in Action*)

“Your runtime is your foundation. It is your ground, and when your ground is solid then you can move on top of it with a lot of confidence at a steady reasonable pace...”

Why Choose Elixir?

- **First-Class Paradigms:** Concurrency, testing, and comprehensive documentation are treated as first-class language citizens.
- **Ecosystem Frameworks:**
 - **Web:** Phoenix and LiveView enable real-time, single-page collaborative applications without complex JavaScript client states.
 - **Embedded:** The **Nerves Framework** allows developers to flash minimal Linux kernels booting straight into the BEAM on Raspberry Pi class hardware.

Part 2: Functional Programming Foundations

Imperative vs. Declarative Mindsets

- **Imperative Programming:** Focuses closely on mutable data transformations. Code behaves like a sequential recipe instructing the machine exactly *how* a data state must be modified to reach a result.
- **Declarative Programming:** Focuses cleanly on the target result. Explicit control flows are abstractly omitted, describing *what* the application must structurally achieve.

Core Functional Properties in Elixir

- **Immutability & Persistence:** Data cannot be modified in-place once written. Purely functional structures achieve *persistence*, safely retaining reference paths to previous configurations without data corruption. Variables can be rebound to new states.
- **Expressions Everywhere:** Blocks, conditionals, and assignments lack statement separation keywords (like `;`); everything evaluates directly to a value.
- **Tail Recursion over Loops:** The language completely lacks traditional looping constructs (`for`, `while`). Iteration is achieved via function recursion, optimized at runtime to match execution costs.
- **Pure Functions & Concurrency:** Pure functions yield no side effects (I/O, shared memory mutations) and are strictly idempotent. Because pure operations cannot interfere with one another, they can be processed concurrently with zero race condition risks.

Part 3: Language Walkthrough & Syntax Primitives

Core Technical Types

- `integer`: Arbitrary sized values. Standard division (`/`) transitions values to floats; strict integer operations utilize `div/2` and `rem/2`.
- `atom`: Immutable constants where their name is their value (e.g., `:ok`, `:error`). **Warning:** Atoms are part of a global system index and are **never garbage collected**.
- `list`: Immutable linked list data structure (`[head | tail]`). Fast structural prepending, slow sequential index access.

- tuple: Fixed-size arrays defined with curly braces {}. Fast for matching structure patterns, expensive to dynamically expand.

Pattern Matching Primitives

The = symbol is not an assignment operator; it acts as a structural **match operator**.

```
# Basic structural binding
iex(1)> {status, value} = {:ok, 200}
{:ok, 200}

# Enforcing an explicit match check
iex(2)> {:ok, result} = {:error, "Failed"}
** (MatchError) no match of right hand side value: {:error, "Failed"}

# The underscore (_) behaves as a write-only discard wildcard
iex(3)> [head | _tail] = ["Bonobo", "Gorilla", "Gibbon"]
```

The Pipe Operator (|>)

To eliminate nested functional configurations and intermediate variable bindings, the pipe operator pushes the evaluation output of a preceding function into the first parameter slot of the following operation:

```
# Procedural nesting style:
report(process(lookup(input("Name?"), query)))

# Elegant pipeline format:
input("Name?")
|> lookup(query)
|> process()
|> report()
```

Part 4: The Actor Model & GenServer Architecture

Conceptual Definition

An Actor (referred to natively as a **Process**) is an isolated execution entity encapsulating:

1. A unique isolated **State** memory pool.
2. Targeted **Functionality** instructions.
3. A sequential, FIFO input **Mailbox (Message Queue)**.

Processes operate strictly concurrently and independently. Communication occurs purely through messaging passes. While an actor's mailbox is completely empty, it remains in an idle, inactive sleep state, consuming near-zero CPU cycles.

Implementing an OTP GenServer

The fundamental structural pattern for building persistent actors with state relies on the native GenServer behavior:


```

defmodule StateServer do
  use GenServer

  # --- Client API Implementation ---
  def start_link() do
    GenServer.start_link(__MODULE__, nil)
  end

  def get_value(pid) do
    # 'call' executes a synchronous request blocking the client until a response arrives
    GenServer.call(pid, {:get})
  end

  def set_value(pid, new_val) do
    # 'cast' executes an asynchronous fire-and-forget message immediately returning
    GenServer.cast(pid, {:set, new_val})
  end

  # --- Server Core Callbacks ---
  @impl true
  def init(_) do
    {:ok, 0} # Sets internal starting state to 0
  end

  @impl true
  def handle_call({:get}, _from, state) do
    {:reply, state, state} # Enforces format: {:reply, reply_value, new_state}
  end

  @impl true
  def handle_cast({:set, new_val}, _state) do
    {:noreply, new_val} # Enforces format: {:noreply, new_state}
  end
end

```

Part 5: Fault Tolerance & Supervision Trees

The “Let It Crash” Philosophy

Traditional software engineering embraces defensive patterns—sprinkling complex exception catches throughout core logic loops to manually catch runtime errors.

BEAM applications leverage **offensive programming**: assuming that when a bug surfaces, internal process states are compromised, making recovery guesswork unreliable. Because every process operates in a fully sandboxed memory zone, a crash inside one actor cannot corrupt surrounding execution channels.

The system relies on a **Supervisor process** to observe the crash, wipe out the corrupted process, and immediately spin up a pristine replacement node using a known clean state configuration.

Supervisor Restart Strategies

When a child process crashes, Supervisors evaluate how to restore dependencies based on targeted lifecycle plans:

- **:one_for_one:** Only the specific child actor that crashed is targeted for a restart loop.
- **:one_for_all:** If any child crashes, every single sibling actor managed under that supervisor is terminated and initialized fresh.
- **:rest_for_one:** If a child crashes, it and any subsequent sibling actors defined later within the supervisor initialization list are targeted for restarts.

Part 6: Distributed Edge Mesh with Nerves

- **Unified Virtual Environment:** Nerves changes embedded firmware deployment by converting edge electronics directly into part of a distributed BEAM network mesh.
- **Location-Agnostic Logic:** Because communications between nodes utilize native BEAM communication protocols, the distinction between calling local operations vs. sending data across Edge → Fog → Cloud nodes is completely abstracted away.
- **Data-Centric Deployments:** This allows engineers to shift operational code across edge devices and remote server backends seamlessly based on power budgets, processing requirements, and real-time bandwidth profiles.

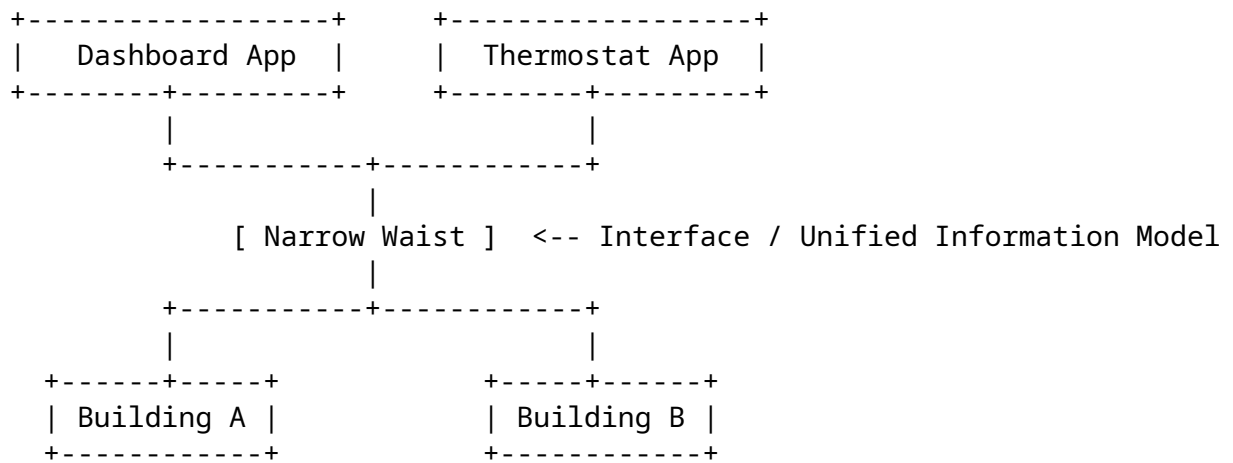
Lecture 3: Metadata and Information Modeling

Part 0: Introduction & Motivation

Cyber-Physical Systems Mapping Problem

Consider a standard building management ecosystem:

- **The Environment:** A physical building composed of rooms, temperature sensors, radiator actuators, and target temperature setpoints per room.
- **The Applications:**
 1. A dashboard displaying a color-coded floor map based on measured room temperatures.
 2. A thermostat application dynamically controlling the temperature parameters of each room.



The Technical Challenge: Connecting Data and Control Planes

Every control/telemetry point (sensor, actuator, setpoint) is identified via a unique UUID but operates under varying network and security access permissions.

- **Scenario:** How do we link applications to the correct hardware points in a building?

Architectural Solutions:

1. **Hardcoded Constants:** Scattering UUIDs directly inside application source code. (Fails to scale; fragile).
2. **Per-Application Maps:** Creating custom mapping tables for each app:

$$room \mapsto (sensor \mid actuator \mid setpoint) \mapsto UUID$$

3. **Shared Semantic Interfaces (The Narrow Waist):** Decoupling applications from physical topologies via a shared information model interface. This approach provides:
 - **Portability:** Apps can run seamlessly on different building layouts without re-compilation or codebase changes.

- **Maintainability:** Topological updates (e.g., moving a sensor) do not break the high-level applications.

Part 1: RDF Ontologies

Basic Semantic Statements

Statements require explicit meaning (*intensions*) rather than assumptions:

Statement: "cec78d2a-14b8-41d6-bf46-c6a4d07574a7 is in room U181"

- cec78d2a-14b8-41d6-bf46-c6a4d07574a7 is a UUID.
- This UUID identifies a physical telemetry data stream.
- U181 represents a specific room.
- **Implied Fact:** The physical data stream originates from sensor equipment installed inside room U181.

Understanding Ontologies

An ontology provides a domain-specific vocabulary to ensure humans and machines have a shared understanding of data representations:

- **Formal Naming & Definition:** Utilizes structured, standardized machine-readable languages.
- **Types, Properties, and Relations:** Defines things through their classes, associated properties, and explicitly declared relationship models.
- **Domain Limitations:** Ontologies target specific areas (e.g., the *Brick Schema* for building topologies), but multiple ontologies can be merged within a single system.

NOTE: The Human vs. Machine Agreement Gap

Humans are highly adaptive and can resolve slightly mismatched mental models dynamically. In contrast, machines require absolute precision. Disagreements in data representation between separate software components directly produce structural faults and system failures. Ontologies serve as machine-readable contracts to prevent these discrepancies.

RDF: Resource Description Framework

The Semantic Triple

RDF represents knowledge using **Triples**:

Subject × Predicate × Object

- A database of these relationships is called a **Triple Store**.
- Real-world facts are defined exclusively through these explicit, connected relationships, not through isolated entities.

Subject	Predicate	Object
demo:Marge	person:parentOf	demo:Bart
demo:Marge	rdf:type	person:Female
person:Female	rdfs:subClassOf	person:Human

Namespaces & CURIs

To prevent global naming collisions, RDF uses namespaces resolved through **Compact URIs (CURIs)**:

- foaf:Person expands directly to `http://xmlns.com/foaf/0.1/Person`.
- Performing an HTTP GET request on a CURI with the header `Accept: application/rdf+xml` fetches the formal definition documents for that namespace.

Turtle Syntax (.ttl)

Turtle (*Terse RDF Triple Language*) is a compact text-based formatting standard for serialization:

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix brick: <https://brickschema.org/schema/1.1.0/Brick#> .
@prefix demo: <http://ontologies.sdu.dk/Demo#> .
```

```
demo:floor a brick:Floor .
demo:room a brick:Room ;
          rdfs:label "U181" ;
          brick:isLocatedIn demo:floor .
```

SPARQL & Query Complexity

SPARQL Query Pattern Matching

SPARQL matches patterns at the triple level. Below is an example querying for room sensors and actuators:

```
SELECT DISTINCT ?room_name ?sensor_uuid ?setpoint_uuid ?actuator_uuid
WHERE {
  ?room      rdf:type/rdfs:subClassOf* brick:Room .
  ?sensor    rdf:type/rdfs:subClassOf* brick:Temperature_Sensor .
  ?setpoint  rdf:type/rdfs:subClassOf* brick:Temperature_Setpoint .
  ?actuator  rdf:type/rdfs:subClassOf* brick:Radiator_Valve_Position .

  ?sensor    brick:pointOf ?room .
  ?setpoint  brick:pointOf ?room .
  ?actuator  brick:pointOf ?room .

  ?room      brick:label ?room_name .
  ?sensor    brick:label ?sensor_uuid .
  ?setpoint  brick:label ?setpoint_uuid .
```

```

    ?actuator brick:label      ?actuator_uuid .
}

```

Triple Store Query Complexity

Executing triple matching queries introduces significant computation costs:

- **Table Scans:** Without optimization, matching N variables in a query requires $1 + n$ nested full table scans, causing execution times to grow exponentially under large models.
- **Indexing Options:** Dedicated RDF engines attempt to speed up lookups using specialized indices:
 - Subject-Predicate indices
 - Subject-Object indices
 - Predicate-Object indices
- **Trade-off:** If the query's path patterns are highly expressive, engines must fall back to expensive table scans. Some architectures compromise by dropping full RDF flexibility to optimize graph-mode indices under OWL constraints.

Ontology Construction (OWL & RDFS)

OWL (Web Ontology Language)

OWL extends basic RDF with formal, object-oriented semantics:

- **Cardinality Constraints:** Limits on relationship counts (e.g., a person can have at most one biological mother).
- **Relationship Inferences:** Declares transitive, symmetric, or inverse relationships.
- **Classification and Equivalency:** Rules to evaluate subsumption (general vs. specific classes) or bridge external schemas.

Key Constructors

- **Classes & Subclasses (`rdfs:subClassOf`):** Builds class DAG hierarchies.
- **Object Properties (`owl:ObjectProperty`):** Defines relations pointing to other URI entities.
- **Datatype Properties (`owl:DatatypeProperty`):** Defines relations pointing to literal values (using XSD schemas: int, double, boolean, string, dateTime).
- **Domain Restrictions (`rdfs:domain`):** Restricts the valid subject classes of a relationship:

Subject $\in$ Domain

- **Range Restrictions (`rdfs:range`):** Restricts the valid object classes of a relationship:

Object $\in$ Range

```
# Defining properties, domain, and range in OWL
demo:hasMother  a          owl:ObjectProperty ;
                 rdfs:domain demo:Person ;
                 rdfs:range  demo:Female .
```

World Assumptions: Open vs. Closed

Closed World Assumption (CWA)

- **Definition:** Any statement that is not currently known to be true is assumed false.
- **Behavior:** Requires a complete, bounded set of facts. This makes structural constraints easily testable and verifiable.

Open World Assumption (OWA)

- **Definition:** The truth of a statement is independent of whether it is currently known.
- **Behavior:** If a constraint is not explicitly found in the database, the system does not assume it is false—it may simply exist elsewhere. This makes complete logical verification impossible, but prevents false negatives when combining distributed metadata models.

TIP: Validators vs. Reasoners

- **Validators (e.g., SHACL):** Validate models against strict closed-world constraints. They ensure spelling errors are flagged, connections are valid, and floating or isolated entities are prohibited.
- **Reasoners:** Leverage open-world rules to infer *new* logical facts from existing statements (e.g., deriving `isMotherOf` from `isParentOf + Female`).

Part 2: Property Graphs with Neo4j

```
(Node: Label {key: value}) ---[:RELATION {key: value}]---> (Node: Label)
```

Graph Theory Primitives

- **Nodes (Vertices):** Entities containing properties, represented as boxes or circles.
- **Edges (Relationships):** Typed, directional connections spanning a source node and destination node.
- **Weighted Graphs:** Graphs where every edge contains numerical values (weights or properties).
- **Paths:** A connected sequence of traversal transitions. Used to calculate distance and reachability metrics.
- **Cycles:** A path that begins and ends at the exact same node. Systems often try to enforce **Directed Acyclic Graphs (DAGs)** to prevent cyclic processing loops.

Neo4j Data Structure Models

```
// High-level conceptual representations of Neo4j primitives
class Node {
    ISet<string> labels;                // Class/Type designations
    IDictionary<string, Object> properties; // Inner Key-Value parameters
    IDictionary<string, ISet<Relationship>> outgoing; // Outward edges by type
    IDictionary<string, ISet<Relationship>> incoming; // Inward edges by type
}

class Relationship {
    string type;                        // Single relation name
    IDictionary<string, Object> properties; // Key-Value parameters
    Node src;                           // Source pointer
    Node dst;                           // Destination pointer
}
```

Querying with Cypher

Anatomy of Cypher Matching Patterns

Cypher models path traversals using visual ASCII art structures:

- Nodes are wrapped in parentheses (variable:Label {property: "value"}).
- Edges are drawn inside brackets - [:RELATION_TYPE] ->.

Cypher Examples

1. Matching Specific Father-Daughter Paths

```
MATCH (father:Person {sex: "male"}),
      (daughter:Person {sex: "female"}),
      (father)-[:parentOf]->(daughter)
RETURN father.name, daughter.name
```

2. Matching Same-Sex Parents

```
MATCH (parent:Person)-[:parentOf]->(child:Person)
WHERE parent.sex = child.sex
RETURN parent.name, child.name
```

Execution Planning & Traversal

- **Index Entry Points:** Neo4j leverages property or label indices to locate initial node start points.
- **Pointer Hopping:** Once entry points are resolved, the engine traverses adjacent nodes directly using pointers (**Index-Free Adjacency**), avoiding expensive runtime table joins.
- **Analysis Tools:**
 - EXPLAIN: Shows the static structural execution path generated by the engine.

- PROFILE: Measures real-time performance profiles, execution speeds, and memory allocations.

Part 3: Semantic Web (RDF) vs. Property Graphs (Neo4j)

Structural Model Comparison

Architectural Property	RDF / SPARQL Ecosystem	Neo4j / Cypher Ecosystem
Industry Standing	Open W3C Semantic Standard	Commercial Product/Implementation
Implementations	Multiple (Fuseki, HodDB, rdflib, etc.)	Neo4j Free / Enterprise editions
Core Database Engine	Triple Store	Property Graph
Primitive Abstraction	Logical Fact (Subject-Predicate-Object)	Node and Edge Entities
Schema/Validation Strength	Strong (using OWL, RDFS, SHACL)	Weak (limited constraints)
Adjacency Model	Scanning with indices	Index-free adjacency (Pointer hopping)
Reasoning Support	Strong (Inference rules natively supported)	Theoretical only

Query Language Comparison

Feature	SPARQL Query Language	Cypher Query Language
Underlying Model	RDF Triple Stores	Property Graph
Primitive Match Unit	Triple patterns	Node/Edge patterns
Pattern matching term	Explicit logical restriction	Descriptive path match pattern
Path Directionality	Unidirectional	Bidirectional
Cross-schema Queries	Supported	Not supported
Evaluation Speed	Variable, dependent on index scanning	Consistent, fast index-free traversals

WARNING: GraphQL Misconception

Despite the naming similarities, **GraphQL is NOT a graph query language**. GraphQL is a tree-based API data-fetching schema protocol. It does not support graph-idiomatic primitives like path matching, cycles, or complex traversal algorithms.

Lecture 4: To C or Not to C

Part 1: Introduction & The Embedded Case for C

Why Choose C for Embedded systems?

While high-level languages offer memory safety, **C remains the dominant language for resource-constrained IoT Edge devices** due to specific structural properties:

- **No Garbage Collection:** Stop-the-world garbage collectors (like in Java, C#, Go, or Python) introduce non-deterministic latencies, which are highly problematic for time-sensitive, weak real-time embedded environments.
- **Minimal Footprint:** Compiles straight to bare-metal machine code, maximizing processor and memory efficiency.
- **Low-Level Access:** Unrestricted direct hardware control via raw pointer arithmetic and bit-level manipulation.
- **Pragmatic Abstraction:** Provides the core building blocks of computing (functions, sequential execution, choices, and memory indirection) without unnecessary runtime overhead.
- **Simplicity:** The entire language core is syntactically simple enough to be taught and mastered quickly.

NOTE: Modern Alternatives

While C is the focus of this module, **Rust** and **Zig** are the primary modern candidates sharing these exact compilation and memory characteristics.

Part 2: Hardware & Processor Execution Model

The Von Neumann Memory Map

A physical microcontroller's address space is typically partitioned into consecutive segments growing from lower to higher memory addresses:

Low Addresses High Addresses
[Memory-Mapped I/O] ---> [Code (Flash)] ---> [Heap] ---> <--- [Stack]

- **Memory-Mapped I/O (MM I/O):** Special memory addresses mapped directly to hardware peripheral registers (e.g., GPIO, ADC, Timers). Writing to these addresses controls the physical hardware.
- **Code (Text Segment):** Read-only segment where compiled machine instructions are stored.
- **Heap:** Dynamically allocated memory space that grows upward toward high addresses.
- **Stack:** Function call frames, local variables, and return addresses. It grows downward toward lower addresses.

Step-by-Step CPU Execution Tracing

To understand how C instructions map to hardware, consider an array summation loop.

System Parameters:

- **Memory Data Array** starting at Address 80 containing three integer values: [36, 42, -3].
- **Hardware Registers:**
 - pc (Program Counter): Points to current executing instruction.
 - addr: Holds the current memory array address pointer.
 - sum: Holds the running accumulated total.
 - i: Running loop index.
 - count: Loop exit boundary (3 iterations).
 - value: Temporary registry buffer for fetched data.

(Diagram)

```
jeq i count done → False
load value addr → add sum sum value
add sum sum value → addi i i 1
addi i i 1 → addi addr addr 4
addi addr addr 4 → goto loop
jeq i count done → True
```

[store sum addr]

Step-by-Step Execution State Table:

Step	Instruction	pc	addr	value	sum	i	count	Notes
0	Initial State	10	80	0	0	1	4	Array base is 80
1	jeq i count done	10	80	0	0	1	4	$1 \neq 4$, loop continues
2	load value addr	14	80	36	0	1	4	Fetch array element 1 at address
3	add sum sum value	18	80	36	36	1	4	$\text{sum} = 0 + 36$
4	addi i i 1	22	80	36	36	2	4	Increment loop counter
5	addi addr addr 4	26	84	36	36	2	4	Move pointer by 4 bytes (32-bit i
6	goto loop	30	84	36	36	2	4	Jump back to loop start
7	jeq i count done	10	84	36	36	2	4	$2 \neq 4$, continue
8	load value addr	14	84	42	36	2	4	Fetch array element 2 at address
9	add sum sum value	18	84	42	78	2	4	$\text{sum} = 36 + 42$
10	addi i i 1	22	84	42	78	3	4	Increment loop counter
11	addi addr addr 4	26	88	42	78	3	4	Move pointer to address 88
12	goto loop	30	88	42	78	3	4	Jump
13	jeq i count done	10	88	42	78	3	4	$3 \neq 4$, continue
14	load value addr	14	88	-3	78	3	4	Fetch array element 3 at address
15	add sum sum value	18	88	-3	75	3	4	$\text{sum} = 78 + (-3)$
16	addi i i 1	22	88	-3	75	4	4	Increment loop counter
17	addi addr addr 4	26	92	-3	75	4	4	Move pointer to address 92
18	goto loop	30	92	-3	75	4	4	Jump
19	jeq i count done	10	92	-3	75	4	4	$4 = 4$, Jump to done
20	store sum addr	34	92	-3	75	4	4	Write accumulated sum 75 to add

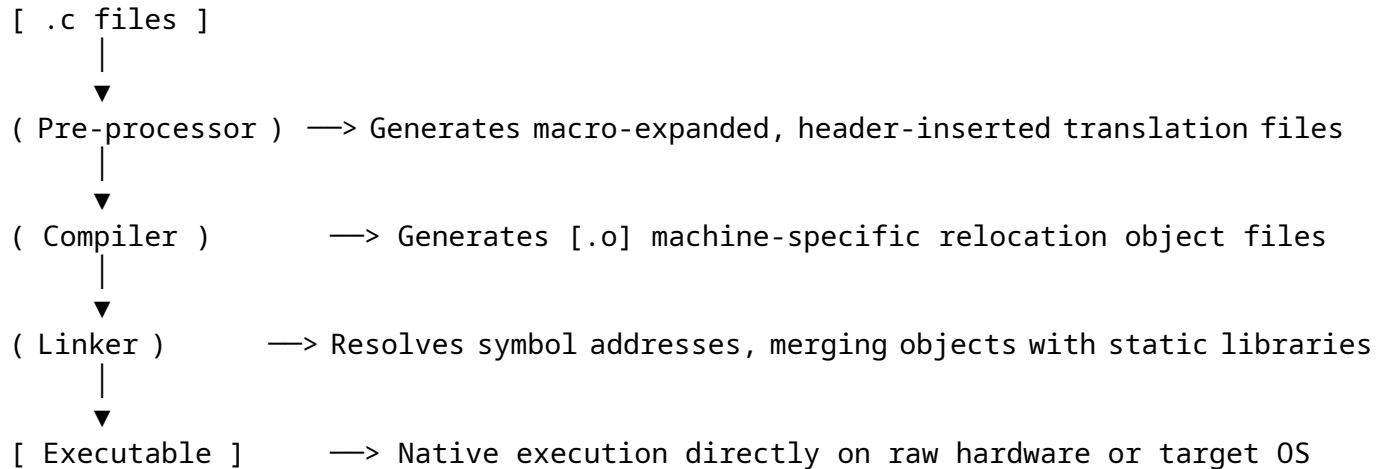
Part 3: Compilation Pipelines

Unlike runtime-interpreted systems, C uses ahead-of-time compilation. Below is a structural comparison of Java and C build chains:

Java Compilation Model

[.java files] $\rightarrow$ (Compiler) $\rightarrow$ [.class bytecode] $\rightarrow$ (Archival) $\rightarrow$ [.jar] $\rightarrow$ (JVM)

C Compilation Model



Part 4: Basic Language Abstractions

Structural Matrix of Data and Instructions

Architectural Property	Data Representation	Instructional Representation
Repetition	Array	while, do-while, for
Sequence	struct	; (Statement separator), Function block
Choice	union	if-else, switch
Indirection	Pointer	Pointer dereferencing

C Type Rules

- **Simple Types (Integers):** Sizes are platform-dependent but must strictly adhere to:

$$\text{sizeof(char)} \leq \text{sizeof(short)} \leq \text{sizeof(int)} \leq \text{sizeof(long)}$$

- **Floats:** float and double are modeled after standard IEEE 754 float specs.
- **Booleans:** C has no native logical boolean primitive at its core; **any non-zero integer represents true**, and **zero represents false**.
- **Strings:** Strings are not objects; they are raw, continuous, zero-terminated arrays of characters (char array ending with `\0`).

Formatted Output (printf)

Uses the C **ellipsis operator** (...) to implement variadic parameters:

```
#include <stdio.h>
printf("The temperature is %2.1f degrees Celsius.\n", 21.78);
```

- **printf**: Outputs formatted streams to standard output (stdout).
- **sprintf**: Formats and prints target data straight into an allocated string variable buffer (char*).

Part 5: Logic and Control Flow Constructs

Goto Statements: The Structured Cleanup Pattern

While considered bad practice in desktop application development, **the goto statement is extremely useful in low-level embedded system code** to prevent nested “Pyramid of Doom” conditional error tracking when managing hardware locks or allocations.

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

#define BUF_SIZE (16)

int main(int argc, char* argv[]) {
    char* filename = argv[1];

    // Attempt Heap Allocation
    char* buf = (char*) malloc(BUF_SIZE * sizeof(char));
    if (!buf) goto failure; // Error routing

    // Attempt File Stream Allocation
    FILE* f = fopen(filename, "r");
    if (!f) goto cleanup_buf;

    size_t actual = fread(buf, sizeof(char), BUF_SIZE, f);
    if (actual != BUF_SIZE) goto cleanup_file;

    // Successful operations
    printf("File read successful: '%s'\n", filename);

    // Orderly teardown
cleanup_file:
    fclose(f);
cleanup_buf:
    free(buf);
    return 0;

failure:
    printf("Fatal allocation failure. Error code: %d\n", errno);
    return 1;
```

```
}
```

Part 6: Data Constructs

Custom Type Definitions (typedef)

typedef creates a convenient alias for existing primitive or complex structures.

```
typedef unsigned short uint16_t; // Guarantees a 2-byte unsigned type on 32-bit MCUs
```

WARNING: Overflow Hazard

Under-allocated types wrap around silently when they exceed or fall below their bounds:

```
uint16_t var = 0;
var--; // var silently wraps around to 65535
```

Enumerations (enum)

Enums map names to integer constants.

```
typedef enum {
    CAT_UNKNOWN,      // Defaults to 0
    CAT_STRING,        // Defaults to 1
    CAT_INTEGER = 10, // Overrides default value sequence
    CAT_FLOAT          // Increments from preceding definition -> evaluates to 11
} category_t;
```

Unions

Unions allow multiple members to share the exact same physical memory footprint. The total memory allocated for a union equals the size of its largest data member.

```
typedef union {
    double reading; // 8 bytes
    int count;      // 4 bytes
} payload_t;       // Total allocated union size: 8 bytes
```

WARNING: Memory Overlap Risk

Modifying one union property directly overwrites the bytes of the others. Writing to `p.count` compromises the floating-point bitmask representation of `p.reading`.

Structs

Structs allocate consecutive blocks of memory to hold grouped sequential values.

```
typedef struct {
    double x;
    double y;
} point_t;
```

- **Access Modes:**

- Dot operator (p.x): Used when referencing direct, non-pointer struct variables.
- Arrow operator (p->x): Used when dereferencing structural pointers to access members: (*p).x.

Uninitialized Arrays: Stack vs. Heap Allocation

```
// Stack Allocation (Hazardous if left uninitialized!)
int values[10]; // Contains whatever garbage memory was left over on the stack segment

// Heap Allocation (Initialized safely with calloc, or manually)
int* values = (int*) malloc(10 * sizeof(int));
```

Part 7: Pointer Mechanics & Function Indirection

Core Rules of Pointers

1. **Pointers are simply addresses:** At the hardware level, a pointer variable is simply an integer holding a target memory address.
2. **The Address-Of Operator (&):** Fetches the physical address of an active variable.
3. **The Dereference Operator (*):** Accesses or writes to the value stored at the address pointed to by the pointer.
4. **Pointer Arithmetic is typed:** Incrementing a pointer (ptr++) increases the address value by the size of the underlying type in bytes.
5. **Casting via void*:** Any pointer type can be cast to any other pointer type by going through the generic void* intermediary.

(Diagram)

Pointer Variable: holds address 0x2000 → Dereference
 [Memory Address 0x2000: holds value 42]

Function Pointers

C allows functions to be treated as variables. This is the cornerstone of writing polymorphic code, callbacks, and dispatch tables in embedded systems:

```
#include <stdio.h>
```

```
// Define a type alias for a function pointer accepting and returning an integer
typedef int (*int2int_function) (int);
```

```
int incr(int i) { return i + 1; }
```

```
void map(int data[], int length, int2int_function fun) {
    for (int i = 0; i < length; i++) {
        data[i] = fun(data[i]); // Execute target function pointer
    }
}
```

Part 8: Precompiler Directives

The preprocessor performs simple string manipulation on your source files before the code compiles:

- **Macros (#define):** Performs text replacements across compilation runs.
- **Inline Math Macros:** Always wrap macro parameters in parentheses to prevent math precedence issues during expansion:

```
#define RADIUS2DIAMETER(r) ((r) * 2)
```

- **Conditional Compilation (#ifdef):** Toggles code inclusion based on flags passed to the compiler at build time:

```
#ifdef BENCHMARK
// Benchmark logic is only compiled if -DBENCHMARK is passed to gcc
#endif
```

- **Header Guards:** Crucial to prevent compiler double-definition errors when parsing recursive #include trees:

```
#ifndef __POINT_H
#define __POINT_H

// Header prototypes and structures

#endif
```

Part 9: The Ten Commandments of C

To survive writing system-level code in C, every software engineer must adhere to these defensive rules:

1. **Run lint frequently** and study its compiler warnings with care.
2. **Thou shalt not follow the NULL pointer**, for chaos and segfaults await thee at its end.
3. **Cast function arguments** to expected types if they are not matching already to prevent compiler promotion surprises.
4. **Meticulously declare return types** in libraries if headers omit them.
5. **Thou shalt strictly check array bounds** on all loops and string arrays to prevent Buffer Overflows.
6. **Always check function error return codes**, even if error-handling code triples your codebase's physical size.
7. **Study your libraries** and do not reinvent the wheel without direct, system-specific reasons.
8. **Write clean, standard style code** (e.g., One True Brace Style) to keep logic obvious to other engineers.
9. **Ensure your external global variables and identifiers are unique** in their first six characters to prevent linking errors on legacy toolchains.

10. **Do not assume the world is a VAX**—always write portable, architecture-independent code that does not rely on target-specific endianness assumptions.

Lecture 5: Building a Toolchain

Part 1: Physical Signals & Circuit Fundamentals

Analog Signals

- **Definition:** A signal is the physical evolution of the value of some metric over time.
- In electrical and embedded systems, the primary metric of interest is **voltage** (V).
- Measuring voltage allows us to bridge the physical world and the digital/cyber world.

Voltage & Reference Grounds (GND)

- **Definition:** Voltage is a difference in electrical potential between two points.
- **Implication:** Because voltage is a relative metric, it must always reference two specific physical points in a circuit.
- **Floating Circuits:** Connecting two independent circuits with only a single wire will not drive a standing current. The circuits are said to “float” relative to one another. A common **Ground (GND)** wire is mandatory to establish a shared reference.

NOTE: Relative Voltage Reference Example

If you connect the 3V line of *Circuit A* directly to the ground (GND) of *Circuit B*:

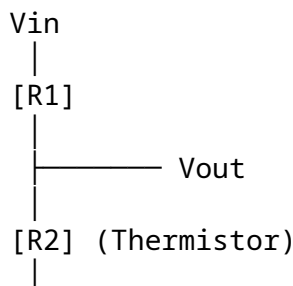
- The 3V rail of *Circuit B* becomes equivalent to 6V from the contextual perspective of *Circuit A*.
- The ground (GND) of *Circuit A* becomes equivalent to $-3V$ from the contextual perspective of *Circuit B*.

Transducers

- **Definition:** A transducer is a physical device that converts energy from one form to another.
- **Inputs include:** Light, heat, mechanical force, and electromagnetic radiation.
- **Outputs include:** A representative electrical signal (commonly modulated as voltage).

Part 2: Thermistors & Voltage Dividers

A **thermistor** (R_2) is a specific class of transducer whose physical resistivity changes as a reversible function of ambient temperature.



GND

The Voltage Divider Equation

Because microcontrollers cannot measure raw resistance directly, we use a voltage divider circuit to translate variable resistance into a readable voltage output (V_{out}):

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2}$$

Operational Assumptions (PTC Thermistor Behavior)

Assuming a Positive Temperature Coefficient (PTC) thermistor where resistance increases with temperature:

- **High Temperature Scenario:**

$$T \uparrow \Rightarrow R_2 \uparrow \Rightarrow V_{out} \uparrow$$

- **Low Temperature Scenario:**

$$T \downarrow \Rightarrow R_2 \downarrow \Rightarrow V_{out} \downarrow$$

Part 3: Input & Output (I/O) Regimes

Modern embedded microcontrollers handle physical interaction via discrete digital or modulated analog regimes:

Output Strategies

1. **Digital Output:** Direct square-wave logic toggling cleanly between 0 (GND) and 1 (V_{CC}).
2. **Digital-to-Analog Converter (DAC):** Generates a true, stepped approximation of a smooth continuous analog waveform directly from binary values.
3. **Pulse Width Modulation (PWM):** Approximates an analog output voltage by rapidly toggling a digital pin. The perceived average analog voltage is determined by the ratio of the **duty-cycle** to the overall signal **period**:

$$V_{avg} = V_{CC} \cdot \frac{\text{duty-cycle}}{\text{period}}$$

Input Strategies

1. **Digital Input:** Evaluates incoming external voltages against internal logic thresholds, mapping them to a binary 0 or 1.
2. **Analog-to-Digital Converter (ADC):** Samples a continuous external analog voltage and quantizes it into a discrete numeric digital representation.

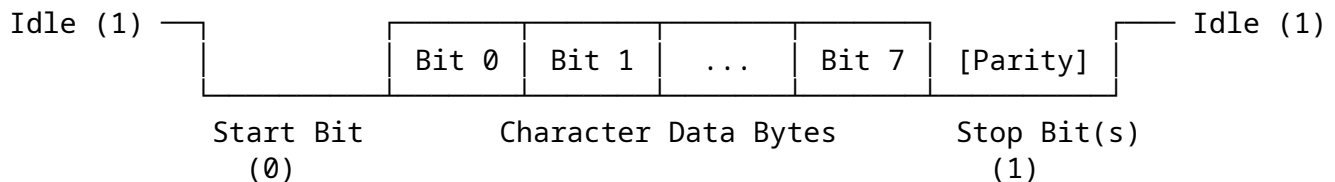
Part 4: Serial Communication & RS232

Introduction to Serial Protocols

- **Definition:** Serial communication transfers data sequentially, one individual bit after the other, over a shared medium.
- **Common Frameworks:** SPI (“spy”), I²C (“I-squared-C”), and RS232 (UART).
- In its simplest physical format, **RS232** is a three-wire protocol:
 1. **Tx:** Transmission line (outgoing data).
 2. **Rx:** Reception line (incoming data).
 3. **GND:** Shared ground reference line.

RS232 Frame Format

Data is packaged and sent at a pre-configured **baud rate** (bits per second) within a structured frame:



- **Start Bit (1 bit):** Forces the line low (0) to signal the receiver to begin synchronization.
- **Character Data (7 to 8 bits):** The payload byte containing the actual data bits.
- **Parity Bit (Optional, 1 bit):** Used for fundamental error-checking.
- **Stop Bits (Typically 1 to 3 bits):** Forces the line high (1) to reset the communication link and prepare for the next frame.

Part 5: Parsing & Dispatching Incoming Data

When designing software to process serial commands arriving on a device, you must make two primary architectural design choices:

1. Data Formatting Choice

- **Machine-Readable Format:** Highly structured binary packets. Minimal runtime overhead to parse, but difficult for developers to debug without translation tools.
- **Human-Readable Format:** Plaintext ASCII commands (e.g., JSON or CSV strings). Extremely simple to debug and monitor, but introduces parsing latency and string conversion overhead on resource-constrained microcontrollers.

2. Dispatch Mechanism

Once data is ingested, the application must map the payload to the correct internal execution routing:

- **Branch Chain:** Sequential conditional evaluations (if-else or switch blocks).
- **Table Lookup:** An array of function pointers indexed by an execution opcode. Very fast and predictable.
- **Map Lookup:** Associating command string keys with logic execution blocks.
- **Deterministic Finite Automaton (DFA):** Resolving command structures progressively using a state machine layout.

Lecture 6: Sampling Physical Phenomena

Part 1: High-Level Overview & Pipelines

To bring physical events into a digital cyber-physical system, signals must traverse a dedicated data collection pipeline:

(Diagram)

```
Continuous Signal → Sampler  
Sampler → Discrete Signal  
Discrete Signal → Interpolator  
Interpolator → Reconstructed Signal
```

The Processing Stages:

1. **The Sampler:** Captures the value of a continuous physical signal (e.g., voltage) at discrete points in time.
2. **The Interpolator:** Reconstructs the continuous signal from the discrete points at the receiving or processing end.

Part 2: The Temporal Aspect of Sampling

Sensing continuous real-world metrics relies on capturing consecutive snapshots at defined intervals.

Key Metrics

- **Sampling Interval / Period (T):** The precise time duration between two consecutive samples.
- **Sampling Frequency / Rate (f_s):** The average number of discrete samples obtained in a single second:

$$f_s = \frac{1}{T}$$

The Nyquist-Shannon Sampling Theorem

To guarantee perfect mathematical reconstruction of a signal without introducing distortion, you must sample at a rate greater than twice the highest frequency component contained in the original signal:

$$f_s > 2 \cdot f_{max}$$

WARNING: Nyquist Violations & Aliasing

Under-sampling ($f_s \leq 2 \cdot f_{max}$) causes high-frequency waveforms to masquerade as lower-frequency signals upon reconstruction. This phenomenon is known as **aliasing**:

- **In Movies (Wagon-Wheel Effect):** Rotational speeds of wheel spokes exceeding half the camera's frame capture rate (f_s) make wheels appear to spin backward or stand still.
- **In Photos (Moiré Patterns):** Interference grids that appear when a fine fabric pattern conflicts with the spatial grid resolution of an image sensor's color filter array.
- **In Audio:** High-frequency tones fold down into audible, distorted low-frequency hums.

Resampling

When adapting a recorded timeseries (a mapped database of time $\rightarrow$ value pairs) for storage, analysis, or transmission, you must change its temporal frequency:

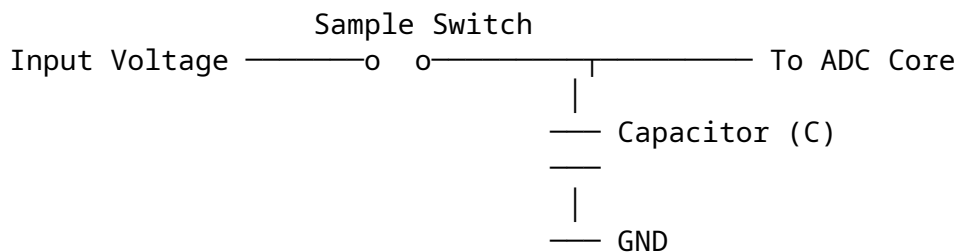
- **Downsampling:** Reducing the sampling rate. This cuts off extreme values, smooths transient fluctuations, and risks introducing aliasing if low-pass anti-aliasing filtering is not applied beforehand.
- **Upsampling:** Increasing the sampling rate. This generates missing points between actual measurements, requiring interpolation (linear, polynomial, or spline).

Part 3: Analog to Digital Conversion (ADC)

Modern microcontrollers rely on internal **Analog-to-Digital Converters (ADCs)** to quantize analog voltages.

The Sample and Hold (S&H) Method

To ensure the input voltage does not fluctuate while the converter is calculating bits, ADCs employ a sample-and-hold circuit:



1. **Flush:** Drain any remaining electrical charge in the capacitor by briefly routing it directly to Ground (GND).
2. **Wait:** Disconnect the capacitor from GND, ensuring the input pathway is isolated and ready.
3. **Sample:** Close the sample switch, connecting the capacitor to the input signal to charge it over a defined sample time window.
4. **Hold:** Open the switch. The capacitor holds an isolated, static voltage equivalent to the input voltage at the moment the switch opened.
5. **Compare:** Evaluate this trapped charge against known voltage references using comparative hardware to resolve the individual digital bits.

ADC Structural Block Diagram

(Diagram)

Input Channels → Mux
Scaling Reference → Mux
Clock Gen → Sample & Hold
Sample & Hold → ADC Core
V_{R+} High Ref → ADC Core
V_{R-} Low Ref → ADC Core
ADC Core → Memory / Registers

Comparator Step Implementations

- **Flash ADCs (Parallel):** Compares the held voltage against all steps in a voltage divider chain simultaneously. Extremely fast, but requires massive hardware overhead ($2^N - 1$ comparators for an N -bit converter).
- **Successive Approximation Register (SAR) ADCs (Sequential):** Uses a binary search algorithm to resolve the digital representation bit-by-bit from MSB (Most Significant Bit) to LSB (Least Significant Bit). Highly energy-efficient and common in microcontrollers.
- **Linear vs. Logarithmic ADCs:** Linear ADCs partition the range ($V_{R-} \leq V \leq V_{R+}$) into equally sized voltage steps (quantization intervals). Logarithmic ADCs partition step sizes exponentially to preserve precision for very small signal levels (common in audio/decibel measurements).

Part 4: Signal Imperfections & Noise

Raw digital signals captured from physical environments are inherently imperfect.

Noise Floor & Signal-to-Noise Ratio (SNR)

The **noise floor** represents the cumulative background noise generated by sensor physics, electromagnetic interference, and digital quantization steps. The ratio of actual signal strength to this noise floor is the **SNR**:

$$\text{SNR} = 10 \cdot \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right)$$

Averaging to Improve SNR

If you average N consecutive samples of a static or slow-moving signal:

1. The deterministic signal components combine linearly ($N \cdot \text{signal}$).
2. The uncorrelated, random thermal noise components cancel out partially, growing only by $\sqrt{N}$.
3. Consequently, the effective SNR improves by a factor of $\sqrt{N}$:

$$\text{SNR}_{\text{improvement}} = \sqrt{N} \implies 3 \text{ dB improvement per doubling of } N$$

Outliers and Black Swans

- **Outliers:** Vaguely defined as measured observations lying significantly outside the brunt of a distribution. They are often removed (via median filters or threshold checks) to clean the dataset.
- **Black Swans:** Low-probability, high-impact events.

DANGER: The Danger of Outlier Removal

Classifying extreme spikes as “sensor noise” or “outliers” and stripping them from raw datasets can blind your systems to rare but highly significant structural shifts. If your baseline models fail to account for these events, they trigger catastrophic failures when reality breaks your structural assumptions.

- *Historical Examples:* The 2011 Fukushima nuclear accident, the 2008 Financial Crisis, the 2020 COVID-19 Pandemic.

Part 5: Bandwidth & Offloading Demands

The resolution and speed at which you sample dictate the raw bandwidth required to stream metrics from the edge.

Bandwidth Matrix (f_s vs. Resolution Depth)

The raw required bitrate (B) is calculated as:

$$B = f_s \cdot \text{Resolution Depth (bits)}$$

Bit-Depth	1 Hz	10 Hz	100 Hz	1.0 kHz	10.0 kHz	100.0 kHz	1.0 MHz
6b	6 bps	60 bps	600 bps	6.0 kbps	60.0 kbps	600.0 kbps	6.0 Mbps
8b	8 bps	80 bps	800 bps	8.0 kbps	80.0 kbps	800.0 kbps	8.0 Mbps
10b	10 bps	100 bps	1.0 kbps	10.0 kbps	100.0 kbps	1.0 Mbps	10.0 Mbps
12b	12 bps	120 bps	1.2 kbps	12.0 kbps	120.0 kbps	1.2 Mbps	12.0 Mbps
16b	16 bps	160 bps	1.6 kbps	16.0 kbps	160.0 kbps	1.6 Mbps	16.0 Mbps

Practical Bandwidth Math Application

Problem Scenario:

We have a **12-bit ADC** with a known **2-bit noise floor**, sampling at 8 kHz. We want to offload this stream over a serial UART link.

1. Theoretical Bandwidth (No Overhead): Since the 2 least significant bits (LSBs) are corrupted by noise, they do not contain real information. The useful signal depth is $12 \text{ b} - 2 \text{ b} = 10 \text{ b}$ per sample.

$$B_{\text{theoretical}} = 10 \text{ bits} \cdot 8000 \text{ Hz} = 80 \text{ kbps}$$

2. Practical Bandwidth (UART Framed Edition): To stream this over UART, we package the 10-bit payload into bytes. 10 bits of data require **2 bytes** (16 bits) of payload space if unpacked, or we must chunk the bitstream.

Assuming we chunk the data as raw byte streams (8 bits data blocks) over a standard UART frame format containing:

- 1 start bit
- 8 character data bits
- 0 parity bits
- 2 stop bits

Each 8 bits of actual data requires transmitting **11 physical line bits** (representing a 37.5% structural overhead penalty):

$$B_{\text{practical}} = 80 \text{ kbps} \cdot \frac{11 \text{ framing bits}}{8 \text{ data bits}} = 110 \text{ kbps}$$

WARNING: The Critical UART Alignment Flaw

While 110 kbps fits mathematically within standard baud rates (e.g., 115200 baud), this raw streaming approach lacks **frame alignment wrappers**. If a single bit error occurs on the serial line, the receiver's byte parser will slip out of alignment with the 10-bit sample boundaries, corrupting all subsequent readings.

Lecture 7: Radio Communication

Part 1: High-Level Overview & Frame Formats

Transitioning from wired connections (e.g., UART/RS232) to wireless links transforms how embedded systems serialize and packetize telemetry data.

Bit-Level Hardware Frame Layout

To transmit frames efficiently over a shared wireless medium, data is structured into compact, bit-aligned frames.

channel (6b)		type (4b)		source (8b)		destination (8b)		size (6b)		payload (64b)
--------------	--	-----------	--	-------------	--	------------------	--	-----------	--	---------------

- **Header Elements (32 bits total):**
 - **Channel (6 bits):** Logical radio channel selector.
 - **Type (4 bits):** Frame categorization/payload identifier.
 - **Source (8 bits):** Hardware address of the sender node.
 - **Destination (8 bits):** Hardware address of the target recipient node.
 - **Size (6 bits):** Identifies the length of the trailing payload segment (0 to 2^6 bytes).
- **Payload (0 to 64 bytes / 0 to 2^6 bytes):** Actual application data stream.
- **CRC (8 bits):** Cyclic Redundancy Check byte for error detection.

Packed Struct Bitfield Implementation in C

To prevent compiler padding from expanding the memory footprint of frames, developers leverage the `__attribute__((packed))` compiler directive alongside C bitfields:

```
#include <stdio.h>
```

```
typedef struct frame_t {  
    unsigned int channel    : 6;    // 6 bits  
    unsigned int type      : 4;    // 4 bits  
    unsigned int source     : 8;    // 8 bits  
    unsigned int destination : 8;    // 8 bits  
    unsigned int size       : 6;    // 6 bits  
    char*        payload;    // 64-bit pointer on modern systems (8 bytes)  
    unsigned int src        : 8;    // 8 bits  
} __attribute__((packed)) frame_t;
```

```
int main(int argc, char* argv[]) {  
    frame_t f;  
    f.type = 3;  
  
    printf("Type of frame: %u\n", f.type);  
    printf("Size of structure: %lu bytes\n", sizeof(frame_t));  
}
```

```
// Theoretical Bit Sum: 6 + 4 + 8 + 8 + 6 + 64 (pointer) + 8 = 104 bits / 8 = 13 bytes
printf("Calculation: (6.0+4+8+8+6+64+8)/8 = %f\n", (6.0+4+8+8+6+64+8)/8);
return 0;
}
```

Part 2: The Rules of the Game & EM Spectrum

The Electromagnetic Spectrum & Frequencies

Radio waves are classified by their physical oscillation frequency.

- **Regulated Spectrum:** Licensed bands reserved strictly for dedicated institutional services (e.g., cellular, aviation).
- **Open/Unlicensed Spectrum (ISM Bands):** Shared bands (e.g., 2.4 GHz, 868 MHz) where IoT systems operate.
- **The Frequency Trade-off:** Under equivalent encoding parameters and transmitter power budgets:

Lower Frequency $\implies$ Longer Range | Lower Throughput

Higher Frequency $\implies$ Shorter Range | Higher Throughput

Signal Propagation & The Inverse Square Law

An omnidirectional electromagnetic field spreads outward in three dimensions. The signal's intensity (i) is inversely proportional to the square of the distance (d) from the source:

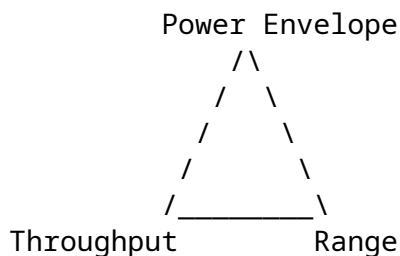
$$i \propto \frac{1}{d^2}$$

WARNING: Operational Consequence

Because signal intensity decays polynomially with distance, trying to increase transmission range by simply boosting output power is highly inefficient.

Part 3: The Radio Performance Space

Designing wireless devices involves navigating a strict three-way trade-off. Improving any single parameter forces a compromise in at least one of the remaining two:



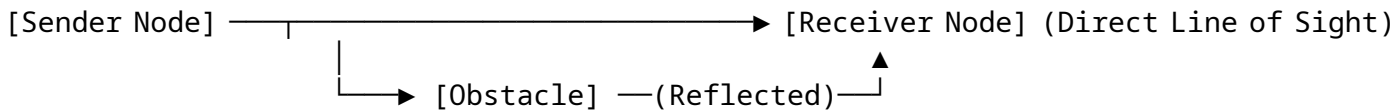
- **Throughput:** Data transfer rates (bits/second).
- **Power Envelope:** Energy consumption (directly limiting battery life).

- **Range:** Maximum physical link distance.

Part 4: Noise & Environmental Imperfections

Wireless signals must compete with environmental dynamics:

- **The Noise Floor:** Inherent electromagnetic background radiation picked up by antennas when no active signals are being transmitted.
- **Multipath Propagation:** Radio waves reflect off surfaces, absorb into materials, pass through transparent mediums, and split into multiple path trajectories.



- **Interference Primitives:**
 - **Constructive Interference:** Reflected paths arrive in-phase, reinforcing and boosting the signal amplitude.
 - **Destructive Interference:** Reflected paths arrive out-of-phase, partially or fully canceling out the incoming signal.
- **Shared Media & Collisions:** Because the wireless medium is inherently shared, multiple nearby transmitters can emit overlapping signals in both time and space. The resulting constructive and destructive interference corrupts the packets, resulting in **collisions** that lose data.

Error Protection: Cyclic Redundancy Checks (CRC)

To handle bitflips caused by electromagnetic noise, packets include a CRC block. Receivers recalculate the checksum on arrival and silently discard any corrupted frames.

Part 5: Network Topologies & ACKs

Structural Link Layouts

Line: (Node) — (Node) — (Node)

Star:

```

  (Node)      (Node)
    \        /
     — (Node) —
    /        \
  (Node)      (Node)
  
```

Ring:

```

  (Node) — (Node)
    |       |
  (Node) — (Node)
  
```

- **Line:** Daisy-chained sequential links. High latency at the ends.
- **Star:** Peripheral nodes communicate exclusively through a central hub. Simple, but the hub is a single point of failure.

- **Ring:** Connected circular loops.
- **Tiered / Tree:** Hierarchical parent-child nodes. Common in large building automations.
- **Mesh:** Highly redundant, peer-to-peer links. If one path fails, the network dynamically self-heals by routing packets through alternative neighbors.

Acknowledgement (ACK) Architectures

Since packet loss is common on wireless links, networks use ACKs and automatic retransmissions. Three main strategies define when ACKs are sent:

1. **Immediate ACK:** Sent immediately after successfully receiving each individual frame.
2. **Delayed/Block ACK:** Sent in bulk after an entire multi-frame packet is processed.
3. **Piggybacked ACK:** Held and appended directly to outgoing data frames traveling in the opposite direction.

Part 6: The OSI Network Model

The open systems interconnection (OSI) framework partitions communication architectures into abstraction layers.

NOTE: SDU Academic OSI Model Variant (Levels 1-10)

While standard networking curriculums stop at the 7th (Application) layer, SDU's IoT curriculum humorously extends this stack to model real-world organizational and socio-political dependencies:

- **Level 10 — Government:** Legislation, standards compliance, and radio spectrum laws.
- **Level 9 — Organization:** Enterprise directives and operational guidelines.
- **Level 8 — Individual:** Application users and human communication requirements.
- **Level 7 — Application:** APIs and high-level software interfaces.
- **Level 6 — Presentation:** Encryption, decryption, and data serialization (e.g., JSON, Protocol Buffers).
- **Level 5 — Session:** Connection establishment, teardown, and session persistence.
- **Level 4 — Transport:** Port multiplexing, connection-oriented packet tracking, and flow control.
- **Level 3 — Network:** Multi-hop routing, path calculation, and logical addressing.
- **Level 2 — Data Link:** Framing, local MAC addressing, and reliable single-hop frame delivery.
- **Level 1 — Physical:** Physical layer bits/symbols, frequency modulation, and electromagnetic links.

Part 7: TinyOS CC2420 Radio Stack (TEP 126)

TinyOS abstracts low-power IEEE 802.15.4 operations using a modular, highly layered component stack:

(Diagram)

```
Active Message Layer → Adds
Unique Send → Assigns
Packet Link → Handles
Low-Power Listening → Controls
Unique Receive → Filters
TinyOS Network → Ensures
CSMA Layer → Carrier
```

[Physical Transceiver]

Stack Components Explained:

1. **Active Message Layer:** Introduces a port-like abstraction to dispatch incoming packet types to their registered application handler.
2. **Unique Send:** Assigns sequential, unique packet IDs to outgoing streams to assist downstream deduplication.
3. **Packet Link:** Manages automatic retransmissions at the link layer based on transactional ACK feedback.
4. **Low-Power Listening (LPL):** Periodically wakes up the radio transceiver to sample the channel. If a preamble is detected, it stays awake to capture the packet; otherwise, it goes back to sleep to save power.
5. **Unique Receive:** Monitors incoming packet IDs and discards duplicate packets caused by retransmissions.
6. **TinyOS Network:** Handles header translation and network-level interoperability.
7. **CSMA (Carrier-Sense Multiple Access):** Listens to the physical channel before transmitting. If another device is transmitting, it backs off to prevent collisions.

Lecture 8: Radio Standards

INFO: Student-Driven Lecture This lecture consists of student group presentations on radio standards and related techniques. Topics range from physical-layer fundamentals to full protocol stacks.

Spread Spectrum & Frequency Hopping

Spread spectrum intentionally widens a signal across a broader frequency band than strictly required, making it more resistant to interference and interception.

Frequency Hopping Spread Spectrum (FHSS) — invented by Hedy Lamarr (patent 1942, inspired by piano-roll hopping) — switches the carrier frequency in a pseudo-random sequence known to both sender and receiver.

- Bluetooth uses **Adaptive Frequency Hopping (AFH)**: 79 channels at 1 MHz spacing across 2.4 GHz. Detects occupied channels (e.g. WiFi) and excludes them from the hop sequence.
- **Direct Sequence Spread Spectrum (DSSS)**: multiplies data with a pseudo-noise chip sequence, spreading it across a wide band. Used in WiFi 802.11b and GPS.

FHSS vs DSSS trade-off: FHSS avoids narrowband interference by hopping away; DSSS tolerates wideband noise through processing gain.

Beamforming & Backscatter

Beamforming focuses RF energy directionally rather than omnidirectionally: - Multiple antennas transmit the same signal with controlled phase/amplitude offsets - Constructive interference forms a beam toward the target; destructive interference reduces side lobes - Used in 802.11ac/ax (WiFi 5/6) and 5G massive MIMO

Backscatter communication uses no active transmitter on the tag side: - Tag reflects and modulates an incident carrier wave (from a reader or ambient source) - Extremely low power — passive RFID tags harvest energy from the reader's field - **Ambient backscatter:** piggybacks on existing TV/WiFi signals; no dedicated reader needed

Software Defined Radio (SDR)

Traditional radios implement signal processing in dedicated hardware. **SDR** moves this into software:

- A wideband ADC digitises a large chunk of spectrum
- All modulation/demodulation, filtering, and decoding runs in software (CPU/FPGA/GPU)
- Reconfigurable: same hardware can decode FM, ADS-B (aircraft), LoRa, GSM, etc.
- Popular hardware: RTL-SDR (~€20 DVB-T dongle), HackRF, USRP

GNURadio is the dominant open-source SDR framework: visual flow-graph programming of signal processing chains.

Low-Power Listening (LPL) & Clear Channel Assessment (CCA)

Battery-powered IoT nodes must minimise idle radio time. Two complementary strategies:

Low-Power Listening

- Radio wakes up briefly at regular intervals (duty cycle) to sample the channel
- If energy is detected, stays awake to receive; otherwise sleeps again
- **B-MAC** (TinyOS): simple LPL with long preamble — sender transmits a preamble long enough to span the receiver's entire sleep interval
- **X-MAC**: short strobed preambles with embedded address; receiver can acknowledge early, cutting preamble short

Clear Channel Assessment (CCA)

- Before transmitting, sense whether the channel is idle
- **CSMA/CA** (Carrier Sense Multiple Access / Collision Avoidance): sense → random backoff → sense again → transmit
- **RSSI threshold**: if measured received signal strength is below a threshold, channel is deemed clear
- **Energy Detection (ED)** vs **Carrier Sense (CS)**: ED is simpler; CS can distinguish own network's preamble

Mesh Networking & Routing

In a mesh, nodes relay packets for their neighbours, extending range and providing redundancy.

Distance Vector Routing

Each node maintains a table: {destination → (next_hop, cost)}. Periodically broadcasts its table to neighbours; neighbours update their own tables (Bellman-Ford).

Counting to infinity problem: if a link fails, nodes may enter a loop of ever-increasing route costs before converging.

Mitigations: - **Split horizon**: don't advertise a route back to the neighbour you learned it from - **Poison reverse**: advertise failed routes with cost ∞ - **Route poisoning + hold-down timer**: immediately broadcast ∞ , then ignore updates for a hold-down period

Link State Routing

Each node floods its complete local link state to all nodes; everyone builds the full topology graph and runs Dijkstra. Faster convergence but higher overhead. Used in OSPF.

RPL (IPv6 Routing Protocol for Low-Power and Lossy Networks)

Designed for 6LoWPAN/Thread networks. Builds a **Destination-Oriented DAG (DODAG)** rooted at a border router. Nodes select parents based on an **Objective Function** (e.g. ETX — Expected Transmissions).

Eventual Consistency & Vector Clocks

Distributed IoT systems often cannot guarantee strong consistency (no shared global clock, partitioned networks).

Eventual consistency: given no new updates, all replicas will eventually converge to the same value. Acceptable for sensor readings but not for billing or safety-critical control.

Vector clocks track causality without a global clock: - Each node maintains a vector $V[i]$ of logical timestamps for every node - On send: increment own entry, attach vector - On receive: component-wise max of local and received vectors, then increment own entry - Event A happened-before B if $V_A[i] \leq V_B[i]$ for all i and strict for at least one - Concurrent events: neither dominates the other $\rightarrow$ conflict must be resolved

Lamport timestamps: scalar simplification; preserves happens-before but loses concurrent distinction.

OTA (Over-The-Air) Updates

Firmware updates without physical access are essential for deployed IoT fleets.

Key challenges: - Limited flash (often only $2\times$ image slots) - Unreliable radio links — partial update must not brick device - Security — authenticate and verify integrity before applying

Common approach (dual-bank / A/B): 1. Download new image into the inactive bank 2. Verify checksum/signature 3. Set boot flag to new bank 4. Reboot; bootloader switches to new image 5. Application confirms successful boot; otherwise bootloader rolls back

ESP-IDF OTA: uses `esp_https_ota` component; verifies SHA-256 and RSA signature. `esp_ota_mark_app_valid_cancel_rollback()` must be called by the application to confirm the update.

NTP (Network Time Protocol)

Synchronises clocks over a network to UTC.

Hierarchy (stratum): - Stratum 0: atomic clocks, GPS receivers (reference clocks) - Stratum 1: NTP servers directly connected to stratum 0 - Stratum 2: synced from stratum 1; and so on - Practical limit ~ 15 strata

Round-trip delay compensation:

$$\text{offset} = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}, \quad \text{delay} = (T_4 - T_1) - (T_3 - T_2)$$

Where T_1 =client send, T_2 =server receive, T_3 =server send, T_4 =client receive.

Accuracy: ~1-10 ms over the internet; sub-ms on a LAN. Unsuitable for hard real-time — use PTP (IEEE 1588) for s-level precision.

WiFi (802.11)

Standard	Band	Max Rate	Notes
802.11b	2.4 GHz	11 Mbps	DSSS
802.11g	2.4 GHz	54 Mbps	OFDM
802.11n	2.4/5 GHz	600 Mbps	MIMO
802.11ac	5 GHz	6.9 Gbps	MU-MIMO, beamforming
802.11ax	2.4/5/6 GHz	9.6 Gbps	OFDMA, target wake time

2.4 GHz channels (EU): 13 channels spaced 5 MHz, each 22 MHz wide → only channels 1, 6, 11 are non-overlapping.

WPA2 4-way handshake (after 802.1X/PSK authentication):

1. AP → STA: ANonce
2. STA → AP: SNonce + MIC
3. AP → STA: GTK + MIC
4. STA → AP: ACK

Derives the **PTK** (Pairwise Transient Key) from ANonce, SNonce, PMK, and both MAC addresses.

Eduroam uses 802.1X with RADIUS federation; your home institution's RADIUS server proxies authentication.

ESP-WIFI-MESH & ESP-NOW

ESP-WIFI-MESH (Espressif): - Self-organising mesh over standard WiFi (2.4 GHz 802.11)
- One root node bridges to a router; others relay - Uses a tree topology; automatic root election and self-healing - Suitable for smart lighting, large-venue sensor grids

ESP-NOW: - Connectionless peer-to-peer protocol, no router needed - Up to 250-byte payloads; registered peer list (by MAC address) - ~1 ms latency, ideal for remote controls, fast sensor bursts - Security: AES-128 per peer pair

Bluetooth Low Energy (BLE)

BLE separates two roles via **GAP (Generic Access Profile)**: - **Advertiser:** broadcasts packets on channels 37, 38, 39 (2.402, 2.426, 2.480 GHz) - **Scanner/Initiator:** listens on advertising channels, initiates connection

Advertising interval: 20 ms – 10.24 s. Longer interval saves power; shorter improves discoverability.

GATT (Generic Attribute Profile) defines the data model for connected operation: - **Profile** → **Service** (UUID) → **Characteristic** (UUID + value + properties) - Properties: Read, Write, Notify, Indicate - **Notify**: server pushes updates without client polling; no acknowledgement - **Indicate**: like Notify but requires acknowledgement

BLE topology: point-to-point (central/peripheral), broadcast (beacons), or mesh (Bluetooth Mesh spec).

IEEE 802.15.4 & 6LoWPAN

802.15.4 is the PHY/MAC foundation for low-rate WPANs: - 2.4 GHz (global) or 868/915 MHz; 250 kbps at 2.4 GHz - Max frame: 127 bytes - CSMA-CA MAC; optional beacon-enabled superframe (GTS slots) - Used by Zigbee, Thread, WirelessHART

6LoWPAN adapts IPv6 over 802.15.4: - Header compression (40-byte IPv6 header → as few as 2 bytes) - Fragmentation of IPv6 MTU (1280 bytes) into 127-byte frames - Mesh addressing header for multi-hop routing

Zigbee & Z-Wave

Zigbee (802.15.4 + Zigbee stack): - Star, tree, or mesh topologies; coordinator + routers + end devices - Device types: Coordinator (one per network), Router, End Device (sleepy) - Application profiles: Home Automation, Smart Energy, ZigBee 3.0 (unified) - Range: 10–100 m; up to 65,000 nodes per network - 16 channels in 2.4 GHz band; interference with WiFi (see Q6 in exercises)

Z-Wave: - Sub-GHz (868.42 MHz EU, 908.42 MHz US) → avoids 2.4 GHz congestion - Mesh, max 232 nodes - Proprietary (Silicon Labs); interoperability guaranteed within certified devices - Lower data rate (~100 kbps) but better wall penetration than 2.4 GHz

Thread & Matter

Thread: - IPv6 mesh over 802.15.4 using RPL routing - Border Router connects Thread mesh to IP network (via OpenThread Border Router) - No single point of failure: any router-capable device can become Border Router - Used in: Nest, Apple HomePod (mini), Google Home

Matter (formerly CHIP — Connected Home over IP): - Application layer interoperability standard (Apple, Google, Amazon, Samsung) - Runs over Thread (for low-power devices), WiFi, or Ethernet - Uses IPv6, mDNS for discovery, CASE/PASE for commissioning - Single app can control devices from any certified manufacturer

LoRa & LoRaWAN

LoRa (physical layer, proprietary Semtech): - Chirp Spread Spectrum (CSS): frequency sweeps encode data; robust to Doppler and multipath - Parameters: Spreading Factor (SF7-SF12), Bandwidth (125/250/500 kHz), Coding Rate - Higher SF → longer range, slower data rate, more time-on-air

SF	Approx. data rate	Approx. range (rural)
7	~5.5 kbps	~2 km
9	~1.8 kbps	~5 km
12	~0.3 kbps	~15 km

LoRaWAN (MAC + network layer, LoRa Alliance): - Star-of-stars: devices → gateways → Network Server → Application Server - Device classes: - **Class A**: uplink-triggered; two short RX windows after each uplink (lowest power) - **Class B**: scheduled downlink slots via beacon - **Class C**: continuously receiving (highest power) - **OTAA** (Over-the-Air Activation): device joins by exchanging JoinRequest/JoinAccept; derives session keys - **ABP** (Activation By Personalisation): hardcoded session keys (less secure) - Security: AES-128 NwkSKey (network integrity) + AppSKey (payload encryption) - Frame counter prevents replay attacks - Duty cycle: 1% in EU 868 MHz ISM band

The Things Network (TTN): community-run LoRaWAN network; free for low-volume IoT.

Sigfox

- Ultra-Narrowband (UNB) BPSK, 100 Hz bandwidth
 - 868 MHz (EU), 12 bytes uplink / 8 bytes downlink max
 - Very low power; ~140 messages/day limit
 - Proprietary network (Sigfox SA owns the infrastructure)
 - No mesh — pure star topology, centralised decoding across multiple base stations
-

NB-IoT (Narrowband IoT)

- 3GPP standard (Rel-13+); runs on licensed LTE spectrum
- 200 kHz bandwidth in-band, gu

Lecture 9: Stream Processing

Part 0: Anatomy of an IoT Cloud

An IoT cloud ingestion backend deals with massive scale and highly parallelized data flows. Understanding its architectural constraints is critical for system design.

(Diagram — IoT Cloud Architecture)

```
Device Stream 1 → Broker / Ingest Tier
Device Stream 2 → Broker / Ingest Tier
Device Stream N → Broker / Ingest Tier
Broker / Ingest Tier → Proc Node 1
Broker / Ingest Tier → Proc Node 2
Proc Node 1 → Durable DB
Proc Node 2 → Durable DB
Proc Node 1 → Outgoing Stream 1
Proc Node 2 → Outgoing Stream 2
```

Key System Properties

- **Stream Density:** Massive numbers of concurrent incoming data streams.
- **Multi-Tenancy:** Large consumer/customer pools sharing close to the exact same backend processing setups.
- **Privacy Controls:** Strict isolation guarantees are legally/contractually mandated to prevent cross-tenant data leakage.
- **Internal Processing Motivations:**
 - Delivering consumer-facing application features.
 - Real-world evaluation and telemetry tracking of physical product lines.
 - Pattern detection for R&D and future product cycles.
 - System health, diagnostic, and uptime monitoring.
- **Fan-Out Profile:** A relatively small subset of compiled analytical insights are presented to the end customer, alongside a potentially massive volume of secondary outgoing notification/actuation streams.

Part 1: Stream Processing Graphs & Nodes

A **Stream Process** is a self-contained piece of execution logic defined by:

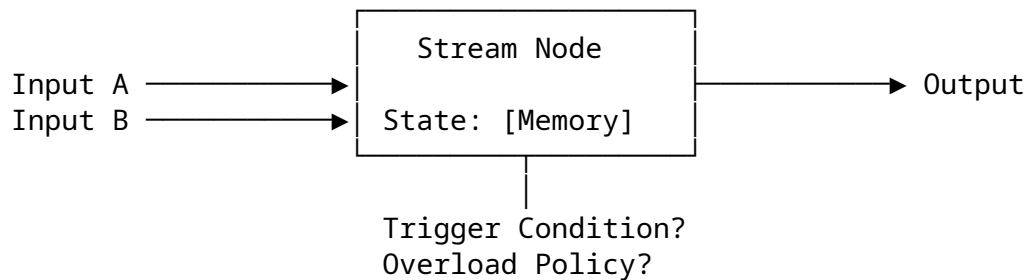
1. **State:** Private, localized memory pool.
2. **Inputs:** Zero or more input streams.
3. **Outputs:** Zero or more output streams.

Special Node Types

- **Source:** A node that only produces output streams (no upstream inputs).

- **Sink:** A node that only consumes input streams (no downstream outputs).

Specific Processing Properties & Edge Cases



- **Overload Policy:** How does the node behave if data is consumed at a lower rate than it is being produced?
 - *Buffering:* Retain incoming items in memory, introducing memory exhaustion risks and latency penalties.
 - *Dropping:* Discard excess incoming items, losing historical completeness to preserve low latency.
- **The COP Theorem:** Under overload conditions, a stream processing system can either guarantee **Completeness** (retaining all data) or **Timeliness** (processing data quickly), but never both:

$$\text{Overload Condition} \implies \text{Completeness} \oplus \text{Timeliness}$$

- **Consumability:** Does processing a message consume and delete it from the stream, or should the message be retained globally? If retained, for how long and for which down-stream observers?
- **Trigger Condition:** If a node consumes more than one input stream, when does it execute its logic?
 - *Barrier Synchronization:* Wait for a matching tuple from all input ports (risks blocking if a value on one stream is dropped).
 - *Latest Values:* Fire immediately using the most recently cached values on all ports.
- **Trigger Arbitrariness:** The ability of a node to fire independently of upstream inputs (e.g., executing on periodic intervals to trigger timeouts or state flushes).
- **Batch Size:** Processing items in groups to amortize execution overhead. This introduces a strict trade-off between throughput and latency:

$$\text{Larger Batch Size} \implies \text{Higher Throughput} \mid \text{Higher Latency}$$

- **Fault Tolerance:** Recovery policy when an execution node unexpectedly crashes (e.g., cold restarts, state recovery from checkpoints, or hot-failover replicas).

Part 2: Concurrency Concerns & remedies

Designing high-throughput stream processing backends requires minimizing standard operating system performance bottlenecks:

1. Context Switching Overhead

Switching CPU execution contexts at the OS level (preempting one process or thread to run another) is incredibly expensive due to cache invalidation and register swapping.

- **Remedy:** Avoid mapping every stream or actor directly to an OS thread. Instead, leverage light-weight user-space cooperative multitasking or scheduler multiplexing.

2. Side-Effects & Cache Misses

Code containing side-effects (mutating shared global states or executing arbitrary I/O) invalidates CPU cache hierarchies, causing severe latency spikes.

- **Remedy:** Keep code functional and pure where possible. Immutable data structures allow trivial, lock-free parallelization with high CPU L1/L2 cache locality.

3. Sharing & Memory Layout

In a shared-nothing architecture, states are fully isolated. Any state update can be resolved locally by a single dedicated node, eliminating expensive distributed database locks.

WARNING: Robert Love's Performance Constant (GUADEC 2005)

"Going to disk is 25 million times slower than hitting a general purpose register.
Design accordingly."

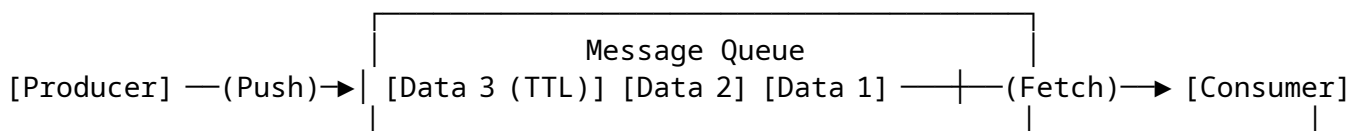
$$t_{\text{disk}} \approx 2.5 \times 10^7 \cdot t_{\text{register}}$$

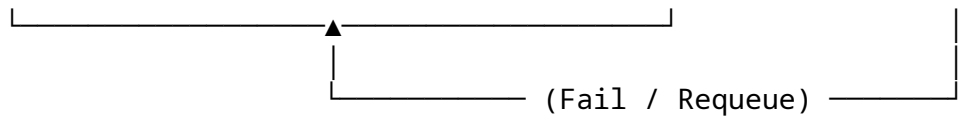
Part 3: Message Queues vs. Event Busses

Depending on the non-functional requirements of your application, choose either a message queue or an event bus architecture:

1. Message Queues (Focus: Completeness)

- **Goal:** Every message must be processed reliably, regardless of whether downstream consumers are currently available.
- **Lifecycle:** Employs a Time-To-Live (TTL) and acknowledgement (ACK) loop. If a consumer fails during a processing step, the message is requeued.
- **IoT Use Case:** Ingestion pipelines for sensor metrics where data loss is unacceptable (e.g., billing, auditing, categorization).





2. Event Busses (Focus: Timeliness)

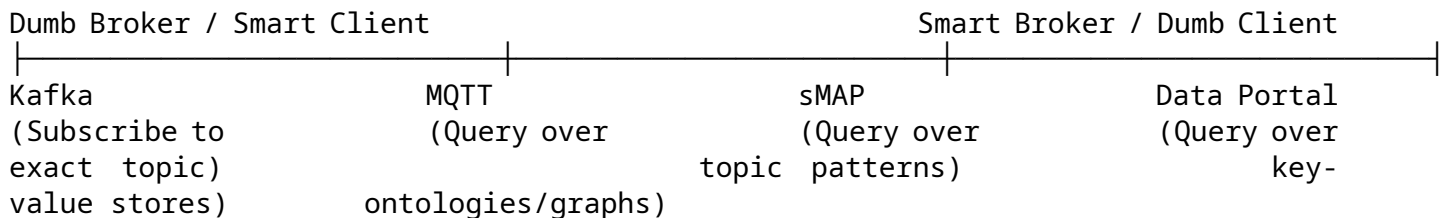
- **Goal:** Real-time updates where low latency is critical, and stale or delayed data is useless.
- **Lifecycle:** Obsolete or delayed values are aggressively dropped. If a consumer is unavailable, it simply misses the transient event window.
- **IoT Use Case:** Streaming clock synchronization, real-time location tracking, and immediate timezone adjustments.

Part 4: The Publish-Subscribe Pattern

The **Publish-Subscribe (Pub/Sub)** pattern decouples producers and consumers through an intermediary message broker.

The Broker Intelligence Spectrum

The division of labor between brokers and clients varies drastically across protocols:



- **Dumb Broker & Smart Client (e.g., Apache Kafka):** The broker acts as a dumb, high-throughput log. The client is smart—tracking its own partition offsets and filtering out unwanted messages locally.
- **MQTT:** Balanced middle ground. Subscriptions are routed via simple wildcard topic patterns matched at the broker.
- **sMAP:** The broker evaluates subscriptions by querying metadata key-value stores associated with individual active streams.
- **Smart Broker & Dumb Client (e.g., “Data Portal”):** The broker contains a complete semantic understanding of the system, evaluating client subscriptions using highly complex ontological or graph queries (e.g., matching physical relationships in a building).

Part 5: MQTT Protocol Specification

MQTT (Message Queuing Telemetry Transport) is a lightweight, binary publish-subscribe protocol designed for constrained IoT edge nodes.

1. Structured Topic Paths & Wildcards

Topics are hierarchical UTF-8 strings separated by forward slashes (/). They are case-sensitive and do not require pre-registration on the broker.

- **Single-Level Wildcard (+):** Matches exactly one topic level.
 - *Example:* `home/+/temperature` matches `home/kitchen/temperature` and `home/bedroom/temperature`, but not `home/kitchen/fridge/temperature`.
- **Multi-Level Wildcard (#):** Matches any remaining trailing topic levels. Must be placed at the very end of the subscription pattern.
 - *Example:* `home/kitchen/#` matches `home/kitchen/temperature`, `home/kitchen/fridge/temperature`, and `home/kitchen/lights/state`.

2. Quality of Service (QoS) Levels

MQTT implements client-to-broker and broker-to-client delivery guarantees:

- **QoS 0 — At Most Once:** Best-effort delivery. Messages are sent without acknowledgements. High performance, but messages can be lost if connections drop.
- **QoS 1 — At Least Once:** Guarantees delivery. Messages are retransmitted until an acknowledgement is received. Can result in duplicate messages at the receiver.
- **QoS 2 — Exactly Once:** Guaranteed, duplicate-free delivery. Utilizes a four-step handshaking protocol (PUBLISH → PUBREC → PUBREL → PUBCOMP). Slowest, but mathematically exact.

3. Advanced Features

- **Last Will and Testament (LWT):** A client registers a backup message and topic with the broker during connection initialization. If the client unexpectedly disconnects (e.g., power loss, wireless link drops), the broker automatically publishes this “Last Will” message to alert downstream services.
- **Persistent Sessions:** The broker buffers offline messages specifically for disconnected clients, delivering them immediately upon reconnection.

Part 6: Distributed Concerns & System Failures

Operating web-scale stream backends requires designing for inevitable hardware decay:

NOTE: The Distributed Scaling Rule

As the number of machines in a distributed system approaches infinity, the Mean Time to Failure (MTTF) of at least one machine approaches zero:

$$\lim_{N_{\text{nodes}} \rightarrow \infty} P(\text{Failure}) = 1$$

Cloud Orchestration Options

Distributed systems utilize two primary architectural deployment strategies:

1. **Heterogeneous & Low-Granularity Deployments:**

- Handled by external orchestration layers that manage container lifecycles, load balancing, and distributed storage.
- *Typical Stack:* **Kubernetes (K8s), Docker Swarm, Ceph** (distributed filesystem).

2. **Homogeneous & High-Granularity Deployments:**

- Handled directly within the programming runtime itself.
- *Typical Stack:* **The BEAM Virtual Machine (Erlang/Elixir)**, leveraging lightweight processes, supervisors, and native node-to-node cluster clustering.
- *Reality:* Large production environments often combine both strategies (e.g., running clustered BEAM nodes inside Kubernetes pods).

Lecture 10: Performance Evaluation

Part 0: Introduction & Systems Overview

Performance evaluation is not a detached post-implementation phase; it is a cross-cutting concern that integrates deeply with every step of the **System Development Life Cycle (SDLC)**.

(Diagram)

```
Requirements → Informs
Analysis → Informs
Design → Informs
Implementation → Informs
```

```
[Verification]
[(Performance<br>Evaluation)]
```

What is a “System”?

For the purposes of this curriculum, a **system** is defined as:

A collection of (mostly) independent components that have been combined in a particular configuration in order to solve a specific problem.

The Problem of Component Availability

In complex IoT environments, physical edge hardware or external dependencies may be missing or under-provisioned during the testing phase.

- **Remedy:** Isolate the subsystem or component of interest. Replace unavailable devices with **stubs** or **test doubles** to emulate raw telemetry and communication interfaces.

Part 1: Experimental Computer Science

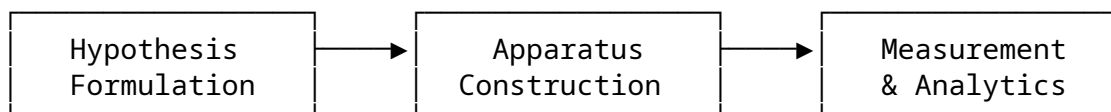
Sensing and networking performance cannot be proved by pure deductive reasoning alone. We must approach non-functional verification empirically.

QUOTE: Peter J. Denning (ACM)

“Science classifies knowledge. Experimental science classifies knowledge derived from observations. The experimenters construct apparatus to measure a given phenomenon; they then attempt to collect data sufficient to support their hypotheses about that phenomenon.”

The Scientific Workflow

To perform true experimental computer science, we follow a rigorous scientific loop:



1. **Hypothesis Formulation:** Stating clear, falsifiable expectations based on non-functional requirements or structural design intuitions.
2. **Apparatus Construction:** Setting up the target system (or an instrumented version of it) to test the hypothesis.
3. **Measurement & Analytics:** Designing tests to extract quantitative data, repeating the process to construct robust statistical distributions.

Hypothesis Examples

- **Good (Falsifiable):**

- *"The system throughput gracefully declines linearly as the number of concurrent connections outnumbers physical CPU cores ($N_{\text{connections}} > N_{\text{cores}}$)."*
- *"The proposed network topology between Odense and Copenhagen limits the packet loss of UDP streams to $\leq 7\%$ over a typical 24-hour cycle."*

- **Bad (Unfalsifiable / Biased):**

- *"The introduction of the Internet will increase the overall productivity of programmers."* (Too vague to isolate parameters or define a metric).
- *"Show that System A is faster than System B."* (Inherently biased toward confirming a predetermined outcome).

WARNING: Repeatability & Reproducibility

For experimental conclusions to be scientifically trustworthy, they **must be repeatable**. This requires:

1. Comprehensive documentation of all environment parameters.
2. Sufficient test iterations to establish statistical significance, smoothing out transient scheduler or network noise.

Part 2: Performance Evaluation Methodology

Systematic performance testing requires carefully isolating variables.

1. Goals

Define your target question explicitly before running tests. Clearly defined goals simplify experimental design and protect the team from confirmation bias.

2. Services

A **service** is any functional outcome provided by the system (e.g., *routing a packet to a gateway, committing a timeseries sample to a database, or firing an actuation trigger on a sensor threshold*).

- **Desirable Outcome:** The service completes successfully within acceptable bounds (e.g., low-latency write).
- **Undesirable Outcome:** The service fails, times out, or delivers corrupted results (e.g., dropped packets).

3. Metrics

Metrics are the concrete criteria used to quantify and evaluate service performance:

- **Latency/Delay:** Time elapsed between a physical event and its digital visualization ($t_{\text{render}} - t_{\text{event}}$).
- **Memory Footprint:** Volatile memory (RAM) allocated to a specific software daemon.
- **Throughput:** Maximum sustained frequency of operations (e.g., DB commits per second).

NOTE: SI vs. Binary Prefixes

Ensure precise unit formatting. While standard metric prefix calculations use SI bases (1 kB = 10^3 bytes), data and address alignments often dictate binary notations (1 KiB = 2^{10} bytes = 1024 bytes).

4. Parameters vs. Factors

- **Parameters:** Static configurations kept constant across all test iterations (e.g., fixed buffer sizes, predefined timeouts, retry limits).
- **Factors:** The independent variables we actively modify to study their impact on the system metrics:

$$\text{Factor Variation} \implies f(\text{Metric Output})$$

If checking the interactions of multiple factors simultaneously (a **full-factorial experiment**), the complexity scales exponentially (2^k configurations for k binary factors). When this is computationally or temporally prohibitive, we utilize heuristic optimizations like **hill-climbing** (varying one factor at a time).

5. Workload

The predefined sequence of inputs fed into the system during testing.

- **Synthetic Workloads:** Generated inputs (e.g., generating 100 fake MQTT payloads per second).
- **Replayed Workloads:** Replaying historical traces from real production databases to capture realistic noise and bursts.

Part 3: The Experimental Testing Pipeline

To execute tests reliably without manually configuring every run, build a structured **Test Harness**:

(Diagram)

```
Factors → Testcase Generator
Testcase Generator → Parameter Mapping
Testcase Generator → Workload Generator
Workload Generator → System Implementation
Parameter Mapping → System Implementation
```

System Implementation → Instrumentation
 Instrumentation → Logfiles
 Logfiles → Data Analysis
 Data Analysis → Visualization

The Clock Alignment Problem in Distributed Systems

Measuring time across multiple IoT nodes introduces massive experimental traps:

- **Single-Machine Timing:** Reliable. Two timestamps captured on a single machine are directly comparable, provided we use **monotonic clocks** ($t_{\text{monotonic}}$) to prevent clock warping from Network Time Protocol (NTP) adjustments.
- **Multi-Machine Timing:** Highly unreliable. Clocks on separate machines drift due to thermal variance and hardware tolerances. In distributed environments, you must either sync clocks with sub-millisecond precision or mathematically quantify and bound the temporal uncertainty:

$$\Delta t_{\text{error}} = |t_A - t_B| \pm \epsilon_{\text{drift}}$$

Part 4: Data Visualization

Once log data is collected, visualization is the fastest way to perform initial sanity checks and find anomalies.

Common Plot Types and When to Use Them

Visualization Type	Best Use Case	Avoid When...
Scatter Plot	Identifying correlation patterns and timeseries progressions	Ingesting massive over-plotted dense datasets without transparency
Histogram	Visualizing raw sample distributions and bin frequencies	Data is highly sparse or bin sizes are chosen arbitrarily
KDE Plot	Smoothing histogram distributions to view density contours	Working with hard-bounded data (can create artificial tails)
CDF Plot	Easily reading exact percentiles (e.g., 95th percentile latency)	Visualizing multimodal or highly split distributions
Violin Plot	Comparing factor-dependent distributions side-by-side	Sample sizes are too small to construct density kernels

Python Visualization Template (pandas & seaborn)

```
import pandas as pd
import seaborn as sns
```

```

import matplotlib.pyplot as plt

# 1. Load telemetry dataset
df = pd.read_csv('dkbirths2019.csv', names=['day', 'birth', 'weekday'])

# 2. Configure Seaborn theme for scientific publication
sns.set_theme(style="ticks")
plt.figure(figsize=(8, 5))

# 3. Generate Violin Plot to evaluate factor-dependent distributions
sns.violinplot(data=df, x='weekday', y='birth', color="lightsteelblue")

# 4. Enforce strict, self-contained plot standards
plt.xlabel('Weekday [ID]', fontsize=11)
plt.ylabel('Births [#]', fontsize=11)
plt.title('Daily Birth Distributions by Weekday (Denmark, 2019)', fontsize=12)

# Ensure axes start at zero unless plotting logarithmic or absolute epoch time
plt.ylim(bottom=0)

# Save plot to vector format to maintain scaling resolution
plt.tight_layout()
plt.savefig('violin_distribution.pdf')

```

Strict Plot Standards Checklist

For any chart submitted in academic projects or system reports, ensure:

- **Self-Containment:** The plot context must be immediately understandable.
- **Strict Labeling:** Every axis must be

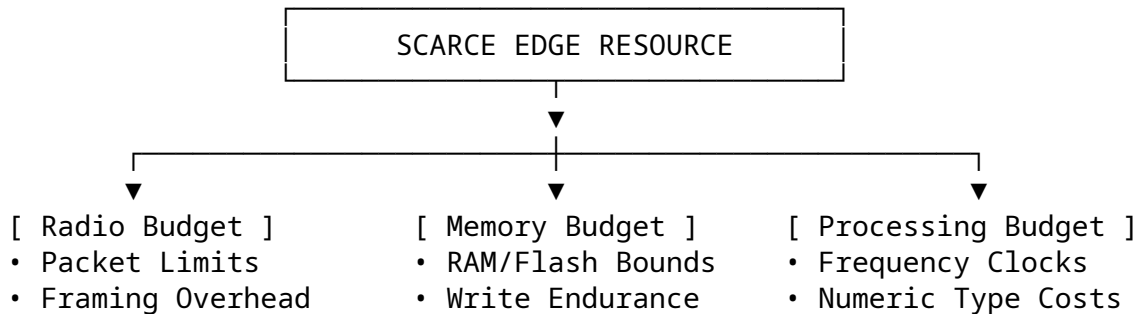
Lecture 11: Budget Management

Part 1: The Core Philosophy of Budgets

In resource-constrained cyber-physical systems, a **budget** represents the absolute physical or structural limit of an operational environment.

NOTE: Academic Definition

A budget represents the availability of a scarce resource. A budget can be consumed, and depending on the system's environment, some budgets can be replenished at a consistent or stochastic rate. Budgets are defining constraints for the operational lifetime and reliability of an IoT device.



The Four Major Budget Domains

1. Radio Budget Wireless communication is often the most restricted and energy-expensive capability on an edge node.

- **Structural Restrictions:**

- Maximum number of packets sent over a defined timeline (e.g., daily limits due to battery or regulatory duty cycle rules).
- Rate of transmission (burst limits vs. cooling periods).
- Maximum payload packet size (MTU limitations).

- **Network Overhead Consumers:**

- **Data Collection:** Raw sensor payload transfers.
- **Control Plane:** Actuation requests and over-the-air (OTA) configuration distribution.
- **Topology Maintenance:** Routing table beacons, neighbor discovery, and network keep-alives.
- **Transport Safety:** Packet retransmissions and acknowledgment frames.

2. Memory Budget Memory limits dictate what software paradigms, operating systems, and local storage mechanisms can be deployed on the edge.

- **Structural Restrictions:**

- **RAM Size:** Limits active execution state, concurrent thread stacks, dynamic heap allocations, and buffer queues.

- **ROM / Internal Flash Size:** Restricts the footprint of the compiled program binary (Code Segment).
- **External Flash / Storage Size:** Limits the capacity for local database logging.
- **Flash Endurance:** Flash memory cannot be written to indefinitely; cells degrade after a certain number of write/erase cycles (typically 10,000 to 100,000 cycles).
- **Erase Unit Size:** Flash must be erased in large blocks (sectors/pages) before individual bytes can be rewritten, introducing a heavy power and time penalty.
- **Architectural Consequences:** Directly influences the trade-off of *local computation vs. immediate transmission* (e.g., compress locally and save to flash, or stream immediately?).

3. Processing Budget Processing constraints determine how much analytical logic can be pushed down to the edge (Edge AI vs. Cloud computing).

- **Structural Restrictions:**
 - **Concurrency Bounds:** Number of physical processor cores, availability of DMA (Direct Memory Access) channels, and dedicated crypto hardware accelerators.
 - **Clock Speed / Frequency:** Limits execution throughput (I instructions per second).
 - **Numeric Processing Unit Limitations:** Low-end microcontrollers (like many ARM Cortex-M0 or ESP32 variants depending on configurations) lack hardwired double-precision Floating Point Units (FPUs).
- **Type Execution Speed Penalties:**
 - **Integers:** Fast, executed in a single clock cycle.
 - **Single-Precision Floats:** Fast on FPUs, slow when emulated in software.
 - **Double-Precision Floats:** Extremely expensive on low-end edge architectures, requiring massive software translation library routines.

4. Energy Budget The absolute binding constraint in modern IoT. If the energy budget is exhausted, all other budgets (processing, memory, radio) immediately drop to zero.

- **Structural Restrictions:** Bounded battery capacity (E_{battery}) or variable environmental replenishment rates (P_{harvest}).

Part 2: Energy Calculations & Power Profiles

Math Foundations

Electrical power (P) is the instantaneous rate at which energy is consumed:

$$P = V \cdot I$$

Where:

- P is Power in Watts (W).

- V is Voltage in Volts (V).
- I is Current in Amperes (A).

Energy (E) is the accumulation of power consumed over a given time frame (t):

$$E = P \cdot t = V \cdot I \cdot t$$

The Active vs. Sleep Duty Cycle Model

To survive years on a small battery, edge nodes operate under strict **duty cycling**: spending the majority of their lifecycle in a low-power deep-sleep mode, waking up briefly to sample and transmit before returning to sleep.

The average power consumption (P_{avg}) is modeled as:

$$P_{\text{avg}} = D \cdot P_{\text{active}} + (1 - D) \cdot P_{\text{sleep}}$$

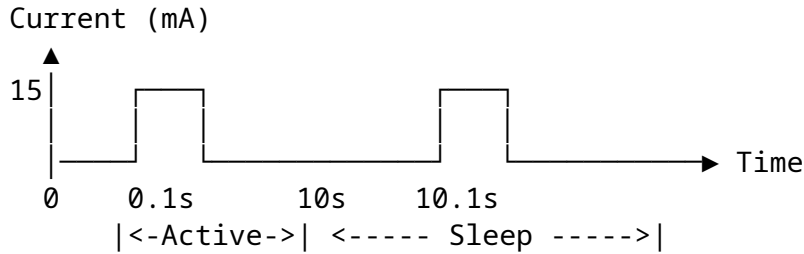
Where D is the duty cycle ratio:

$$D = \frac{t_{\text{active}}}{t_{\text{active}} + t_{\text{sleep}}}$$

Step-by-Step IoT Node Lifetime Calculation

Problem Scenario: An environmental sensor node is powered by a standard **CR2032 coin cell** (3.0 V nominal voltage, 220 mAh total usable capacity).

- **Active State:** Consumes 15 mA for 100 ms once every 10 seconds.
- **Sleep State:** Consumes 15 μA for the remaining 9.9 seconds.



1. Calculate the Duty Cycle (D): The total period $T = t_{\text{active}} + t_{\text{sleep}} = 0.1 \text{ s} + 9.9 \text{ s} = 10 \text{ s}$.

$$D = \frac{0.1 \text{ s}}{10 \text{ s}} = 0.01 \quad (1\% \text{ active duty cycle})$$

2. Calculate the Average Current Draw (I_{avg}): Convert all values to Amperes to avoid scale errors:

- $I_{\text{active}} = 15 \text{ mA} = 0.015 \text{ A}$

- $I_{\text{sleep}} = 15 \mu\text{A} = 0.000015 \text{ A}$

$$I_{\text{avg}} = (0.01 \cdot 0.015 \text{ A}) + (0.99 \cdot 0.000015 \text{ A})$$

$$I_{\text{avg}} = 0.00015 \text{ A} + 0.00001485 \text{ A} = 0.00016485 \text{ A} \approx 164.85 \mu\text{A}$$

3. Calculate Projected System Lifetime (t_{lifetime}): Using the battery capacity of 220 mAh:

$$t_{\text{lifetime}} = \frac{\text{Capacity}}{I_{\text{avg}}} = \frac{220 \text{ mAh}}{0.16485 \text{ mA}} \approx 1334.5 \text{ hours}$$

$$t_{\text{lifetime}} \approx 55.6 \text{ days}$$

Part 3: Power Throttling & Hardware Optimization

To maximize the energy budget, modern hardware architectures employ advanced throttling and isolation mechanisms.

Sleep State Latency Trade-offs

Microcontrollers offer multiple low-power states, but dropping to lower states introduces a wake-up time penalty:

State	Typical Current Draw	Internal Clock Status	Peripherals Enabled	Wake-up Latency
Active	10 – 50 mA	Running at Full Speed	All enabled	0 s (N/A)
Idle / Sleep	1 – 5 mA	Stopped core, fast peripherals	Timers, DMA, Interrupts	$\approx 1 - 5 \mu\text{s}$
Deep Sleep	1 – 50 μA	All internal oscillators stopped	Real-Time Clock (RTC) only	$\approx 1 - 10 \text{ ms}$

Direct Memory Access (DMA) Offloading

In standard operating modes, transferring data from a peripheral (e.g., ADC or UART) to RAM requires the CPU to wake up, load the instructions, and step through the Program Counter (PC).

TIP: The DMA Advantage

Running a microcontroller peripheral in **DMA mode** allows data to bypass the central register pipelines, writing data directly to RAM **without spinning the CPU Program Counter**. This allows the main CPU core to remain in an ultra-low-power sleep state while data streams in from edge sensors.

Abstraction & Power Managers

Operating systems designed for low-power operations (such as **TinyOS**) incorporated automated resource-tracking power managers.

- **Mechanism:** The OS tracks the exact power and clock speed requirements of every active peripheral and software driver.
- **Dynamic Adjustments:** Rather than relying on the programmer to manually toggle sleep modes, the OS automatically throttles down or gates off the clock domain to match the highest requirement of the currently executing thread block.

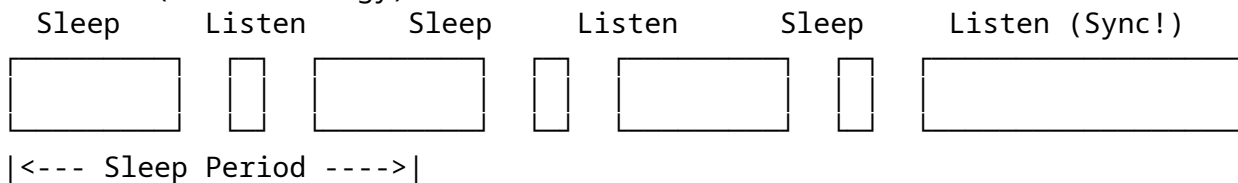
Part 4: Protocol Throttling: Low-Power Listening (LPL)

Running a radio receiver (RX mode) constantly to listen for incoming packets often consumes as much (or more) energy as active transmitting (TX mode). To solve this, wireless MAC protocols employ **Low-Power Listening (LPL)**.

The LPL Mechanism

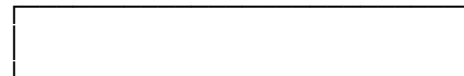
Instead of keeping the radio receiver powered on, the receiver sleeps and only wakes up periodically for a very brief interval to check the channel for radio frequency activity.

RECEIVER (Scarce Energy)



TRANSMITTER (Plentiful Energy)

TX Preamble (Matches Period)



1. **Receiver Behavior:** Listens periodically for activity on the frequency band. If the channel is clear, it immediately sleeps. If RF energy is detected, it stays awake to synchronize and parse the incoming frame.
2. **Transmitter Behavior:** Must guarantee the receiver is awake to hear the transmission. The transmitter prepend its packet with a **preamble whose length matches or exceeds the receiver's sleep period**.

Trade-offs & Optimal Scenarios

- **The Cost Transfer:** Low-Power Listening saves massive amounts of receiver energy by pushing the energy cost onto the transmitter (which must send a long, expensive preamble).
- **LPL is Highly Optimal When:**
 - Receivers operate on highly constrained energy budgets.

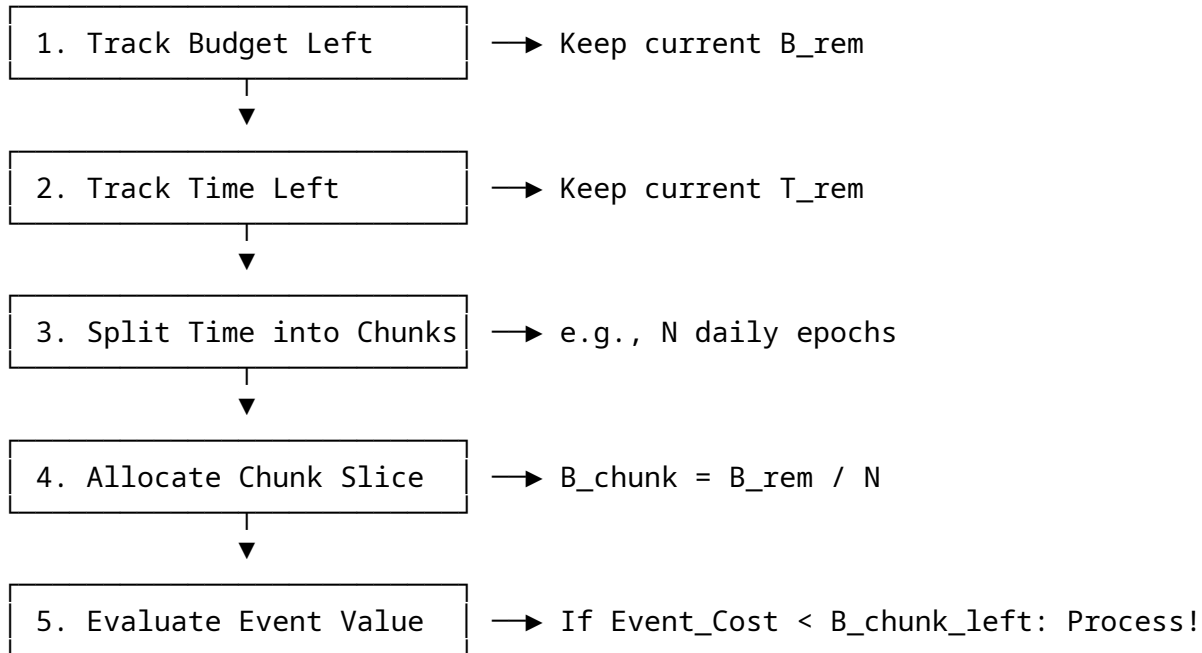
- Wireless transmissions are rare or highly infrequent.
- Transmitters have access to line power (plentiful energy).

Part 5: Algorithms for Active Budget Management

When deploying nodes in environments where energy harvesting rates vary (e.g., solar-powered nodes), devices must dynamically manage their budgets to prevent total depletion.

The Generic 5-Step Budget Allocation Algorithm

To ensure continuous operation without crossing the depletion threshold, you can model active budget management dynamically:



1. **Keep track of the remaining budget** (B_{rem}).
2. **Keep track of the remaining time** (T_{rem}) until the next guaranteed replenishment cycle (or end of system lifetime).
3. **Split the remaining time into sensible operational chunks** (e.g., N intervals, such as 24-hour cycles).
4. **Allocate a slice of the remaining budget to this current chunk:**

$$B_{chunk} = \frac{B_{rem}}{N}$$

5. **Evaluate whether an event is worth processing:** Check if the energy or resource cost of processing the next incoming signal falls within the remaining budget slice allocated to the current chunk. If the current chunk's slice is close to exhaustion, turn off non-essential telemetry features and sleep until the next epoch.

Part 6: Resource Trade-off Matrix

When designing an IoT application, keep the golden rule of embedded systems in mind: **Computation is cheap; Wireless transmission is expensive.**

To optimize your resource budget, always favor local data compression, filtering, and pattern detection over streaming raw sensor readings.

Operational Action	Representative Hardware Metric	Relative Energy Cost Index	Architectural Optimization
MCU Instruction	1 CPU Clock Cycle (16 MHz)	$1\times$ (Nanojoules)	Write tight C code; use compiler optimizations (-O2).
Internal Flash Read	Reading a 32-bit word	$10\times$	Run performance-critical loops from RAM if possible.
Flash Write/Erase	Storing a data telemetry block	$100\times -1,000\times$	Minimize writing frequency; use RAM ring buffers.
Sensor Sampling	Shifting data from an I2C sensor	$1,000\times$	Optimize sampling frequency (f_s) dynamically.
Radio Transmission	Transmitting a framed payload	$100,000\times -1,000,000\times$ (Millijoules)	Compress locally; implement threshold-based updates.

Lecture 12: Annotating Identity, Time and Space

Part 1: Naming

Naming is one of the most fundamental (yet deceptively simple) concepts in computer science. It is **not** about assigning a label to a concept for readability — it is about the ability to **reference** something.

name + context → data

Example 1: `['/home/asjo', '.bashrc']` → `/home/asjo/.bashrc`

Example 2: `['myObject', 'print()']` → `myObject.print()`

Every identity, timestamp, and position annotation in an IoT system is ultimately an act of naming: binding a symbolic reference to a physical or logical entity.

Part 2: Identity

What Is an Identity?

An identity is a **name** for a device. We need to be able to name a device in order to give its associated datastreams meaning:

- **Associate a device to something:** e.g., device 12 produces data on stream 17.
- **Associate something to a device:** e.g., room U181 contains device 12.

NOTE: Multiple Identities A single device may carry **multiple identities** serving different purposes and different levels of exposure (e.g., a hardware serial number for provisioning, a logical UUID for the application layer, and a human-readable label for the dashboard).

Sources of Identity

#	Source	Example
1	Unique device number	Serial from an ID chip, the MCU, or a radio module
2	User-provided	Entered via mobile app during physical installation
3	Physical location-derived	A GPS coordinate set hardcoded at deployment
4	Network-derived (logical location)	DNS name, or enumerated from network topology
5	Context-derived	Combination of what it is, where it is, and what happens around it

Point of Identification: Where Is Identity Rooted?

The identity can live in three places, each with different trust and access implications:

On the Node (Self-aware)	Cloud (Centralized)	Private (User-held)
Location	Description	Use Case
Node	The device describes itself.	Offline deployments, secure enclaves
Cloud	A central registry holds identity.	Large managed fleets
Private	Stored with the end user.	Privacy-preserving / zero-knowledge setups

Key questions for any identity design: - Who knows the identity? - Who can retrieve it, and at what cost?

TIP: Storing identity on the node itself can be surprisingly useful — e.g., a device can prove its own identity without contacting the cloud, enabling offline provisioning and zero-touch enrollment.

Part 3: Time

Keeping Track of Time

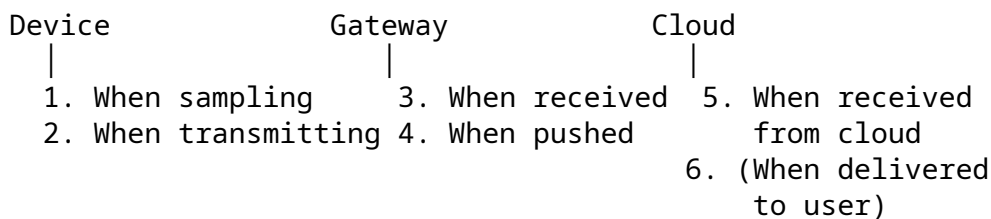
While running, a device is essentially **counting clock ticks**. This counter can be converted to an **uptime** (time since boot).

When a timestamp from a node is received over radio, it is derived from the value of such a counter on the originating device. By **propagating timestamps** across nodes, the network can maintain some degree of a shared sense of time.

Further reading: - Vector clocks (distributed logical time) - Network Time Protocol (NTP)

Point of Timestamping

Timestamps can be applied at many stages in the pipeline:



The **choice of timestamping point** has major consequences for accuracy and latency budgets: - Stamping on the **device** is most accurate, but requires the device to have a synchronized clock. - Stamping at the **gateway** or **cloud** introduces variable network delays.

Sources of Time

On the device:

Source	Notes
Offset (relative time)	Just a counter since boot — no wall-clock knowledge
GNSS (GPS/GLONASS/Galileo/BeiDou)	Accurate, but power-hungry
USB cable on startup	Host injects current time during flashing/debug
Downstream push	Gateway pushes current time to the device
In-channel broadcast	Time announced within the normal data channel
Backchannel broadcast	Dedicated time-sync channel
NTP	Standard internet time protocol

INFO: Why So Many GNSS Systems? **Selective Availability** — governments can degrade or deny GPS accuracy in conflict zones (e.g., during the Kargil War in 1999, the US degraded civilian GPS). Multiple independent GNSS constellations (EU's Galileo, Russia's GLONASS, China's BeiDou) reduce geopolitical dependency.

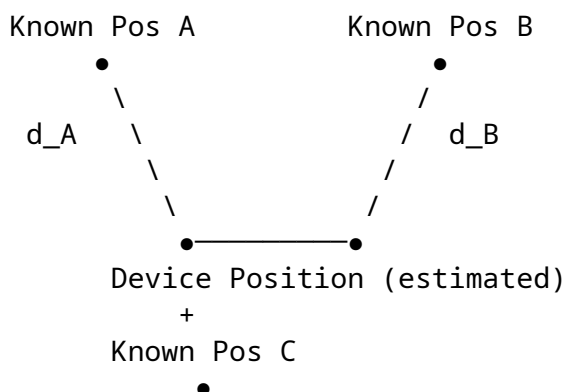
Part 4: Space

Triangulation of Position

Given enough **distances to known positions** with small enough error margins, the actual position of a device can be narrowed down to a reasonable degree of certainty.

- When distances are **exact**: the calculation is trivial geometry.
- When distances are **measured** (noisy): it becomes an optimization problem of minimizing errors (least-squares fitting).

This process can happen **on the device** (low latency, requires local compute) or **in the cloud** (easier to update, requires connectivity).



d_C |

Sources of Position

Direct sources (device knows its own position):

Source	Notes
Hardcoded in firmware or flash	Set at manufacture or deployment; never changes
GPS/GLONASS/Galileo/BeiDou	Accurate, autonomous, but power-intensive
Downstream push	Operator manually sets position via configuration
USB cable on startup	Position injected from host during provisioning

Indirect sources (position inferred via triangulation):

- **Signal strengths from known neighbors** — e.g., using RSSI from mesh neighbors at known coordinates to triangulate position.
- **WiFi / BLE fingerprinting** — matching signal strength patterns against a pre-built map.

WARNING: Indirect Positioning Accuracy Signal-strength-based triangulation is significantly less accurate than GNSS, especially indoors. Multipath reflections (see Lecture 7) introduce large error margins. It is best used for coarse room-level or zone-level presence detection rather than precise coordinates.

Summary: The Three Annotation Axes

Every IoT datastream needs to be annotated along three dimensions to be interpretable:

Dimension	Core Question	Key Challenge
Identity	<i>Which device produced this?</i>	Where is identity rooted? Who can access it?
Time	<i>When was this measured?</i>	Where in the pipeline is the timestamp applied? Clock sync.
Space	<i>Where was this measured?</i>	Is position known directly or triangulated? Accuracy vs. cost.

Together, these annotations transform raw data into **information** (see the Data → Wisdom h

Solutions — Exercises 1: Introduction

INFO: These are model answers. Your own phrasing and examples may differ — focus on whether the *reasoning* aligns.

Q1: IoT Definition

Buzzword definition: IoT = any physical object connected to the internet. Everything from a fridge to a city street lamp is “IoT.” The implication is that connectivity alone makes a device “smart” and inherently valuable.

Technical/course definition: IoT is a *system* that spans multiple tiers (device → fog → cloud) and disciplines (hardware, networking, data management, application logic). The focus is on *sensing the physical world, moving that data reliably through heterogeneous networks, and converting it into actionable information.*

Why the distinction matters for system design: The buzzword framing tempts designers to slap WiFi on a device and call it done. The technical framing forces you to ask: - Where does processing happen? (edge vs. fog vs. cloud) - What are the latency requirements? - How much bandwidth does the raw data consume? - What happens when connectivity is lost?

Ignoring these questions produces systems that generate enormous volumes of useless raw data, saturate networks, or fail catastrophically when the cloud is unavailable.

Q2: The 6 Domains — Smart Building Example

Domain	Example
Data Collection	DS18B20 temperature sensor on each floor reading every 30 s; CO ₂ /PIR occupancy sensors
Data Transportation	Sensors publish over MQTT to a local broker via WiFi; broker forwards to cloud over TLS/TCP
Information/Metadata Management	RDF graph maps sensor IDs to rooms, floors, and buildings; Brick Schema describes relationships
Data Processing	Moving average smooths noisy temperature readings; occupancy model infers whether a room is in use
Decision Support	Dashboard shows per-zone temperature vs. setpoint; alert fires when CO ₂ > 1000 ppm
Actuation	HVAC controller adjusts valve position via Modbus; relay toggles heating element on/off

NOTE: The pipeline is strictly ordered: you cannot actuate without processing, cannot process without transportation, and cannot transport without collection.

Q3: Big Data Consequences

Concrete numbers: - 10,000 buildings $\times$ 200 sensors $\times$ 1 Hz $\times$ ~8 bytes per reading = **16 MB/s** continuous ingest - Per day: ~1.4 TB. Per year: ~500 TB. Over 10 years: ~**5 PB** of raw data. - Processing 1 Hz data from 2 million sensors in real time requires a horizontally-scaled stream processing platform (Kafka + Flink/Spark Streaming).

For the decision:

In favour of “collect everything”: - You cannot retroactively go back and collect data you chose not to store. - ML models may reveal patterns that are not yet known to be relevant (e.g. subtle thermal drift predicting HVAC failure months in advance). - Storage is cheap relative to hardware deployment costs.

Against: - “Patterns might emerge” is not a business requirement — it is data hoarding masquerading as strategy. - Raw 1 Hz sensor data is mostly noise and drift; the useful signal is in aggregates (5-min average, daily peak). - GDPR mandates data minimisation: you may not store personal data longer than necessary for its stated purpose. - Operational cost (bandwidth, ingestion, query) scales with data volume, not just storage.

Better architectural decision: store raw data at high resolution for a short window (e.g. 24 hours) for debugging; downsample to 5-minute averages for 90 days; store only daily statistics for long-term analytics.

Q4: Data → Information → Knowledge → Wisdom

Starting from: 42xhumidityxroom_U181x0.73

Data (raw): 0.73 — a dimensionless number. No context, no unit, no meaning.

Information (data + context): “Room U181 has a relative humidity of 73% as measured by sensor 42 at 14:32 on 2026-02-03.” Context added: unit, timestamp, location, source sensor.

Knowledge (information + pattern/rule): “Room U181 has been above 70% RH for the past 6 hours. According to building standards, sustained RH > 70% promotes mould growth and violates comfort guidelines.” Context added: historical trend, domain rule, threshold.

Wisdom (knowledge + judgment/action): “Room U181’s humidity is critically high due to a faulty dehumidifier unit that was last serviced 18 months ago. Given the current occupancy schedule, the optimal action is to increase ventilation now and schedule maintenance within 48 hours rather than triggering an emergency shutdown that would disrupt 3 ongoing meetings.” Context added: root cause, business context, trade-off reasoning, timing of action.

Q5: Computation Tier Trade-offs

Scenario	Tier	Justification
Detecting a fire alarm threshold breach	Edge	Latency is life-critical (milliseconds matter). Must function even if network is down. Minimal computation: single threshold comparison.
Training a long-term occupancy prediction model	Cloud	Requires months of historical data from many buildings. Needs GPU/distributed compute. No real-time latency requirement.
Aggregating energy usage across 500 buildings	Fog	Needs data from many sites (not local to one device) but doesn't need full cloud — a regional aggregator with moderate compute is sufficient. Can reduce bandwidth by pre-aggregating before sending to cloud.
Real-time control of an industrial robot arm	Edge	Hard real-time (<1 ms control loop). Any cloud round-trip would violate timing. Safety-critical: must not depend on network availability.

Q6: GDPR Implications

When does it become subject to GDPR? Motion/occupancy sensor data becomes personal data under GDPR the moment it can be used — alone or in combination with other data — to identify a natural person. In a typical office: - If a sensor records “desk 14B occupied 09:00–17:30” and desk 14B is assigned to Alice, that is personal data. - Even without explicit ID assignment, if the building only has one desk per person, the occupancy record is personally identifiable. - Pseudonymisation (replacing names with IDs) reduces but does not eliminate GDPR obligations.

Compliance steps: 1. **Legal basis:** establish a lawful basis (legitimate interest, or explicit consent if used for performance monitoring). 2. **Data minimisation:** store aggregated occupancy counts (room-level), not individual desk-level time series, unless operationally required. 3. **Purpose limitation:** only use the data for the stated purpose (e.g. HVAC optimisation), not HR surveillance. 4. **Retention policy:** delete or anonymise records after the retention period. 5. **Transparency:** inform employees (privacy notice) that occupancy data is collected. 6. **DPIA:** conduct a Data Protection Impact Assessment if large-scale systematic monitoring of individuals in a workplace.

Q7: Programming Language Choices

Why Java is unsuitable at the Edge: The JVM (Java Virtual Machine) imposes: 1. **Garbage collector pauses (GC stop-the-world):** the GC can pause all threads for tens to hundreds of milliseconds unpredictably. This violates real-time requirements

(e.g. sampling at exactly 1 Hz, or reacting to a sensor event within 5 ms). 2. **Large runtime footprint:** JVM startup requires ~50–200 MB of RAM. A typical microcontroller (ESP32, STM32) has 256–520 KB of RAM — three orders of magnitude less. 3. **Startup time:** JVM cold-start takes seconds, unacceptable for devices that may need to boot and react within milliseconds.

Why it is acceptable at the Cloud layer: Cloud servers have gigabytes of RAM, multi-core CPUs, and no hard real-time deadlines. GC pauses are amortised across request latency budgets (e.g. 100 ms SLA). Java’s mature ecosystem (Spring, Kafka clients, Hibernate) and excellent tooling make it a productive choice for backend services.

NOTE: Some JVM variants (Azul Zing, GraalVM native image) mitigate GC pauses, but are still not suitable for bare-metal MCU deployment.

Q8: Intelligent vs. Smart

Smart device: reacts to predefined rules or remote commands. A smart bulb can be turned on/off by an app, scheduled, or set to a colour — but only because a human (or a fixed rule) told it to. It does not reason about *why* or *whether* it should act.

Intelligent device: can reason, adapt, and learn autonomously. It would model user behaviour, infer preferences from context, and update its behaviour without explicit reprogramming.

Smart bulb analysis: - It does not reason (no inference engine). - It does not adapt to new contexts without being explicitly reconfigured. - It does not learn (no model updated from experience). - It is “smart” purely in the marketing sense: it has a network interface and programmable state.

What would make it genuinely intelligent: - An on-device or fog-level model trained on past occupancy/preference data. - Ability to infer: “it is 07:00, the user just entered the kitchen, therefore bright warm light is preferred” without a rule for that exact combination. - Contextual awareness: dim automatically when the TV is on, without being told. - In practice: this moves the intelligence to the fog/cloud tier, which aggregates context the bulb cannot observe alone (calendar, location, other devices).

Solutions — Exercises 2: Drinking the Elixir

INFO: Quests 0-2 are setup/exploration tasks with no single correct answer. Solutions here focus on the *questions* embedded in each quest, plus complete code for Quest 8.

Quest 0: Install Livebook

No answer required — this is a setup task. Verify success by navigating to <http://localhost:4000/> and seeing the Livebook dashboard.

Quest 1: Kickstart

Module 1 — Values, Tuples and Pattern Matching

1. `t = {1,2}` vs `{a,b} = {1,2}` Both use the `=` match operator. `t = {1,2}` binds the whole tuple to `t`. `{a,b} = {1,2}` deconstructs: `a` is bound to 1, `b` to 2. The left-hand side is a pattern, not a target variable.

2. Why does `_y` produce a warning? Variables starting with `_` are conventionally “ignored.” Elixir treats `_y` as intentionally unbound: it won’t complain if you don’t use it, but if you try to *read* it, Elixir warns that it was never assigned a meaningful value in the pattern. It behaves like `_` — bound but deliberately unnamed.

3. Circle exercise:

```
d = 30.9
r = d / 2          # → 15.45
circumference = 2 * :math.pi * r    # → 97.07...
```

Module 2 — Booleans, if, string and case

1. `rem(n, 2)` for even/odd: `rem(n, 2)` returns 0 for even numbers and 1 for odd numbers (it’s integer remainder). The birthday exercise uses `rem(participant_count, 2) == 0` to determine if slicing is easy.

2. `r = -3` in the area if: The `if` returns `{:error, :negative_radius}`. Pattern matching `{:success, area} = {:error, :negative_radius}` then raises a `MatchError` because `:success` does not match `:error`. This is intentional — the code propagates an error via the return value, and the caller must handle it.

3. if as case:

```
# if version:
if easy do
  IO.puts("It is easy")
else
  IO.puts("It is hard")
```



```

end

# case version:
case easy do
  true  -> IO.puts("It is easy")
  false -> IO.puts("It is hard")
end

```

Module 3 — Functions

1. Named vs anonymous call syntax: Named functions are resolved at compile time by the module system — no extra indirection needed. Anonymous functions are values stored in variables; the `.` distinguishes a call on a *function value* from a call on a *named function*, preventing ambiguity (e.g. `f(x)` could mean “apply module function `f` to `x`”, whereas `f.(x)` unambiguously means “call the function stored in variable `f`”).

2. Circumference with labelled axes:

```

circumference = fn r -> 2 * :math.pi * r end

data_circumference =
  1..50
  |> Enum.map(fn r -> %{ "radius" => r, "circumference" => circumference.(r) } end)

alias VegaLite, as: Vl
Vl.new(width: 400, height: 300)
|> Vl.data_from_values(data_circumference)
|> Vl.mark(:point)
|> Vl.encode_field(:x, "radius", type: :quantitative, title: "Radius (units)")
|> Vl.encode_field(:y, "circumference", type: :quantitative, title: "Circumference (units)")

```

3. Axis titles in VegaLite docs: The `title:` option on `Vl.encode_field/4` — found in the VegaLite hexdocs under `encode_field/4` → `options` → `:title`.

Module 4 — Lists, Higher-Order Functions, Pipelining and Modules

1. `[h|t] = [1,2,3]`: `h = 1`, `t = [2,3]`. The head is always a single element; the tail is always a list (possibly empty).

2. Pipeline trace:

```

[12, -3, -14, -2, 1, 6, 7, -4, 16, 5, 3, -7, 0, 0, 2, -1, 8]
|> Enum.map(fn e -> e + 1 end)      # each element +1 → [13, -2, -13, -1, 2, ...]
|> Enum.filter(fn e -> e >= 0 end)  # keep non-negatives → [13, 2, 7, 17, 6, 4, 1, 3, 1]
|> Enum.reduce(0, fn e, acc -> e+acc end) # sum → 54 (exact value depends on data)

```

3. Implementing filter:

```

defmodule MyOtherEnum do
  def map([], _fun), do: []
  def map([h|t], fun), do: [fun.(h) | map(t, fun)]

  # Base case: empty list — nothing left to filter
  def filter([], _fun), do: []
  # Recursive case: if head passes the predicate, keep it; otherwise discard it
  def filter([h|t], fun) do
    if fun.(h) do
      [h | filter(t, fun)]
    else
      filter(t, fun)
    end
  end
end
end

```

Module 5 — Maps

1. Missing key: `Map.get(map, :missing)` returns `nil`. Default: `Map.get(map, :missing, "default")` returns `"default"`.

2. Any value as key: Maps in Elixir (backed by HAMT — Hash Array Mapped Trie) hash any Elixir term. Since all terms are immutable and have a well-defined equality, any value can be a key. This is unlike many languages that only allow strings or integers as hash-map keys.

3. MyCalendar implementation:

```

defmodule MyCalendar do
  def generate_month_lengths(year) do
    leap? = rem(year, 4) == 0 and (rem(year, 100) != 0 or rem(year, 400) == 0)
    %{
      jan: 31, feb: (if leap?, do: 29, else: 28), mar: 31,
      apr: 30, may: 31, jun: 30, jul: 31, aug: 31,
      sep: 30, oct: 31, nov: 30, dec: 31
    }
  end

  def generate(min_year, max_year) do
    min_year..max_year
    |> Enum.map(fn y -> {y, generate_month_lengths(y)} end)
    |> Map.new()
  end
end

calendar = MyCalendar.generate(0, 3000)

```

Module 6 — Colors

1. Registry.register in Color.init: It registers the current process (the Color GenServer) under the atom key provided at startup. This makes the process discoverable by name — other code can call `Registry.lookup(BakerRegistry, key)` to find all processes registered under that key without holding a direct PID reference.

2. Why cast not call for slider events: Slider events are fire-and-forget — the UI doesn't need to wait for the GenServer to finish redrawing before continuing to listen for the next event. Using `call` would block the `Kino.listen` callback until the render completes, introducing latency and potential deadlocks.

3. Second inverse Color GenServer:

```
frame2 = Kino.Frame.new() |> Kino.render()
{:ok, pid2} = Color.start_link(frame2)

[{:r, kino_r}, {:g, kino_g}, {:b, kino_b}]
|> Enum.map(fn {channel, kino} ->
  Kino.listen(kino, fn event ->
    v = trunc(event.value)
    Color.update(pid, channel, v)
    Color.update(pid2, channel, 255 - v)
  end)
end)
```

Module 7 — Structs

1. __struct__ key: When you call `Map.put` on a struct, the result is a plain map that still carries the `__struct__: ModuleName` key. This key is how Elixir's pattern matching and `is_struct/1` distinguish structs from plain maps. It also enables struct-specific protocols (like `Inspect`) to format values correctly.

2. birthday/1:

```
defmodule Person2 do
  defstruct [:name, :age, :nationality, :alive?]

  def birthday(person) do
    Map.put(person, :age, person.age + 1)
  end
end
```

3. Average age handling nil:

```
valid_ages = people |> Enum.map(&&1.age) |> Enum.filter(&(&1 != nil))
avg = Enum.sum(valid_ages) / length(valid_ages)
```

José Valim has age: `nil` — filter it out before summing or `Enum.sum` will crash trying to add `nil` to an integer.

Module 8 — Circles

1. Flip negative radii:

```
circles = Enum.map(circles, fn c ->
  if c.r < 0, do: %{c | r: c.r * -1}, else: c
end)
# or more concisely:
circles = Enum.map(circles, fn c -> %{c | r: abs(c.r)} end)
```

2. double_area function: $A = \pi r^2$, so $2A = \pi r_{out}^2$ means $r_{out} = r_{in} \cdot \sqrt{2}$.

```
double_area = fn r -> r * :math.sqrt(2) end
circles = Enum.map(circles, fn c -> %{c | r: double_area.(c.r)} end)
```

3. Maximum distance between centers:

```
max_dist =
  circles
  |> Enum.flat_map(fn c1 ->
    Enum.map(circles, fn c2 ->
      :math.sqrt(:math.pow(c2.x - c1.x, 2) + :math.pow(c2.y - c1.y, 2))
    end)
  end)
  |> Enum.max()
```

4. Preprocessing for VegaLite:

```
data = Enum.map(circles, fn c -> %{"x" => c.x, "y" => c.y, "r" => c.r} end)
```

Module 9 — Fibonacci

1. Pattern matching on function heads: Each `def index(n)` clause has a different pattern (`0`, `1`, `n` when `n > 1`). Elixir tries each clause top-to-bottom and calls the first that matches. This eliminates `if/case` entirely — the dispatch is in the function signature.

2. Making `Concurrent.map` actually concurrent: The process helper must spawn a new process and communicate via `send/receive`:

```
defmodule Concurrent do
  def map(elements, fun) do
    parent = self()
    # fork: spawn one process per element
    pids = Enum.map(elements, fn element ->
      spawn(fn -> send(parent, {self(), fun.(element)}) end)
    end)
    # join: collect results in order
    Enum.map(pids, fn pid ->
      receive do
        {^pid, result} -> result
      end
    end)
  end
end
```

```
end
end
```

3. Sequential vs concurrent vs parallel: - **Sequential:** one task at a time, in order. No overlap. - **Concurrent:** structured so that multiple tasks *can* make progress in overlapping time windows — but may run on a single CPU core via time-slicing. - **Parallel:** physically executing multiple tasks simultaneously on multiple CPU cores. Requires both concurrent structure and hardware support.

Module 10 — Concurrency (Bakery)

1. Message flow when Customer starts: `Customer.init` → calls `Baker.order(product)` → `Registry.lookup(BakerRegistry, product)` finds baker PIDs → picks one at random → `GenServer.cast(baker_pid, {:order, self()})` → baker sleeps (simulating baking) → `GenServer.cast(customer_pid, {:cake, product, name})` → `Customer.handle_cast` logs receipt and orders again.

2. Why Enum.random(workers): Multiple bakers can register under the same product key (:duplicate registry). `Enum.random` load-balances across them — without it, only one baker would ever receive orders. If only one baker is registered, `Enum.random` just returns that one.

3. Clerk design:

State: `%{order_counts: %{product => count}, name: string}`

Messages handled:

- `{:list_products, client_pid}` → reply with list of available products
- `{:order, product, customer_pid}` → forward to a baker; increment count
- `{:cake, product, baker_name, customer_pid}` → forward cake to customer

The Clerk decouples customers from bakers: customers only know clerk PIDs, clerks only know baker PIDs via Registry. Adding more clerks or bakers requires no changes to customer or baker code.

Quest 2: Getting Acquainted

Exploration task. Points to cement from each workbook:

Basic data types: integers, floats, atoms (`:ok`, `:error`), tuples (`{:ok, value}`), lists `[1, 2, 3]`, maps `%{key: value}`, binaries/strings.

Control structures: `if/else` is an expression; `cond` is a multi-branch `cond`; `case` matches against patterns (much more common than `if`).

Pattern matching: the `=` operator is a *match* operator, not assignment. `{a, b} = {1, 2}` binds `a=1`, `b=2`. Failed matches raise `MatchError`. Used everywhere: function heads, `case`, `receive`.

Pipelines: `|>` passes the left-hand result as the *first argument* of the right-hand function. Makes data transformation chains readable without nested function calls.

Primitive concurrency: `spawn/1` creates a lightweight process. `send/2` and `receive` are the only inter-process communication primitives. Processes share nothing (message passing, not shared memory).

Quest 3: Understanding the LiveMD Format

LiveMD is valid Markdown.

The `.livemd` file is a Markdown document where each code cell is a fenced code block (````elixir`). Livebook reads the Markdown AST and renders each code block as an executable cell, storing outputs separately. The file itself remains plain Markdown that can be read in any text editor or rendered on GitHub.

Structure:

- ``# Heading`` → section headers (also used as cell group delimiters by Livebook)
 - ````elixir ... ```` → executable Elixir cells
 - Regular Markdown prose → documentation cells
 - Special `<!-- livebook:{...} -->` HTML comments carry Livebook metadata (e.g. cell IDs, output caching) without breaking standard Markdown rendering.
-

Quest 4: Sources of Information

The `documentation.livemd` workbook links to the standard Elixir documentation ecosystem:

- **Livecoding videos:** [Elixir's official YouTube channel](#) and [ElixirConf talks on YouTube](#).
 - **Documentation for a specific module:** hexdocs.pm — search by module name (e.g. `Enum`, `String`, `GenServer`). In Livebook you can also use `h(Enum.map)` in an IEx cell.
 - **Textual language introduction:** elixir-lang.org/getting-started — the official Getting Started guide; also [Elixir School](#).
-

Quest 5: Experimenting with Pipelines

The final cell in the Pipelines workbook produces `MapSet.new([0, 1, 3, 4])`.

To get a list instead of a MapSet: - **Remove:** `MapSet.new/1` at the end of the pipeline. - **Add:** `Enum.uniq/1` (to preserve the deduplication behaviour that `MapSet` provided).

```
# Original (simplified):
result =
  some_list
  |> Enum.filter(...)
  |> Enum.map(...)
  |> MapSet.new()      # ← remove this

# Replacement:
```

```
result =
  some_list
  |> Enum.filter(...)
  |> Enum.map(...)
  |> Enum.uniq()      # ← add this (returns a list)
```

Effect of disabling `String.downcase/1`: String comparison in Elixir (and most languages) is case-sensitive. Without `downcase`, "Hello" and "hello" are treated as different values. This means: - Deduplication steps that relied on case-insensitive equality will now keep both variants. - Filtering steps that match against lowercase strings will miss uppercase inputs. - The resulting set/list will contain more (possibly duplicate-looking) entries.

Quest 6: Working with Functions

High-level explanation of `plotting-functions.livemd`: The notebook defines mathematical functions as Elixir anonymous functions, generates a list of {x, y} data points using a list comprehension, and plots them using `VegaLite` (a Vega-Lite specification wrapper). It then demonstrates a live/animated plot that updates at a configurable interval using `Kino.VegaLite`.

Comprehension explanation:

```
for x <- 0..100, do: {x / 10, :math.sin(x / 10)}
```

This generates a list of 101 {x, sin(x)} tuples for x from 0.0 to 10.0 in steps of 0.1. The `for` comprehension is syntactic sugar over `Enum.map` with an optional filter (`:when` guard).

Filling area below the curve — add `"fill": "tozeroy"` to the mark options:

```
VegaLite.mark(vl, :area, tooltip: true)
# or equivalently in the spec:
# %{"type" => "area", "line" => true}
```

Dynamic plot questions:

- **interval variable:** controls the delay (in ms) between each update tick — adjusting it speeds up or slows down the animation.
 - **2nd argument to `Kino.VegaLite.listen`:** a callback function that receives the widget handle; it is called each time the timer fires and is responsible for computing/pushing new data.
 - **Return value of `fun`:** a list of data point maps (e.g. [%{"x" => x, "y" => y}]). `VegaLite` needs map-encoded data, not raw tuples.
 - **`Kino.VegaLite.push`:** appends new data points to the live chart without re-rendering the entire chart from scratch — it streams data into the running Vega-Lite spec.
 - **window option in `Kino.VegaLite.push`:** limits how many data points are retained in the chart at once (a sliding window). Without it, the chart accumulates all points indefinitely and becomes slow.
-

Quest 7: Servers Everywhere!

Why is the server implementation called a “callback”? GenServer provides the generic server *loop* (receive messages, dispatch, send replies, handle crashes). Your module only provides the *application-specific* logic — what to do when a particular message arrives. GenServer calls your functions (`handle_call`, `handle_cast`, `handle_info`) at the right moments, just like a framework calling your registered event handlers. Your module is the “callback module” because control is *inverted*: the framework calls you, not the other way around.

Why have both `call` and `cast`? - `call` is synchronous: the caller blocks until the server sends a reply. Use when you need a return value or need to know the operation completed. - `cast` is asynchronous (fire-and-forget): the caller continues immediately. Use for side-effectful operations where you don’t need confirmation.

If you **only used `call`**: every operation would block the caller. This is fine for low-frequency interactions but causes head-of-line blocking — a slow call blocks all subsequent calls from the same process.

Only calls and every call takes a long time: The caller process (and any process waiting on it) would be blocked for the duration of each call. The server’s mailbox would grow if multiple callers pile up. In the worst case this creates deadlocks if two GenServers call each other simultaneously. You’d also lose concurrency benefits entirely — all callers serialise through the slow server.

Registry patterns: Beyond simple {key, value} storage, Registry supports: - **Unique keys** (:unique) — one process per key; used for named process registries. - **Duplicate keys** (:duplicate) — multiple processes per key; enables pub/sub (all subscribers under the same topic key receive broadcasts via `Registry.dispatch/3`).

Quest 8: World of Zuul — Fixing say

What mechanism makes sessions independent? Each tab creates its own Session GenServer process. BEAM processes share no memory; each holds its own state (current room, name, client frame). Messages to one session process have no effect on another.

The problem: when a player issues `say hello`, only their own frame is updated. Other players in the same room are not notified because the Space (room) server does not track which players are currently in it.

Solution approach — track players in each Space server:

The Space GenServer needs to maintain a set of {player_pid, client_id} pairs for currently-present players. Update it on enter/leave events, and broadcast on say.

```
# In the Space GenServer state, add a field:
defstruct [:name, :exits, :description, players: []]

# Handle a player entering the room:
def handle_call({:enter, player_pid, client_id}, _from, state) do
  new_state = %{state | players: [{player_pid, client_id} | state.players]}
  {:reply, :ok, new_state}
end
```



```

# Handle a player leaving the room:
def handle_call({:leave, player_pid}, _from, state) do
  new_state = %{state | players: Enum.reject(state.players, fn {pid, _} -> pid == player_pid}
  {:reply, :ok, new_state}
end

# Handle a say command – broadcast to all players in the room:
def handle_call({:say, speaker_name, message}, _from, state) do
  formatted = "#{speaker_name} says: #{message}\n"
  Enum.each(state.players, fn {_pid, client_id} ->
    Kino.Frame.append(client_id, Kino.Text.new(formatted))
  end)
  {:reply, :ok, state}
end

```

In the Session GenServer, intercept the say command:

```

# In handle_cast for parsing player input:
"say " <> message ->
  Space.say(state.current_room, state.name, message)
  {:noreply, state}

```

And call Space.enter/leave when a player moves between rooms:

```

# On move:
Space.leave(state.current_room, self())
Space.enter(new_room, self(), state.client_frame)

```

TIP: The client_id (Kino frame handle) needs to be stored in the Session state at connection time and passed to the Space on enter.

Solutions — Exercises 3: Metadata — Adventures in Semantic Madness

INFO: RDF/SPARQL and Neo4j/Cypher solutions are both provided for Quests 3-6. The course uses a Brick-like building model with rooms, sensors, streams, and units.

Quest 1: Exploring the Lecture Example

High-level structure of the lecture building model:

The model represents a multi-floor building with the following entity types: - Building / Floor / Room — physical containment hierarchy - Sensor — a physical device (temperature, humidity, occupancy) - Stream / Point — a live data feed associated with a sensor - Unit — the unit of measurement (Celsius, %, boolean)

Key relationships: - hasPart / contains — Floor contains Rooms; Building contains Floors - isLocatedIn — Sensor is located in a Room - provides — Sensor provides a Stream - hasUnit — Stream has a Unit

Where the streams are crossed: The lecture example intentionally has a stream that is assigned the wrong unit — e.g., a temperature sensor's stream is labelled as reporting in % (humidity unit) rather than C. The consequence: any application that queries for “temperature streams in Celsius” will miss this sensor, while an application querying for humidity streams will incorrectly include temperature data. The model is *syntactically* valid but *semantically* wrong — schema validation (Quest 6) is the remedy.

Quest 2: Exploring the Source

Exploration task — no single answer. The key structural insight to verify:

RDF (rdf.livemd): triples are stored as {subject, predicate, object}. Querying uses SPARQL SELECT with WHERE patterns. The graph is schema-free by default; anyone can add any triple.

Neo4j (neo4j.livemd): nodes have labels (:Room, :Sensor), relationships have types ([:CONTAINS], [:PROVIDES]). Cypher reads like ASCII art of the graph structure: (a)-[:REL]->(b).

Quest 3: A New Query

Goal: list all rooms that are contained in a building AND have a stream reporting in Kelvin.

SPARQL (RDF)

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX unit: <http://qudt.org/vocab/unit/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
```

```

SELECT ?room WHERE {
  ?building rdf:type brick:Building .
  ?building brick:hasPart ?room .
  ?room rdf:type brick:Room .
  ?sensor brick:isLocatedIn ?room .
  ?sensor brick:provides ?stream .
  ?stream brick:hasUnit unit:Kelvin .
}

```

TIP: The exact predicate names depend on your lecture's vocabulary. If the model uses `contains` instead of `hasPart`, adjust accordingly.

Cypher (Neo4j)

```

MATCH (b:Building)-[:CONTAINS]->(r:Room)
      <-[:LOCATED_IN]-(s:Sensor)-[:PROVIDES]->(st:Stream)-[:HAS_UNIT]->(u:Unit)
WHERE u.name = "Kelvin"
RETURN r.name AS room

```

Explanation: the pattern chains four relationships. Neo4j traverses the graph following the arrows; Cypher verifies the label and property constraints at each node.

Quest 4: Universe Expansion

Adding a Building concept and connecting it to the existing model.

RDF / Turtle

```

@prefix brick: <https://brickschema.org/schema/Brick#> .
@prefix ex: <http://example.org/building/> .

```

```

# Declare the Building type
brick:Building a rdfs:Class .

```

```

# Add a concrete building instance
ex:BuildingA a brick:Building ;
             rdfs:label "Building A" .

```

```

# Connect to existing floors
ex:BuildingA brick:hasPart ex:Floor1 .
ex:BuildingA brick:hasPart ex:Floor2 .

```

Verification query (SPARQL):

```

SELECT ?floor WHERE {
  ex:BuildingA brick:hasPart ?floor .
  ?floor rdf:type brick:Floor .
}

```

Cypher (Neo4j)

```
// Create building node
CREATE (b:Building {name: "Building A"})

// Connect to existing floors
MATCH (f:Floor) WHERE f.name IN ["Floor 1", "Floor 2"]
MATCH (b:Building {name: "Building A"})
CREATE (b)-[:CONTAINS]->(f)
```

Verification query:

```
MATCH (b:Building {name: "Building A"})-[:CONTAINS]->(f:Floor)
RETURN b.name AS building, f.name AS floor
```

Quest 5: Temperature Actuator (Thermostat)

New types needed: - Setpoint — a target temperature value (data point, not a live sensor reading) - Actuator — a controllable device (e.g. radiator valve) - TemperatureActuator — subtype of Actuator

On the relationship question: provides is appropriate for sensors→streams. For actuators, the relationship is inverted: the thermostat application *writes to* the actuator. A new relationship controls or actuates is semantically cleaner than reusing provides.

RDF / Turtle — adding types and instances

```
# Type declarations
brick:Actuator a rdfs:Class .
brick:TemperatureActuator rdfs:subClassOf brick:Actuator .
brick:Setpoint a rdfs:Class .
brick:controls rdf:type rdf:Property .

# Room U181 thermostat
ex:ThermostatValve_U181 a brick:TemperatureActuator ;
    brick:isLocatedIn ex:Room_U181 ;
    ex:actuatorID "valve://building-a/floor-1/room-181" .

ex:TempSetpoint_U181 a brick:Setpoint ;
    brick:isLocatedIn ex:Room_U181 ;
    ex:defaultValue "21.0"^^xsd:float ;
    brick:hasUnit unit:DegreeCelsius .
```

Query — extract thermostat IDs per room:

```
SELECT ?room ?tempStreamID ?setpointID ?actuatorID WHERE {
    ?room rdf:type brick:Room .

    # Temperature sensor stream
    ?tempSensor brick:isLocatedIn ?room ;
        brick:provides ?tempStream .
```

```

?tempStream brick:hasUnit unit:DegreeCelsius .
BIND(str(?tempStream) AS ?tempStreamID)

# Setpoint
?setpoint rdf:type brick:Setpoint ;
          brick:isLocatedIn ?room .
BIND(str(?setpoint) AS ?setpointID)

# Actuator
?actuator rdf:type brick:TemperatureActuator ;
          brick:isLocatedIn ?room ;
          ex:actuatorID ?actuatorID .
}

```

Cypher (Neo4j)

```

// Add actuator and setpoint for each room
MATCH (r:Room)
CREATE (a:Actuator:TemperatureActuator {
    id: "valve://" + toLower(replace(r.name, " ", "-")),
    room: r.name
})
CREATE (sp:Setpoint {default: 21.0, unit: "Celsius", room: r.name})
CREATE (r)-[:HAS_ACTUATOR]->(a)
CREATE (r)-[:HAS_SETPOINT]->(sp)

// Thermostat query – all IDs per room
MATCH (r:Room)
    -[:HAS_SENSOR]->(s:Sensor)-[:PROVIDES]->(st:Stream)-[:HAS_UNIT]-
>(u:Unit {name: "Celsius"}),
    (r)-[:HAS_SETPOINT]->(sp:Setpoint),
    (r)-[:HAS_ACTUATOR]->(a:Actuator)
RETURN r.name AS room, st.id AS temp_stream, sp.default AS setpoint, a.id AS actuator_id

```

What happens when a room is missing a required component? In SPARQL, the WHERE clause requires *all* triple patterns to match. A room missing a temperature sensor, setpoint, or actuator will produce **no row** in the result — it is silently excluded. This is the “open world assumption” of RDF: absence of a triple does not mean the thing doesn’t exist; it just means it’s not in the graph.

In Neo4j/Cypher, MATCH similarly returns no result for incomplete rooms. You can expose missing components explicitly with OPTIONAL MATCH:

```

MATCH (r:Room)
OPTIONAL MATCH (r)-[:HAS_SENSOR]->(s:Sensor)-[:PROVIDES]->(st:Stream)
OPTIONAL MATCH (r)-[:HAS_SETPOINT]->(sp:Setpoint)
OPTIONAL MATCH (r)-[:HAS_ACTUATOR]->(a:Actuator)
RETURN r.name,
    st.id AS temp_stream,
    sp.default AS setpoint,
    a.id AS actuator_id,

```

```

CASE WHEN st IS NULL THEN "MISSING SENSOR"
      WHEN sp IS NULL THEN "MISSING SETPOINT"
      WHEN a IS NULL THEN "MISSING ACTUATOR"
      ELSE "OK" END AS status

```

Quest 6: Schema Validation

What a model checker would look like: A model checker would be a program (or SHACL/OWL reasoner) that traverses the graph and asserts invariants. It would report violations as a list of {node, rule, description} triples.

Rules currently missing from the model:

Rule	Description
Unit consistency	Every Stream must have exactly one hasUnit triple; the unit must be from a controlled vocabulary
Sensor-room binding	Every Sensor must be located in exactly one Room
Room containment	Every Room must be part of exactly one Floor; every Floor part of one Building
Stream type correctness	A temperature sensor must provide a stream with a temperature unit (Celsius, Fahrenheit, Kelvin), not a humidity unit
Actuator completeness	If a room has a Setpoint, it must also have a compatible Actuator and a live temperature Stream
No orphan streams	Every Stream must be provided by exactly one Sensor

Formal description (SHACL example for unit consistency):

```

ex:StreamShape a sh:NodeShape ;
  sh:targetClass brick:Stream ;
  sh:property [
    sh:path brick:hasUnit ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:class qudt:Unit ;
    sh:message "A Stream must have exactly one hasUnit from the QUDT unit vocabulary." ;
  ] .

```

Implementation options:

- **SHACL** (RDF): declarative shape language; validators (e.g. pySHACL) check the graph against shapes and report violations. No code required.
- **OWL + reasoner** (RDF): declare axioms (owl:someValuesFrom, owl:cardinality); a reasoner (Pellet, Hermit) infers consequences and flags inconsistencies.
- **Custom Cypher assertions** (Neo4j): write a set of Cypher queries that each return rows when a rule is violated. Run them as part of a CI/nightly check.
- **Elixir/Python validation script**: query the graph, apply rules programmatically, output a report. Most flexible but most work to maintain.

NOTE: SHACL is the practical standard for production RDF systems. For the course, a small set of Cypher or SPARQL “violation queries” (queries that should

return zero rows) is a pragmatic implementation.

Solutions — Exercises 4: C — Into the Muck

Quest 1: Making That First Compile

```
# Compile
gcc -o hello hello.c

# Run
./hello
# Output: Hello, World
```

The main function signature `int main(int argc, char* argv[])` is the standard C entry point. `argc` is the argument count (always ≥ 1 , includes the program name), `argv` is the array of argument strings. `printf` writes to `stdout`; `\n` is a newline. Returning `0` signals success to the shell.

Quest 2: Offsetting the Index

Step 5 — `index0.c` output explanation

```
#include <stdio.h>
#define SIZE 10

int main(void) {
    int a[SIZE];
    for (int i = 0; i < SIZE; i++) {
        printf("%d: %d\n", i, a[i]);
    }
    return 0;
}
```

`a` is declared but not initialised. In C, local arrays contain **garbage** — whatever bytes happened to be at that stack address. The values printed are unpredictable and will differ between runs/compilers/OS. This is **undefined behaviour** if you read before writing.

Step 6 — `index0b.c`

```
for (int i = 0; i < SIZE; i++) {
    a[i] = i;
}
```

Now `a[0]=0`, `a[1]=1`, ..., `a[9]=9`. The loop initialises before reading.

Steps 7-10 — index1.c (1-indexed array trick)

```
#include <stdio.h>
#define SIZE 10

int main(void) {
    int a[SIZE];
    for (int i = 0; i < SIZE; i++) a[i] = i;

    int* b = &a[0];    // b points to a[0]
    b--;               // move b one int BEFORE a[0] → b+1 == &a[0]

    // Loop from 1 to SIZE (inclusive)
    for (int i = 1; i <= SIZE; i++) {
        printf("%d: %d\n", i, b[i]);
    }
    return 0;
}
```

How it works: - `b = &a[0]` makes `b` point to the first element. - `b--` moves the pointer back by `sizeof(int)` bytes — now `b` points *before* the array. - `b[1]` is equivalent to `*(b+1)` which is `a[0]`. `b[2]` is `a[1]`, etc. - The loop runs `i = 1..SIZE` so `b[i]` addresses `a[i-1]` — exactly 1-indexed behaviour.

Step 11 — `b[0]` explanation: `b[0]` dereferences the pointer one position *before* the array. This is a read of memory that does not belong to `a`. It is **undefined behaviour** in standard C. In practice on a typical stack, it reads whatever was at that stack address — often the previous stack frame's data, or a saved register. The value is unpredictable.

Quest 3: In Support of a Hero — TSP

Greedy nearest-neighbour heuristic — simple, not optimal, but runs in $O(n)$:

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

#define CITY_COUNT (15)

char* citynames[CITY_COUNT] = {
    "Esbjerg", "Helsingør", "Herning", "Horsens", "Kolding",
    "København", "Næstved", "Odense", "Randers", "Roskilde",
    "Silkeborg", "Vejle", "Viborg", "Aalborg", "Århus",
};

int distances[CITY_COUNT][CITY_COUNT] = {
    { 0, 269, 82, 98, 65, 260, 211, 123, 149, 230, 105, 74, 126, 198, 134},
    {269, 0, 226, 173, 206, 40, 105, 157, 166, 55, 191, 196, 207, 200, 150},
    { 82, 226, 0, 63, 79, 230, 202, 121, 76, 202, 36, 59, 45, 117, 77},
```

```

{ 98, 173, 63, 0, 48, 172, 139, 62, 68, 142, 40, 26, 75, 132, 40},
{ 65, 206, 79, 48, 0, 196, 148, 59, 114, 165, 77, 25, 110, 176, 88},
{260, 40, 230, 172, 196, 0, 71, 141, 180, 31, 196, 190, 218, 224, 156},
{211, 105, 202, 139, 148, 71, 0, 89, 174, 50, 175, 150, 204, 232, 142},
{123, 157, 121, 62, 59, 141, 89, 0, 121, 110, 102, 64, 136, 186, 86},
{149, 166, 76, 68, 114, 180, 174, 121, 0, 156, 44, 90, 42, 65, 36},
{230, 55, 202, 142, 165, 31, 50, 110, 156, 0, 169, 160, 193, 206, 130},
{105, 191, 36, 40, 77, 196, 175, 102, 44, 169, 0, 53, 35, 99, 41},
{ 74, 196, 59, 26, 25, 190, 150, 64, 90, 160, 53, 0, 86, 151, 65},
{126, 207, 45, 75, 110, 218, 204, 136, 42, 193, 35, 86, 0, 72, 63},
{198, 200, 117, 132, 176, 224, 232, 186, 65, 206, 99, 151, 72, 0, 101},
{134, 150, 77, 40, 88, 156, 142, 86, 36, 130, 41, 65, 63, 101, 0},
};

int main(void) {
    bool visited[CITY_COUNT] = {false};
    int route[CITY_COUNT + 1];
    int total = 0;

    int current = 0;    // start at Esbjerg (index 0)
    visited[0] = true;
    route[0] = 0;

    for (int step = 1; step < CITY_COUNT; step++) {
        int nearest = -1;
        int min_dist = INT_MAX;
        for (int j = 0; j < CITY_COUNT; j++) {
            if (!visited[j] && distances[current][j] < min_dist) {
                min_dist = distances[current][j];
                nearest = j;
            }
        }
        visited[nearest] = true;
        route[step] = nearest;
        total += min_dist;
        current = nearest;
    }

    // Return to start
    total += distances[current][0];
    route[CITY_COUNT] = 0;

    // Print result
    printf("Route:\n");
    for (int i = 0; i <= CITY_COUNT; i++) {
        printf("  %s\n", citynames[route[i]]);
    }
    printf("Total distance: %d km\n", total);
}

```

```
return 0;
}
```

Expected greedy output (starting from Esbjerg): Esbjerg → Kolding → Vejle → Horsens → Silkeborg → Herning → Viborg → Randers → Århus → Aalborg → ... → Esbjerg Total ≈ 920-1000 km depending on tie-breaking.

NOTE: The greedy nearest-neighbour does **not** guarantee the optimal solution. The true TSP optimum for these 15 cities is shorter. For an exact solution you would implement branch-and-bound or use the Held-Karp algorithm ($O(2^n)$).

Trying all starting cities improves the greedy result — run the above with `current = s` for each `s` in `0..14` and keep the shortest overall tour.

Quest 4: The Final Test — Memory Layout Analysis

```
int* values = (int*) malloc(LENGTH * sizeof(int));
// LENGTH = 12, sizeof(int) = 4 on 64-bit → 48 bytes allocated

for (int i = 0; i < LENGTH; i++) {
    values[i] = i;    // fills: 0,1,2,3,4,5,6,7,8,9,10,11
}

for (int i = 0; i < LENGTH/2; i++) {    // i = 0..5
    printf("%u: %d\n", i, (int)(((long*)values)[i]));
}
```

Key facts: - `sizeof(int)` = 4 bytes on a 64-bit system - `sizeof(long)` = 8 bytes on a 64-bit Linux/macOS system (not Windows where `long` = 4) - `(long*)values` reinterprets the same 48-byte buffer as an array of 6 long values - Each long slot covers 8 bytes = 2 consecutive int values

Memory layout (little-endian, 64-bit):

Byte offset:	0	4	8	12	16	20	24	28	32	36	40	44
int values:	0	1	2	3	4	5	6	7	8	9	10	11
long[0]	0 + 1 << 32											
long slots:	[0,1]	[2,3]	[4,5]	[6,7]	[8,9]	[10,11]						

On a **little-endian** system, int 0 occupies the low 4 bytes of `long[0]` and int 1 occupies the high 4 bytes:

```
long[0] = (int[1] << 32) | int[0] = (1 << 32) | 0 = 4294967296
long[1] = (3 << 32) | 2 = 12884901890
...
```

But the code casts back to `int` with `(int)(((long*)values)[i])`, which **truncates** to the low 32 bits:

```
(int) long[0] = (int)(4294967296) = 0    → low 32 bits of (1<<32 | 0)
(int) long[1] = (int)(12884901890) = 2
```

```
(int) long[2] = 4
(int) long[3] = 6
(int) long[4] = 8
(int) long[5] = 10
```

Output:

```
0: 0
1: 2
2: 4
3: 6
4: 8
5: 10
```

Explanation: Casting `int*` to `long*` merges pairs of ints into single longs (each long straddles two int slots). Re-casting to `int` discards the high half of each long, leaving only the even-indexed original integers (0, 2, 4, 6, 8, 10). The loop runs only $\text{LENGTH}/2 = 6$ times because there are only 6 long-sized slots in the 48-byte buffer.

WARNING: This code relies on `sizeof(long) == 8` (true on 64-bit Linux/macOS, false on Windows where long is 4 bytes). On Windows the output would be 0,1,2,3,4,5 — every int, since long and int are the same size. This is why `stdint.h` types (`int32_t`, `int64_t`) are preferred in portable code.

Solutions — Exercises 5: Toolchain — A Device Takes Its First Step

INFO: Quests 1-2 are hardware setup tasks. Solutions here focus on the conceptual questions and complete code for Quests 3-6.

Quest 1: Early Steps — Conceptual Notes

Finding the serial port:

```
# Linux — before plugging in:
ls /dev/tty*
# After plugging in, new entry appears, typically:
/dev/ttyUSB0    # CP210x USB-to-UART bridge (common on ESP32 DevKit)
/dev/ttyACM0    # CDC-ACM (less common)

# macOS:
ls /dev/cu.*
# Typically: /dev/cu.usbserial-XXXX
```

Building and flashing:

```
cd esp-idf
. ./export.sh                # source the environment
cd examples/get-started/hello_world
idf.py set-target esp32
idf.py menuconfig            # configure if needed
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor # flash and open serial monitor
```

What hello_world does: prints “Hello world!” then restarts after 10 seconds in an infinite loop, logging the restart reason each cycle. It demonstrates `esp_restart()`, `printf`, and basic FreeRTOS `app_main` entry.

Baud rate: the `hello_world` example uses 115200 baud (default ESP-IDF console UART rate). Set your serial terminal to match.

Quest 2: Blinking Away

ESP32-DevKitC-VE built-in LED: GPIO 2 on most variants (check your specific board schematic — some use GPIO 33 or have no onboard LED at all).

blink2 — two LEDs at different speeds using task parameters:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
```

```

typedef struct {
    int pin;
    int delay_ms;
} blink_params_t;

void TaskBlink(void *pvParameters)
{
    blink_params_t* params = (blink_params_t*) pvParameters;
    uint8_t state = 1;
    while (1) {
        gpio_set_level(params->pin, state);
        vTaskDelay(params->delay_ms / portTICK_PERIOD_MS);
        state = !state;
    }
}

// Statically allocate params (must outlive the task)
static blink_params_t led1 = {.pin = 33, .delay_ms = 500};
static blink_params_t led2 = {.pin = 32, .delay_ms = 1500};

void app_main(void)
{
    gpio_reset_pin(led1.pin);
    gpio_set_direction(led1.pin, GPIO_MODE_OUTPUT);
    gpio_reset_pin(led2.pin);
    gpio_set_direction(led2.pin, GPIO_MODE_OUTPUT);

    xTaskCreate(TaskBlink, "Blink1", 4096, &led1, 1, NULL);
    xTaskCreate(TaskBlink, "Blink2", 4096, &led2, 1, NULL);
}

```

NOTE: pvParameters is a void* — cast it to your struct pointer inside the task. The struct must be allocated in static or heap memory (not a local variable in app_main) because app_main may complete before the tasks do.

Quest 3: Serial Count

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/uart.h"
#include "driver/gpio.h"

#define UART_NUM    UART_NUM_0
#define BUF_SIZE    256
#define BAUD_RATE   115200

```

```

// Initialise UART0 (connected to USB)
static void uart_init(void) {
    uart_config_t cfg = {
        .baud_rate   = BAUD_RATE,
        .data_bits   = UART_DATA_8_BITS,
        .parity       = UART_PARITY_DISABLE,
        .stop_bits   = UART_STOP_BITS_1,
        .flow_ctrl    = UART_HW_FLOWCTRL_DISABLE,
    };
    uart_param_config(UART_NUM, &cfg);
    uart_driver_install(UART_NUM, BUF_SIZE * 2, 0, 0, NULL, 0);
}

// Read one line (up to \n or \r\n) into buf
static void uart_readline(char* buf, int maxlen) {
    int pos = 0;
    while (pos < maxlen - 1) {
        uint8_t c;
        int n = uart_read_bytes(UART_NUM, &c, 1, portMAX_DELAY);
        if (n <= 0) continue;
        if (c == '\r') continue; // ignore CR
        if (c == '\n') break;
        buf[pos++] = (char)c;
    }
    buf[pos] = '\0';
}

// Count down from n to 0 over UART
void countdown(uint8_t n) {
    for (int i = (int)n; i >= 0; i--) {
        printf("%d\n", i);
        fflush(stdout);
        vTaskDelay(200 / portTICK_PERIOD_MS);
    }
}

void app_main(void) {
    uart_init();
    char buf[32];
    while (1) {
        printf("Enter a number: ");
        fflush(stdout);
        uart_readline(buf, sizeof(buf));
        int n = atoi(buf);
        if (n > 0) {
            countdown((uint8_t)n);
        } else {
            printf("Invalid input.\n");
        }
    }
}

```

```
}  
}
```

TIP: printf on ESP-IDF uses the UART0 console by default. fflush(stdout) forces the buffer to flush immediately — otherwise output may be delayed.

Quest 4: Reporting Changes (Polling)

```
#include <stdio.h>  
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
#include "driver/gpio.h"  
  
#define INPUT_PIN 4    // adjust to your wiring  
  
void app_main(void) {  
    gpio_reset_pin(INPUT_PIN);  
    gpio_set_direction(INPUT_PIN, GPIO_MODE_INPUT);  
  
    int prev = gpio_get_level(INPUT_PIN);  
  
    while (1) {  
        int curr = gpio_get_level(INPUT_PIN);  
        if (curr != prev) {  
            printf("%d\n", curr);    // "1" on rise, "0" on fall  
            fflush(stdout);  
            prev = curr;  
        }  
        vTaskDelay(10 / portTICK_PERIOD_MS);    // 10 ms poll interval  
    }  
}
```

Wiring: button → GPIO pin AND through $\geq 330\Omega$ to GND; other side of button → 3.3V. Without the pull-down resistor the pin floats and will read noise.

Bonus: Interrupt-based detection

```
#include <stdio.h>  
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
#include "driver/gpio.h"  
  
#define INPUT_PIN 4  
  
static void IRAM_ATTR gpio_isr_handler(void* arg) {  
    int level = gpio_get_level(INPUT_PIN);  
    // ISRs must be minimal — avoid printf here; use a queue in real code
```



```

    ets_printf("%d\n", level); // ets_printf is ISR-safe
}

void app_main(void) {
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << INPUT_PIN),
        .mode          = GPIO_MODE_INPUT,
        .pull_up_en     = GPIO_PULLUP_DISABLE,
        .pull_down_en   = GPIO_PULLEDOWN_DISABLE,
        .intr_type      = GPIO_INTR_ANYEDGE, // trigger on both edges
    };
    gpio_config(&io_conf);
    gpio_install_isr_service(0);
    gpio_isr_handler_add(INPUT_PIN, gpio_isr_handler, NULL);

    while (1) {
        vTaskDelay(1000 / portTICK_PERIOD_MS); // idle; ISR does the work
    }
}

```

NOTE: In production code, ISRs should post to a FreeRTOS queue and let a normal task handle printing/processing. `ets_printf` is used above as a shortcut because `printf` is not ISR-safe.

Quest 5: Command and Control

Complete cnc implementation with all four extra commands:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/uart.h"
#include "driver/gpio.h"

#define UART_NUM    UART_NUM_0
#define BUF_SIZE    256
#define BAUD_RATE   115200
#define MAX_ARGS    16

// — UART helpers —

static void uart_init(void) {
    uart_config_t cfg = {
        .baud_rate = BAUD_RATE, .data_bits = UART_DATA_8_BITS,
        .parity    = UART_PARITY_DISABLE, .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
    };
}

```

```

    uart_param_config(UART_NUM, &cfg);
    uart_driver_install(UART_NUM, BUF_SIZE * 2, 0, 0, NULL, 0);
}

static void uart_readline(char* buf, int maxlen) {
    int pos = 0;
    while (pos < maxlen - 1) {
        uint8_t c;
        if (uart_read_bytes(UART_NUM, &c, 1, portMAX_DELAY) <= 0) continue;
        if (c == '\r') continue;
        if (c == '\n') break;
        buf[pos++] = (char)c;
    }
    buf[pos] = '\0';
}

// — Command type —————

typedef void (*command_t)(int argc, char* argv[]);

typedef struct {
    char*      name;
    command_t fn;
} dispatch_entry_t;

// — Command implementations —————

void command_echo(int argc, char* argv[]) {
    for (int i = 1; i < argc; i++) {
        printf("%s", argv[i]);
        if (i < argc - 1) printf(" ");
    }
    printf("\n");
}

void command_pin_high(int argc, char* argv[]) {
    if (argc < 2) { printf("usage: pin-high <pin>\n"); return; }
    int pin = atoi(argv[1]);
    gpio_reset_pin(pin);
    gpio_set_direction(pin, GPIO_MODE_OUTPUT);
    gpio_set_level(pin, 1);
    printf("pin %d HIGH\n", pin);
}

void command_pin_low(int argc, char* argv[]) {
    if (argc < 2) { printf("usage: pin-low <pin>\n"); return; }
    int pin = atoi(argv[1]);
    gpio_reset_pin(pin);
    gpio_set_direction(pin, GPIO_MODE_OUTPUT);

```

```

    gpio_set_level(pin, 0);
    printf("pin %d LOW\n", pin);
}

void command_pin_read(int argc, char* argv[]) {
    if (argc < 2) { printf("usage: pin-read <pin>\n"); return; }
    int pin = atoi(argv[1]);
    gpio_reset_pin(pin);
    gpio_set_direction(pin, GPIO_MODE_INPUT);
    printf("pin %d = %d\n", pin, gpio_get_level(pin));
}

void command_reverse(int argc, char* argv[]) {
    for (int i = argc - 1; i >= 1; i--) {
        printf("%s", argv[i]);
        if (i > 1) printf(" ");
    }
    printf("\n");
}

// — Dispatch table —————

#define COMMAND_COUNT 5

dispatch_entry_t command_dispatch_table[COMMAND_COUNT] = {
    {"echo",      command_echo},
    {"pin-high",  command_pin_high},
    {"pin-low",   command_pin_low},
    {"pin-read",  command_pin_read},
    {"reverse",   command_reverse},
};

// — Dispatch function —————

void dispatch(char* line) {
    char* argv[MAX_ARGS];
    int   argc = 0;

    // Tokenise: replace spaces with \0, record start of each token
    char* p = line;
    while (*p && argc < MAX_ARGS) {
        // Skip leading spaces
        while (*p == ' ') p++;
        if (*p == '\0') break;
        argv[argc++] = p;
        // Advance to next space or end
        while (*p && *p != ' ') p++;
        if (*p == ' ') *p++ = '\0';
    }
}

```

```

    if (argc == 0) return;

    // Look up command
    for (int i = 0; i < COMMAND_COUNT; i++) {
        if (strcmp(argv[0], command_dispatch_table[i].name) == 0) {
            command_dispatch_table[i].fn(argc, argv);
            return;
        }
    }
    printf("unable to comply: unknown command '%s'\n", argv[0]);
}

// — Entry point —
void app_main(void) {
    uart_init();
    char buf[BUF_SIZE];
    printf("CNC ready. Commands: echo, pin-high, pin-low, pin-read, reverse\n");
    while (1) {
        printf("> ");
        fflush(stdout);
        uart_readline(buf, sizeof(buf));
        dispatch(buf);
    }
}

```

Design notes: - The dispatch table decouples parsing from implementation: adding a new command is a one-line table entry + one function. - `atoi` silently returns 0 for non-numeric strings — a more robust implementation would use `strtol` and check `errno`. - The tokeniser modifies the input string in-place (writes `\0` bytes) which is fine since `buf` is a local copy.

Quest 6: Bridging Devices

This quest is a hardware integration exercise. The key insight is that Device 1 (cnc) uses pin-high/pin-low to drive a GPIO line, which Device 2 (reporter) reads. They share a common ground.

Physical wiring:

```

Device 1 GPIO_OUT — 100Ω — Device 2 GPIO_IN
Device 1 GND       ————— Device 2 GND

```

Verification sequence:

1. ``pin-high 33`` on Laptop A → Device 2 prints ``1`` on Laptop B
2. ``pin-low 33`` on Laptop A → Device 2 prints ``0`` on Laptop B

Bonus: Automated bridge in Python

```
import serial
import threading

port_a = serial.Serial('/dev/ttyUSB0', 115200, timeout=0.1) # cnc device
port_b = serial.Serial('/dev/ttyUSB1', 115200, timeout=0.1) # reporter device

def read_reporter():
    while True:
        line = port_b.readline().decode('utf-8', errors='ignore').strip()
        if line:
            print(f"[Device B] GPIO level: {line}")

threading.Thread(target=read_reporter, daemon=True).start()

print("Commands: 'high', 'low', 'quit'")
while True:
    cmd = input("> ").strip().lower()
    if cmd == 'high':
        port_a.write(b'pin-high 33\n')
    elif cmd == 'low':
        port_a.write(b'pin-low 33\n')
    elif cmd == 'quit':
        break

port_a.close()
port_b.close()
```

Solutions — Exercises 6: Sampling Physical Phenomena

Quest 1: Seeing Potential

Which type should hold a raw ADC result? `int` — the ESP32 ADC returns values in the range 0-4095 (12-bit). `int` (32-bit signed) is more than sufficient and is the type returned by `adc_oneshot_read()`.

sample() function and ADC setup:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_adc/adc_oneshot.h"

#define ADC_CHANNEL  ADC_CHANNEL_6    // GPIO34 on ESP32 DevKit
#define ADC_UNIT      ADC_UNIT_1

static adc_oneshot_unit_handle_t adc_handle;

void adc_init(void) {
    adc_oneshot_unit_init_cfg_t unit_cfg = {
        .unit_id = ADC_UNIT,
        .ulp_mode = ADC_ULP_MODE_DISABLE,
    };
    adc_oneshot_new_unit(&unit_cfg, &adc_handle);

    adc_oneshot_chan_cfg_t chan_cfg = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,    // 12-bit
        .atten    = ADC_ATTEN_DB_11,        // 0-3.9V input range
    };
    adc_oneshot_config_channel(adc_handle, ADC_CHANNEL, &chan_cfg);
}

// Returns raw ADC value 0-4095
int sample(void) {
    int raw;
    adc_oneshot_read(adc_handle, ADC_CHANNEL, &raw);
    return raw;
}

void app_main(void) {
    adc_init();
    while (1) {
        printf("raw: %d\n", sample());
        fflush(stdout);
        vTaskDelay(200 / portTICK_PERIOD_MS);
    }
}
```

NOTE: ADC_ATTEN_DB_11 gives the widest input range (up to ~3.9V) at the cost of some accuracy at low voltages. For a potentiometer spanning the full 0-3.3V range this is the right choice.

Quest 2: Guessing Game

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/uart.h"
#include "esp_adc/adc_oneshot.h"

// (assume adc_init and sample() from Quest 1 are included)

#define UART_NUM    UART_NUM_0
#define BUF_SIZE    64
#define MAX_VALUE   100    // secret number range: 1-100
#define MAX_TRIES   7      // log2(100) ≈ 6.6, so 7 gives a fair binary search

static void uart_readline(char* buf, int maxlen) {
    int pos = 0;
    while (pos < maxlen - 1) {
        uint8_t c;
        if (uart_read_bytes(UART_NUM, &c, 1, portMAX_DELAY) <= 0) continue;
        if (c == '\r') continue;
        if (c == '\n') break;
        buf[pos++] = (char)c;
    }
    buf[pos] = '\0';
}

void app_main(void) {
    // uart_init() and adc_init() assumed already called
    char buf[BUF_SIZE];

    while (1) {
        printf("\nShall we play a game? (y/n): ");
        fflush(stdout);
        uart_readline(buf, sizeof(buf));
        if (buf[0] == 'n') esp_restart();
        if (buf[0] != 'y') continue;

        // Produce secret: scale ADC 0-4095 to 1-MAX_VALUE
        int secret = (sample() * MAX_VALUE / 4095) + 1;
    }
}
```

```

printf("Guess a number between 1 and %d. You have %d tries.\n",
      MAX_VALUE, MAX_TRIES);

bool done = false;
uint8_t count = 0;

while (!done) {
    printf("You have guessed %d %s. Please try again: ",
          count, count == 1 ? "time" : "times");
    fflush(stdout);

    uart_readline(buf, sizeof(buf));

    // Parse — handle non-numeric input
    char* end;
    long guess = strtol(buf, &end, 10);
    if (end == buf || guess < 1 || guess > MAX_VALUE) {
        printf("Invalid input. Enter a number between 1 and %d.\n", MAX_VALUE);
        continue;
    }

    count++;

    if ((int)guess == secret) {
        printf("You got it!\n");
        done = true;
    } else if ((int)guess < secret) {
        printf("Too low.\n");
    } else {
        printf("Too high.\n");
    }

    if (!done && count >= MAX_TRIES) {
        printf("the only winning move is not to play\n");
        done = true;
    }
}
}
}

```

Balance rationale: MAX_VALUE = 100 and MAX_TRIES = 7 allows an optimal binary-search player to always win ($\lceil \log_2(100) \rceil = 7$). A random guesser wins with probability roughly $1 - (99/100)^7 \approx 7\%$.

Quest 3: Getting Real — LMT86 Temperature Conversion

LMT86 formula (from datasheet Section 8.3.1):

$$V_{out} = -6.25 \cdot T + 1496 \quad (\text{mV})$$

Rearranging for temperature:

$$T = \frac{1496 - V_{out}}{6.25} \quad (C)$$

ADC → voltage: the ESP32 ADC at ADC_ATTEN_DB_11 maps 0-4095 to approximately 0-3900 mV. The conversion is:

$$V_{mV} = \frac{raw \times 3900}{4095}$$

Combined convert() function:

```
// Input: raw ADC value (0-4095, 12-bit)
// Output: temperature in C as int8_t
int8_t convert(int raw) {
    // Step 1: raw → millivolts (ADC_ATTEN_DB_11 → ~3900 mV full scale)
    // Use integer arithmetic; scale up to preserve precision before dividing
    int v_mv = (raw * 3900) / 4095;

    // Step 2: voltage → temperature (LMT86: T = (1496 - V_mv) / 6.25)
    // Multiply by 100 to avoid float, then divide by 625 (6.25 * 100)
    int t_x100 = (1496 - v_mv) * 100;
    int t_deg = t_x100 / 625;

    return (int8_t) t_deg;
}
```

Why preserve precision: if you compute $(1496 - v_mv) / 6$ first you lose the fractional part. Multiplying numerator by 100 and dividing by 625 keeps the result accurate to 0.5C before truncation.

WARNING: The LMT86 is polarity-sensitive. The flat side faces you; pins left-to-right are: V+ (3.3V), Vout (to ADC), GND. Reversed polarity destroys the sensor.

Expected raw values:

- Room temperature (~22C): $V_{out} \approx 1496 - 6.25 \times 22 \approx 1359$ mV → raw ≈ 1426
- Body temperature (~36C): $V_{out} \approx 1496 - 6.25 \times 36 \approx 1271$ mV → raw ≈ 1334
- Boiling water should NOT be tested — the sensor's max rating is 130C but the plastic housing may fail.

Quest 4: Cracks are Forming

Path B — Python scatter plot and heatmap

What you observe in the scatter plot: - Initially (sensor at room temperature): tight cluster of readings around a constant raw value. - After placing sensor on a hot/cold surface: readings shift quickly then asymptotically approach a new equilibrium — an exponential decay (Newton's law of cooling/heating). - Some noise/spread around the mean at each temperature — this is ADC quantisation noise + thermal noise + power supply ripple.

Heatmap implementation:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from pathlib import Path

# — Parameters —————
X_BINS = 100    # time buckets
Y_BINS = 50     # value buckets

def load_csv(path):
    df = pd.read_csv(path, names=['time', 'value'])
    return list(zip(df['time'], df['value']))

def ts2heatmap(pairs):
    times = [p[0] for p in pairs]
    values = [p[1] for p in pairs]
    t_min, t_max = min(times), max(times)
    v_min, v_max = min(values), max(values)
    t_range = t_max - t_min or 1
    v_range = v_max - v_min or 1

    heatmap = [[0] * Y_BINS for _ in range(X_BINS)]
    for t, v in pairs:
        x = min(int((t - t_min) / t_range * X_BINS), X_BINS - 1)
        y = min(int((v - v_min) / v_range * Y_BINS), Y_BINS - 1)
        heatmap[x][y] += 1

    return heatmap, t_min, t_max, v_min, v_max

def heatmap_max(heatmap):
    return max(max(row) for row in heatmap)

def gen_colormap(max_val):
    def color(count):
        intensity = count / max_val if max_val > 0 else 0
        r = int(255 * intensity)
        g = int(80 * intensity)
        b = int(255 * (1 - intensity))
        return f"#{r:02x}{g:02x}{b:02x}"
    return color

def plot_heatmap(pairs, outfile='heatmap.pdf'):
    hm, t_min, t_max, v_min, v_max = ts2heatmap(pairs)
    max_val = heatmap_max(hm)
    colormap = gen_colormap(max_val)

    t_step = (t_max - t_min) / X_BINS
    v_step = (v_max - v_min) / Y_BINS

```

```

fig, ax = plt.subplots(figsize=(12, 6))
for x in range(X_BINS):
    for y in range(Y_BINS):
        if hm[x][y] > 0:
            rect = patches.Rectangle(
                (t_min + x * t_step, v_min + y * v_step),
                t_step, v_step,
                linewidth=0,
                facecolor=colormap(hm[x][y])
            )
            ax.add_patch(rect)

ax.set_xlim(t_min, t_max)
ax.set_ylim(v_min, v_max)
ax.set_xlabel('Time [epoch]')
ax.set_ylabel('Raw ADC value')
ax.set_title('Temperature sensor heatmap')
plt.tight_layout()
plt.savefig(outfile)
print(f"Saved {outfile}")

```

```

pairs = load_csv('readings.csv')
plot_heatmap(pairs)

```

Heatmap vs scatter plot insights: - The scatter plot shows every individual point equally — dense regions look similar to sparse ones. - The heatmap reveals *density*: you can see where readings cluster most frequently (equilibrium temperature bands), the speed of transitions (wide vertical band = slow change, narrow = fast), and multimodal distributions (two stable temperatures during the experiment). - Scales poorly for very large datasets as a scatter plot (overplotting hides structure); the heatmap bins the data so it always renders in constant time regardless of point count.

Quest 5: Root-Causing Noise

What you observe: - On **wall power**: more noise in the ADC readings — higher variance, sometimes 50/60 Hz ripple visible as periodic spikes. - On **battery**: cleaner signal, lower noise floor.

Source of the difference: The AC mains supply introduces ripple into the laptop's DC power rail (even after regulation). This ripple couples into the ESP32's power supply via the USB cable. The ADC reference voltage (VREF) is nominally 1.1V but is derived from the internal bandgap; the ADC input stage is not fully isolated from supply noise. The result is that power-supply fluctuations add directly to the measured voltage, appearing as ADC noise.

A battery produces a much cleaner DC supply with no 50/60 Hz component. This is why precision analog measurements on embedded systems often use decoupling capacitors (0.1F + 10F in parallel on the power pin) to filter supply noise, and why ADC readings are averaged over multiple samples.

Quest 6: Laying Pipes — FreeRTOS Pipeline

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_adc/adc_oneshot.h"

// (adc_init and sample() from Quest 1 assumed available)

#define TX_BUFFER_SIZE 10

QueueHandle_t tx_queue;

void TaskSample(void* pvParameters) {
    QueueHandle_t output_queue = (QueueHandle_t) pvParameters;
    while (1) {
        int value = sample();
        while (xQueueSendToBack(output_queue, &value, 10) != pdTRUE);
    }
}

void TaskTransmit(void* pvParameters) {
    QueueHandle_t input_queue = (QueueHandle_t) pvParameters;
    while (1) {
        int value;
        while (xQueueReceive(input_queue, &value, 10) != pdPASS);
        printf("%d\n", value);
        fflush(stdout);
    }
}

void app_main(void) {
    adc_init();
    tx_queue = xQueueCreate(TX_BUFFER_SIZE, sizeof(int));
    xTaskCreate(TaskSample, "Sample", 4096, tx_queue, 1, NULL);
    xTaskCreate(TaskTransmit, "Transmit", 4096, tx_queue, 1, NULL);
}
```

How it works: - tx_queue is a ring buffer of 10 int slots managed by FreeRTOS. - TaskSample runs continuously, filling the queue; it blocks (retries) if the queue is full. - TaskTransmit blocks waiting for items; when one arrives it prints and loops. - FreeRTOS schedules both tasks on the same core (or across cores on ESP32's dual-core); the queue provides safe inter-task communication with no shared-variable race conditions.

Why a queue instead of a global variable? A global int shared between tasks would require a mutex to prevent a torn read/write (the sample task writes while the transmit task is mid-read). The queue provides atomic enqueue/dequeue, back-pressure (producer

blocks when full), and makes the data-flow direction explicit in the code structure.

Queue size trade-off: - Too small (e.g. 1): if TaskTransmit is briefly delayed (e.g. UART flush), TaskSample blocks and misses samples. - Too large: consumes more RAM. At `sizeof(int) = 4 bytes`, 10 entries costs only 40 bytes — negligible.

Solutions — Exercises 7: Radio — That Old Wireless Magic

Quest 1: Establish an Access Point

SSID and **password** are environment-specific — note them down before proceeding. The key requirement is that the device running the logger (your laptop) and the ESP32 are on the *same* subnet so ping and TCP both route correctly.

Firewall note (Linux):

```
# Allow inbound on port 8000 (adjust interface as needed)
sudo ufw allow 8000/tcp
# Or disable the firewall temporarily for testing:
sudo ufw disable
```

Firewall note (macOS): System Preferences → Security & Privacy → Firewall → Firewall Options → uncheck “Block all incoming connections”, or add your socket-dumper binary as an allowed application.

Quest 2: Obtaining an IP Address

How SSID/password make it into sdkconfig

sdkconfig is generated/updated by `idf.py menuconfig`. The `Kconfig.projbuild` file in `main/` declares project-specific configuration items using Kconfig syntax. When `menuconfig` runs it reads all Kconfig and `Kconfig.projbuild` files in the component tree and presents a menu; the chosen values are written to `sdkconfig` as `CONFIG_*` macros.

Kconfig.projbuild role: - Declares `EXAMPLE_WIFI_SSID` and `EXAMPLE_WIFI_PASSWORD` as string config items with defaults. - `menuconfig` exposes them under *Example Configuration*. - The build system generates `sdkconfig.h` with `#define CONFIG_EXAMPLE_WIFI_SSID "..."` which `station_example_main.c` includes.

Refactored `hl_wifi.h`

```
#ifndef HL_WIFI_H
#define HL_WIFI_H

#include <stdint.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <errno.h>

typedef void (*connect_callback_t)(void);
typedef struct sockaddr_in sockaddr_in_t;

void hl_wifi_init(connect_callback_t* callback);
```

```

sockaddr_in_t    hl_wifi_make_addr(char* ip, uint16_t port);
int              hl_wifi_tcp_connect(sockaddr_in_t addr);
void             hl_wifi_tcp_tx(int sock, void* buffer, uint16_t length);

#endif /* HL_WIFI_H */

```

hl_wifi.c skeleton (WiFi init)

```

#include "hl_wifi.h"
#include <string.h>
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"
#include "esp_wifi.h"
#include "esp_event.h"
#include "esp_log.h"
#include "nvs_flash.h"

#define WIFI_SSID      CONFIG_EXAMPLE_WIFI_SSID
#define WIFI_PASS      CONFIG_EXAMPLE_WIFI_PASSWORD
#define WIFI_CONNECTED_BIT BIT0

static EventGroupHandle_t s_wifi_event_group;
static connect_callback_t* s_callback;

static void event_handler(void* arg, esp_event_base_t event_base,
                        int32_t event_id, void* event_data)
{
    if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();
    } else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_DISCONNECTED) {
        esp_wifi_connect();
    } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t* event = (ip_event_got_ip_t*) event_data;
        printf("Got IP: " IPSTR "\n", IP2STR(&event->ip_info.ip));
        xEventGroupSetBits(s_wifi_event_group, WIFI_CONNECTED_BIT);
        if (s_callback && *s_callback) {
            connect_callback_t callback = *s_callback;
            callback();
        }
    }
}

static void wifi_init_sta(connect_callback_t* callback)
{
    s_callback = callback;
    s_wifi_event_group = xEventGroupCreate();

    ESP_ERROR_CHECK(esp_netif_init());

```

```

ESP_ERROR_CHECK(esp_event_loop_create_default());
esp_netif_create_default_wifi_sta();

wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
ESP_ERROR_CHECK(esp_wifi_init(&cfg));

esp_event_handler_instance_t instance_any_id;
esp_event_handler_instance_t instance_got_ip;
ESP_ERROR_CHECK(esp_event_handler_instance_register(WIFI_EVENT,
                                                    ESP_EVENT_ANY_ID,
                                                    &event_handler,
                                                    NULL,
                                                    &instance_any_id));
ESP_ERROR_CHECK(esp_event_handler_instance_register(IP_EVENT,
                                                    IP_EVENT_STA_GOT_IP,
                                                    &event_handler,
                                                    NULL,
                                                    &instance_got_ip));

wifi_config_t wifi_config = {
    .sta = {
        .ssid      = WIFI_SSID,
        .password   = WIFI_PASS,
    },
};
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
ESP_ERROR_CHECK(esp_wifi_start());

xEventGroupWaitBits(s_wifi_event_group, WIFI_CONNECTED_BIT,
                    pdFALSE, pdFALSE, portMAX_DELAY);
}

void hl_wifi_init(connect_callback_t* callback)
{
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ret = nvs_flash_init();
    }
    ESP_ERROR_CHECK(ret);
    wifi_init_sta(callback);
}

```

main/CMakeLists.txt:

```

idf_component_register(SRCS "main.c"
                       "hl_wifi.c"
                       INCLUDE_DIRS ".")

```

Quest 3: Uplink

Why buffer[7]?

uint16_t can hold values 0-65535 (5 digits) + newline \n + null terminator \0 = **7 bytes**. Without the null terminator sprintf would overflow; without the newline the logger cannot split lines.

Full TCP helper implementations

```
sockaddr_in_t hl_wifi_make_addr(char* ip, uint16_t port)
{
    sockaddr_in_t addr;
    addr.sin_addr.s_addr = inet_addr(ip);
    addr.sin_family      = AF_INET;
    addr.sin_port        = htons(port);
    return addr;
}

int hl_wifi_tcp_connect(sockaddr_in_t addr)
{
    int sock = socket(AF_INET, SOCK_STREAM, IPPROTO_IP);
    if (sock < 0) {
        printf("socket() failed: errno %d\n", errno);
        return -1;
    }
    int err = connect(sock, (struct sockaddr*)&addr, sizeof(struct sockaddr_in));
    if (err != 0) {
        printf("connect() failed: errno %d\n", errno);
        close(sock);
        return -1;
    }
    return sock;
}

void hl_wifi_tcp_tx(int sock, void* buffer, uint16_t length)
{
    uint16_t offset = 0;
    while (offset < length) {
        uint16_t remainder = length - offset;
        int sent = send(sock, (char*)buffer + offset, remainder, 0);
        if (sent < 0) {
            printf("send() failed: errno %d\n", errno);
            return;
        }
        offset += sent;
    }
}
```

TaskCount in main.c

```
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "hl_wifi.h"

#define LAPTOP_IP "192.168.1.100"    // replace with your laptop's IP
#define LAPTOP_PORT 8000

static void TaskCount(void* pvParameters)
{
    sockaddr_in_t addr = hl_wifi_make_addr(LAPTOP_IP, LAPTOP_PORT);
    int sock = hl_wifi_tcp_connect(addr);
    if (sock < 0) {
        printf("Failed to connect\n");
        vTaskDelete(NULL);
        return;
    }

    uint16_t counter = 1;
    char buffer[7];
    while (1) {
        sprintf(buffer, "%u\n", counter);
        hl_wifi_tcp_tx(sock, buffer, strlen(buffer));
        counter++;
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

static void connected_callback(void)
{
    xTaskCreate(TaskCount, "Count", 4096, NULL, 1, NULL);
}

void app_main(void)
{
    hl_wifi_init(&connected_callback);
}
```

Quest 4: Temperature Uplink

```
// In TaskCount (renamed TaskTemp), replace the counter loop with:

static void TaskTemp(void* pvParameters)
{
```

```

sockaddr_in_t addr = hl_wifi_make_addr(LAPTOP_IP, LAPTOP_PORT);
int sock = hl_wifi_tcp_connect(addr);
if (sock < 0) {
    printf("Failed to connect\n");
    vTaskDelete(NULL);
    return;
}

char buffer[16];
while (1) {
    int raw = sample();
    int temp = convert(raw);
    sprintf(buffer, "%d\n", temp);
    hl_wifi_tcp_tx(sock, buffer, strlen(buffer));
    vTaskDelay(1000 / portTICK_PERIOD_MS);
}
}

```

sample() and convert() are the functions written in Exercise 6 Quest 3. The overall project structure is:

```

temp2tcp/
├── main/
│   ├── CMakeLists.txt
│   ├── main.c           ← app_main, connected_callback, TaskTemp
│   ├── hl_wifi.c        ← WiFi + TCP helpers
│   └── hl_wifi.h
└── CMakeLists.txt

```

Solutions — Exercises 8: Radio Technology

Q1: Technology Comparison

Technology	Frequency Band	Typical Range	Typical Data Rate	Power Consumption	Primary IoT Use Case
WiFi (802.11n)	2.4 / 5 GHz	50-100 m (indoors)	150-300 Mbit/s	High (~100-500 mW active)	Video streaming, high-bandwidth sensors
Bluetooth LE (BLE)	2.4 GHz	10-100 m	125 kbit/s-2 Mbit/s	Very low (~0.01-15 mW)	Wearables, beacons, proximity sensing
Zigbee (802.15.4)	2.4 GHz	10-100 m per hop	250 kbit/s	Low (~30 mW)	Home automation, mesh sensor networks
LoRa / Lo-RaWAN	868 MHz (EU) / 915 MHz (US)	2-15 km	0.3-50 kbit/s	Very low (~40 mW TX)	Agriculture, smart metering, asset tracking
NB-IoT	Licensed LTE bands	Cellular (km)	~200 kbit/s	Low-medium	Smart meters, utility monitoring
Sigfox	868 MHz (EU)	10-50 km	100 bit/s	Very low	Low-frequency telemetry, alarms

Q2: Technology Selection

Scenario	Best Technology	Justification
Smoke detector, 10-year battery life	Zigbee or BLE	Extremely low duty cycle; sleep most of the time, only transmit on alarm. BLE Advertising or Zigbee End Device sleep modes make decade-scale battery feasible.
Live video from delivery drone	WiFi (802.11ac/ax)	High bandwidth required for video; drone is near a hotspot or uses a dedicated link.

Scenario	Best Technology	Justification
5,000 agricultural sensors over 50 km	LoRaWAN	Long range, low power, star topology scales to thousands of nodes per gateway.
Wearable heart rate monitor → phone	BLE	Short range, low power, standardised GATT profiles for health data, native support on all smartphones.
Smart electricity meter, hourly report	NB-IoT or LoRaWAN	Licensed spectrum (NB-IoT) for reliability and SLA, or LoRaWAN for lowest cost. Hourly data is very low bandwidth.
Real-time control of industrial robot	WiFi or wired (Ethernet)	Low latency (<1 ms) and high reliability needed; WiFi 802.11 with QoS or deterministic Ethernet (TSN).

Q3: Duty Cycle Regulations

a) Time-on-air = 500 ms. At 1% duty cycle the device may transmit 1% of the time. The *off* time must be at least:

$$t_{off} \geq \frac{t_{on}}{0.01} - t_{on} = \frac{500 \text{ ms}}{0.01} - 500 \text{ ms} = 50,000 - 500 = 49,500 \text{ ms} \approx 49.5 \text{ s}$$

b) Maximum packets per hour at 500 ms on-air with 1% duty cycle:

Total air time allowed per hour = 3600 s × 1% = 36 s.

Packets per hour = 36 s / 0.5 s = **72 packets/hour**.

c) At 100 ms on-air per packet, transmitting every 60 s:

Air time per hour = (3600/60) × 0.1 s = 60 × 0.1 = 6 s.

Allowed air time = 36 s. 6 s ≪ 36 s → **Yes, it complies** with significant headroom.

Q4: Spreading Factor (LoRa)

a) Higher SF increases time-on-air (and thus energy per packet) and reduces data rate. The trade-off is **range vs. throughput and power**. SF12 has ~15 dB more link budget than SF7 but takes ~40× longer to transmit the same payload.

b) At SF12, a packet takes 2000 ms on-air. With the 1% duty cycle this requires a mandatory quiet period of:

$$t_{off} \geq \frac{2000 \text{ ms}}{0.01} - 2000 \text{ ms} = 198 \text{ s} \approx 3.3 \text{ minutes}$$

This severely limits transmission frequency — at SF12 you can send at most ~18 packets/hour, compared to ~720/hour at SF7 (50 ms on-air). High SF consumes the duty cycle budget very quickly.

c) Assign lower SF to nearby sensors (they have sufficient link margin) and **higher SF only to distant sensors**. This minimises average time-on-air across the network and maximises total network capacity. ADR (Adaptive Data Rate) in LoRaWAN does this automatically by adjusting SF based on received signal strength.

Q5: Mesh vs. Star Topology

a) In a Zigbee mesh, each router node maintains a routing table. If a router fails, neighbouring nodes detect the broken link (via failed ACKs or periodic neighbour discovery). They update their routing tables and reroute traffic via alternate paths — the mesh self-heals without central coordination, typically within seconds.

b) In LoRaWAN's star-of-stars topology, end devices have no knowledge of each other. If the sole gateway goes offline, **no packets reach the network server** — the system is completely down for that coverage area. There is no mesh fallback. Redundancy must be provided by deploying multiple overlapping gateways.

c) Mesh (Zigbee) is more resilient for a factory floor with RF interference because it can route around failed or jammed nodes. WiFi interference is localised — the mesh finds paths that avoid congested channels. A star topology (LoRa, WiFi to AP) has a single point of failure at the central hub; if machinery interferes with the gateway-to-device link that node is lost.

Q6: Frequency Hopping

a) A fixed frequency is vulnerable to narrowband interference — a single jammer or co-channel device can block all communication. Frequency hopping spreads transmissions across many channels; interference on any one channel only disrupts a fraction of packets, which can be recovered by retransmission.

b) AFH maps the 79 BLE channels and identifies which are occupied by WiFi (high RSSI, high error rate). It then excludes those channels from the hop sequence, dynamically steering BLE away from WiFi channels. Both coexist in 2.4 GHz because BLE hops around the WiFi traffic rather than transmitting directly on top of it.

c) WiFi channel 6 occupies approximately 2.412–2.462 GHz (22 MHz wide). Zigbee channels are 5 MHz wide centred at:

- Channel 11: 2.405 GHz

- Channel 12: 2.410 GHz ← overlaps with WiFi ch 6 lower edge
- Channel 13: 2.415 GHz ← overlap

- ...
- Channel 22: 2.465 GHz ← upper edge

Avoid Zigbee channels 12-22 (approximately) to stay clear of WiFi channel 6. A safe choice is **Zigbee channels 25 or 26** (2.475 GHz and 2.480 GHz) which fall above the WiFi channel 6 upper band edge.

Q7: Security Considerations

a) LoRaWAN packets are **AES-128 encrypted** at the application layer using the AppSKey. Even though the payload travels over shared infrastructure (anyone's gateway can receive it), the ciphertext is unreadable without the key. The network server decrypts with NwkSKey and forwards the still-encrypted payload to the application server, which decrypts with AppSKey.

b) - NwkSKey (Network Session Key): protects the MAC layer — authenticates packets to the network server, enables MIC (Message Integrity Code) verification, and ensures only legitimate devices can communicate with the network. - **AppSKey (Application Session Key):** encrypts the application payload end-to-end between the device and the application server. The network operator never sees plaintext application data.

c) LoRaWAN uses a **frame counter (FCnt)** that increments with every uplink. The network server rejects any packet whose FCnt is not greater than the last accepted value. A replayed packet has the same (or lower) FCnt and is discarded.

Q8: Gateway Density

a) A circle of radius 15 km has area $\pi \times 15 \approx 707$ km. To cover 100 km with non-overlapping circles of this size, **1 gateway** is theoretically sufficient ($100 \text{ km} < 707 \text{ km}$). In practice you deploy ≥ 2 for redundancy.

b) At 5 km effective range, each gateway covers $\pi \times 5 \approx 78$ km. To tile 100 km you need at least:

$$\lceil 100/78 \rceil = \lceil 1.28 \rceil = 2 \text{ gateways minimum}$$

In a realistic hexagonal grid layout with overlap for reliability, **4-6 gateways** is a common rule of thumb for this area and range.

c) TDOA (Time Difference of Arrival) geolocation works as follows: multiple gateways receive the same uplink packet and each records the precise arrival timestamp. The network server computes the *difference* in arrival times between gateway pairs. Each pair defines a **hyperbola** of possible transmitter positions (the locus of points where the time difference is constant). With 3+ gateways you get ≥ 3 hyperbolas whose intersection gives the transmitter's 2D position. GPS-disciplined clocks in gateways provide the nanosecond-precision timestamps required (1 ns corresponds to ~ 30 cm of range error).

Solutions — Exercises 9: Stream Processing — Pushing and Transforming Data

Quest 1: Installing an MQTT Broker

Verification output: After running `mosquitto_pub -t mytopic -m "hello, world"`, Terminal 1 should print:

hello, world

If nothing appears, check that the broker is running (`systemctl status mosquitto` on Ubuntu) and that you are connecting to localhost (the default when `-h` is not specified).

Quest 2: Signal Generator

1. What you see when subscribing to outputs

The generator publishes synthetic time-series values on multiple topics simultaneously (e.g. simulated temperature and humidity for several rooms). Each message is a floating-point number. You see a continuous stream of numeric payloads arriving, interleaved across topics, at whatever publication interval is configured (e.g. every 500 ms).

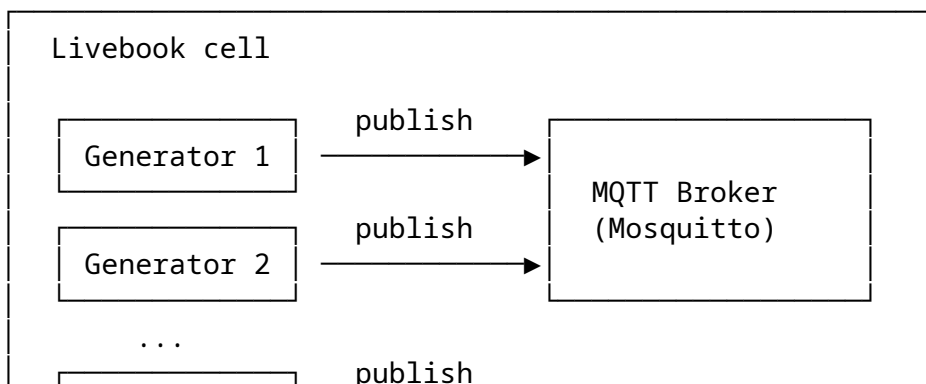
2. How concurrency is achieved

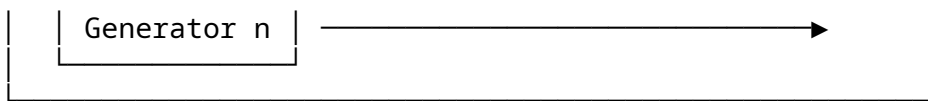
Each signal source is an independent **Elixir process** (lightweight actor). Livebook starts one process per configured signal, all running concurrently under the BEAM scheduler. There is no shared mutable state — each process owns its own counter/state and sends to MQTT independently.

3. Number of processes for n configurations

For n signal configurations there are **n generator processes** plus a small number of infrastructure processes (the Livebook cell process, MQTT client process(es)). The dominant count is **$n + \text{constant overhead} \approx n$ processes**.

4. Process and queue diagram





Each generator process has its own internal message queue (the BEAM mailbox). The MQTT broker's topic queues act as the inter-process communication channel between generators and any downstream subscribers.

Quest 3: Moving Averages

1. What you see when subscribing to outputs

The output topics contain smoothed versions of the raw generator signals. Rapid oscillations are dampened — only trends persist. The published values change more slowly than the raw input and never jump abruptly.

2. The “window”

The window is the fixed-size **sliding buffer** of the most recent k messages used to compute each average. Every new message shifts the window forward: the oldest value is dropped, the new value is added, and a fresh average is computed and published.

3. Unit of values within the window

The values are in the **same unit as the incoming signal** (whatever the generator publishes — e.g. raw ADC counts, C, or a unitless synthetic value). The moving average does not change units; it is dimensionally identical to the input.

4. Consequences of a lost message

A lost message means the window contains a “stale” value (the previous value remains at that slot, or the window advances without the missing reading). This biases the average toward the last known value. If messages are lost frequently the moving average lags the true signal. In pathological cases (many consecutive losses) the average freezes. There is no gap-filling or interpolation by default.

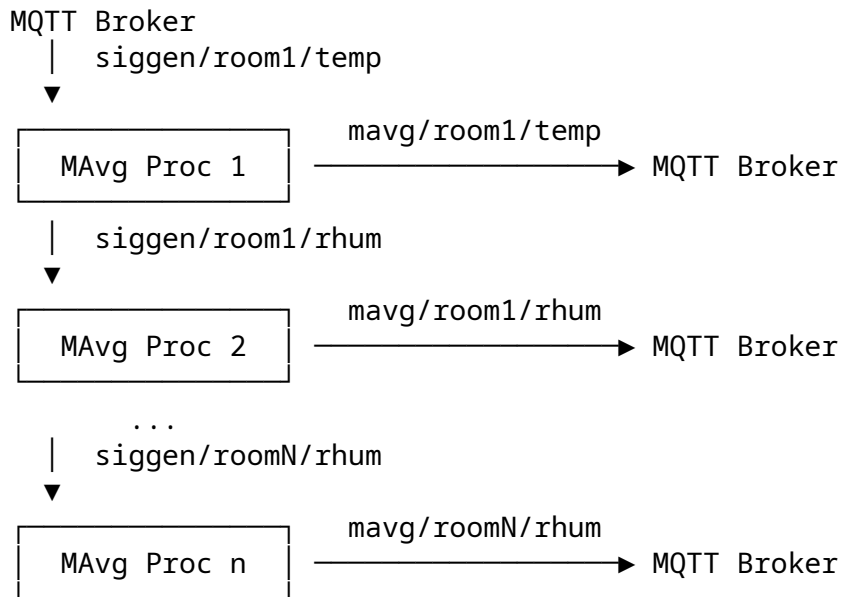
5. How concurrency is achieved

Same mechanism as Quest 2: one Elixir process per input topic/configuration. Each process subscribes, maintains its own window buffer state, and publishes independently. BEAM schedules all processes concurrently.

6. Number of processes for n configurations

n moving-average processes (one per configured input topic), plus the MQTT client process(es) and Livebook overhead. Again approximately **$n + \text{constant}$** .

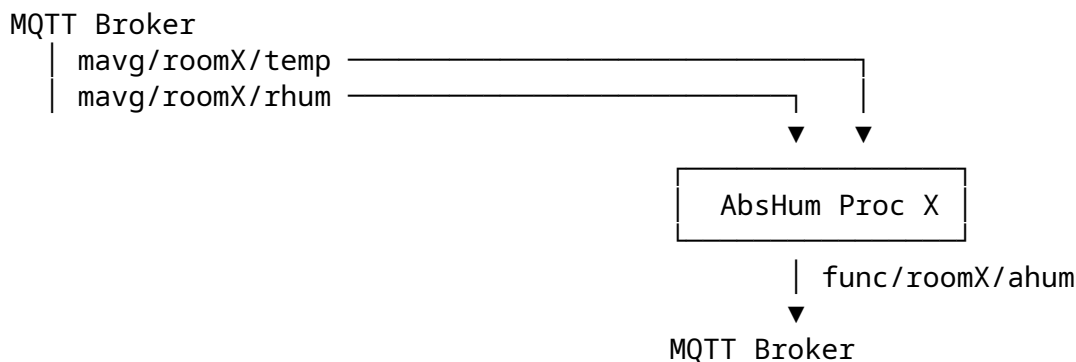
7. Process and queue diagram



Each MAVg process has an internal mailbox queue. The MQTT broker's topic subscriptions act as the inter-process queues connecting generators to averagers.

Quest 4: Achieving Absolute Humidity

1. Data flow diagram



One process per room. Each process subscribes to both `mavg/roomX/temp` and `mavg/roomX/rhum`, buffers the latest value for each, and re-publishes the computed absolute humidity whenever either value is updated.

2. Elixir implementation sketch

```
defmodule AbsHumidity do
  def start(room, broker_url) do
    {:ok, client} = Tortoise.Connection.start_link(
      client_id: "ahum_#{room}",
      server: {Tortoise.Transport.Tcp, host: broker_url, port: 1883},
      subscriptions: [{"mavg/#{room}/temp", 0}, {"mavg/#{room}/rhum", 0}]
    )
```

```

    )
    loop(client, room, nil, nil)
end

defp loop(client, room, temp, rhum) do
  receive do
    {:tortoise, {:publish, _topic_t, payload}} when String.contains?(_topic_t, "temp")
    t = String.to_float(payload)
    maybe_publish(client, room, t, rhum)
    loop(client, room, t, rhum)
    {:tortoise, {:publish, _topic_r, payload}} ->
    r = String.to_float(payload)
    maybe_publish(client, room, temp, r)
    loop(client, room, temp, r)
  end
end

defp maybe_publish(_client, _room, nil, _), do: :ok
defp maybe_publish(_client, _room, _, nil), do: :ok
defp maybe_publish(client, room, t, rh) do
  ahum = 6.112 * :math.exp((17.67 * t / (t + 243.5)) * rh * 2.1674 / (273.15 + t))
  Tortoise.publish(client, "func/#{room}/ahum", Float.to_string(ahum))
end
end

```

Verification: Cross-check against planetcalc.com/2167. At T=20C, RH=50%: H_abs ≈ 8.65 g/m.

3. How concurrency is achieved

One Elixir process per room, all running concurrently under the BEAM scheduler. Each process independently manages its state (latest temp and rhum values).

4. Number of processes for n rooms

n absolute-humidity processes + constant overhead ≈ n processes.

Quest 5: Topic Structure and Latency

Part A: Topic design

```

laptop/uptime                ← integer seconds since boot
sensors/<room>/temperature/celsius ← C readings
sensors/<room>/temperature/fahrenheit ← F (derived)

```

Differentiation: The metric name (uptime vs. temperature) and the unit suffix (celsius/fahrenheit) make each stream unambiguous. A subscriber for sensors/+/temperature/cel. gets all raw readings; sensors/+/temperature/fahrenheit gets all converted readings.

Part B: Timestamp logging

```
# In each subscriber/publisher, attach a monotonic timestamp:
timestamp = System.monotonic_time(:millisecond)
IO.puts("#{topic} @ #{timestamp}: #{payload}")
```

Part C: Discussion

1. Delay through the system

$$\text{delay} = t_{\text{publish_uptime}} \rightarrow t_{\text{broker_receipt}} + t_{\text{broker_forward}} + t_{\text{F subscriber_receipt}}$$

In practice: broker round-trip on localhost $\approx$ 1-5 ms; over LAN $\approx$ 5-50 ms. The Fahrenheit subscriber sees a two-hop delay: uptime source $\rightarrow$ broker $\rightarrow$ Celsius publisher $\rightarrow$ broker $\rightarrow$ Fahrenheit subscriber, so approximately **2 $\times$ single-hop latency**.

2. Adding a room humidity sensor

Add a new topic `sensors/<room>/humidity/relative`. No existing subscriptions are affected; the wildcard `sensors/+ /temperature/+` still only matches temperature topics. The structure scales cleanly.

3. Adding an outdoor temperature sensor

Use `sensors/outdoor/temperature/celsius`. It fits the existing hierarchy. The indoor-to-Fahrenheit converter can subscribe to `sensors/+ /temperature/celsius` and handle all rooms *including* outdoor with one subscription.

4. Moving a process to a friend's laptop

The MQTT broker must be reachable by all participants. If the broker stays local, latency increases by the round-trip time between your laptops (network RTT, typically 1-100 ms on LAN/WiFi). End-to-end latency becomes:

$$\text{delay} \approx 2 \times (t_{\text{local_hop}} + t_{\text{network_RTT}})$$

The formula doubles with each additional hop added across the network boundary.

Quest 6: Data Flows

When to choose which approach

Use **Elixir processes + MQTT** (the approach from Quests 1-5) when: - You need **real-time, low-latency** streaming with immediate per-message reactions. - The processing logic is **stateful per stream** (moving average, thresholding, join). - The deployment is **small to medium scale** with modest throughput (< millions of msgs/sec). - You want **lightweight infrastructure** without cluster management.

Use **Spark + Kafka** (batch/micro-batch stream processing) when: - You have **very high throughput** (millions of events/sec) requiring horizontal scaling. - You need **fault-tolerant, exactly-once** processing with replay. - The computation is **embarrassingly**

parallel and can tolerate some latency (micro-batch windows). - You have operational infrastructure for a Kafka cluster.

When to combine them

Combine both when you need **both real-time reactions and historical analysis**: MQTT/Elixir for the low-latency edge processing (anomaly alerts, device control), Kafka as a durable event log, and Spark for batch analytics (daily aggregations, ML training, compliance reporting). The bridge is a Kafka-MQTT connector or a custom publisher that writes MQTT messages to Kafka topics.

Quest 7: Chatty Room

Why char nick[10] constrains input

nick has 10 bytes: 9 usable characters + null terminator. Any nickname longer than 9 characters will be truncated (if read with fgets and a size limit) or cause a buffer overflow (if read naively with scanf). Always bound UART reads to sizeof(nick) - 1.

Key implementation notes

MQTT config:

```
esp_mqtt_client_config_t mqtt_cfg = {
    .broker.address.uri = BROKER_URL,
};
esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, mqtt_event_handler, NULL);
esp_mqtt_client_start(client);
```

Event handler:

```
static void mqtt_event_handler(void* handler_args, esp_event_base_t base,
                               int32_t event_id, void* event_data) {
    esp_mqtt_event_handle_t event = event_data;
    switch ((esp_mqtt_event_id_t)event_id) {
        case MQTT_EVENT_CONNECTED:
            esp_mqtt_client_subscribe(event->client, TOPIC, 1);
            xTaskCreate(TaskChat, "Chat", 4096, event->client, 1, NULL);
            break;
        case MQTT_EVENT_DATA:
            printf("Received [%.*s]: %.*s\n",
                  event->topic_len, event->topic,
                  event->data_len, event->data);
            break;
        default:
            printf("MQTT event id=%d\n", (int)event_id);
            break;
    }
}
```

TaskChat:

```
static void TaskChat(void* pvParameters) {
    esp_mqtt_client_handle_t client = (esp_mqtt_client_handle_t) pvParameters;
    char line[64];
    char buffer[80];

    while (1) {
        // Read one line from UART
        uart_readline(line, sizeof(line));
        snprintf(buffer, sizeof(buffer), "%s: %s\n", nick, line);
        esp_mqtt_client_publish(client, TOPIC, buffer, 0, 1, 0);
    }
}
```

NOTE: `uart_readline` blocks until a newline is received, so `TaskChat` is suspended until the user types. This is fine for interactive use but means no other work runs on this task in the meantime.

Solutions — Exercises 10: Performance Evaluation — Streaming Humidity

Quest 1: Test Harness

1. How to instrument an arbitrary command

The test harness wraps a target Elixir process (or module function) in a monitoring context. You provide the function/module to benchmark, and the harness spawns it while attaching BEAM process monitoring. In `record.livemd` you typically call a helper like:

```
Harness.run(fn -> MyModule.start(config) end, duration_ms: 10_000)
```

The harness records memory usage, message queue length, and reduction count (CPU proxy) for the instrumented process at configurable intervals, writing results to an in-memory table or CSV.

2. What happens when the instrumented process forks

If the process spawns a child (equivalent to forking), the harness only instruments the **root process** by default — the child is not automatically tracked. You must either pass the child PID to the harness explicitly, or configure recursive process tracking (if supported). Child processes' resource consumption is invisible unless you explicitly monitor them, which can undercount total system load.

3. How to read resulting data after a test run

The harness accumulates samples in an in-memory list (or ETS table) during the run. After the test completes you call a read function such as:

```
results = Harness.results()
```

This returns a list of `{timestamp, memory_bytes, message_queue_len, reductions}` tuples. You can pipe them into Explorer or VegaLite for plotting, or export to CSV with NimbleCSV.

Quest 2: Initial Setup

Verification

After starting all components you should see output on `mosquitto_sub -t "func/room1/ahum" -t "func/room2/ahum"` — absolute humidity values changing over time as the signal generator feeds synthetic temp/rhum through the moving average and absolute-humidity function.

Deviation between signal generator output and final absolute humidity values

The deviation comes from two sources:

Smoothing lag: the moving average introduces a phase lag proportional to the window size. For a window of k samples at publication interval Δt , the effective lag is approximately $k\Delta t/2$. The final ahum value lags the “true” instant ahum that would be computed from the raw signal by this amount.

Quantisation: the moving average is computed in floating-point; there is negligible numerical error.

In practice, with the default signal generator parameters (small window, short interval), the deviation between raw-instant and smoothed-then-converted ahum is typically a few percent of the signal amplitude, depending on how fast the synthetic signals change.

Quest 3: End-to-End Latency

Design questions (answered before running)

1. What does end-to-end latency mean in this system?

The time elapsed between a value being *published by the signal generator* on a `siggen/roomX/*` topic and the corresponding computed absolute humidity *being published* on `func/roomX/ahum`. It is the total pipeline delay from source to sink.

2. Form of the result

Not a single number — latency varies message to message due to OS scheduling, broker queuing, and BEAM scheduler preemption. The result is a **distribution** (histogram or empirical CDF). A CDF is most informative: it shows median, tail latency (p95, p99), and the worst-case observed latency.

3. System boundaries

- **Ingress:** timestamp when the signal generator calls `publish` on the MQTT client (before the packet leaves the process).
- **Egress:** timestamp when the absolute humidity subscriber receives the message from the broker.

Both timestamps must be captured by the same clock to avoid clock-skew error.

4. Relevant services

- Signal generator processes ($n \times 2$ per room: temp + rhum)
- MQTT broker (Mosquitto)
- Moving average processes ($n \times 2$ per room)
- Absolute humidity function processes (n per room)

5. Metrics

- Per-message end-to-end latency (ms)
- Throughput (messages/sec at each stage)
- Message queue length at each process over time (backpressure indicator)
- CPU (reductions) per process

6. Parameters and factors

Parameters (held constant): MQTT broker type (Mosquitto), number of rooms (2), window size (default), machine hardware, OS.

Factors (varied): publication interval (the rate at which the signal generator emits), window size of the moving average. Each combination is one experiment cell.

7. Workload

Synthetic workload — the signal generator produces artificial data. This is appropriate because we control the rate precisely and can repeat the experiment identically. Replay of captured real data would introduce external variability.

8. Experimental design

A **full factorial design** over the two factors: publication interval $\in \{100 \text{ ms}, 500 \text{ ms}, 1000 \text{ ms}\} \times$ window size $\in \{3, 7, 15\}$. That is $3 \times 3 = 9$ cells. Each cell runs for 120 seconds after a 10-second warm-up period. Repeat each cell 3 times and take the aggregate distribution.

9. Which components to instrument

Instrument **the signal generator output** and **the absolute humidity subscriber**. Instrumenting every intermediate stage adds overhead that distorts the measurement. Correlate messages using a unique sequence number embedded in the payload.

10. Post-processing

1. Join generator publish events and subscriber receive events by sequence number.
2. Compute per-message latency = receive_time – publish_time.
3. Aggregate into a distribution (numpy histogram or Elixir statistics library).
4. Compute percentiles: p50, p95, p99, max.

11. Visualization

Empirical CDF (x: latency in ms, y: fraction of messages $\leq x$). CDF is preferred over a histogram because it shows tail latencies clearly without bin-size sensitivity, and multiple experiment cells can be overlaid for direct comparison. One CDF line per (interval, window) combination.

12. Conclusion

Expected finding: latency increases as publication interval *decreases* (higher throughput $\rightarrow$ longer broker and process queues $\rightarrow$ more waiting). Larger window sizes have a negligible effect on latency (the averaging computation is $O(1)$ per message with the circular buffer approach) but increase the *lag* (systematic delay between true and smoothed value). The system is likely latency-dominated by MQTT broker round-trips ($\sim 1\text{--}5$ ms on localhost) rather than computation, so median latency should be in the low-millisecond range under moderate load.

WARNING: Ensure repeatability: flush broker retained messages between runs, use a fresh Livebook session for each cell, and record OS load average and BEAM scheduler utilisation during each run.

Solutions — Exercises 11: Budget Management — Getting the Most for Your Money

Quest 1: Running Average

What the buffer update operations accomplish

```
int old = buffer[index];    // remember the value being evicted from the window
buffer[index] = new;        // overwrite it with the new sample
sum -= old;                 // remove evicted value from running sum
sum += new;                 // add new value to running sum
index = (index + 1) % AVG_WINDOW_SIZE; // advance circular index
```

This is a **circular buffer** implementation of a sliding window. Rather than recomputing the sum of all k elements every iteration ($O(k)$), the running sum is maintained incrementally in $O(1)$ by adding the incoming value and subtracting the outgoing value. The modulo index wraps around so the array is reused in-place.

Complete TaskAvg

```
#include <string.h> // memset

#define AVG_WINDOW_SIZE 7

void TaskAvg(void* pvParameters)
{
    stage_interface_t* pair = (stage_interface_t*) pvParameters;
    QueueHandle_t input_queue = pair->input;
    QueueHandle_t output_queue = pair->output;

    int buffer[AVG_WINDOW_SIZE];
    memset(buffer, 0, AVG_WINDOW_SIZE * sizeof(buffer[0]));
    uint8_t index = 0;
    int sum = 0;

    // TODO: consume first AVG_WINDOW_SIZE-1 messages (see Bonus)

    while (1) {
        int new_val;
        while (xQueueReceive(input_queue, &new_val, 10) != pdPASS);

        int old = buffer[index];
        buffer[index] = new_val;
        sum -= old;
        sum += new_val;
        index = (index + 1) % AVG_WINDOW_SIZE;
    }
}
```

```

        int value = sum / AVG_WINDOW_SIZE;
        while (xQueueSendToBack(output_queue, &value, 10) != pdTRUE);
    }
}

```

Pipeline wiring in app_main:

```

QueueHandle_t avg_queue;
QueueHandle_t tx_queue;
stage_interface_t avg_pair;

avg_queue = xQueueCreate(AVG_BUFFER_SIZE, sizeof(int));
tx_queue = xQueueCreate(TX_BUFFER_SIZE, sizeof(int));
avg_pair = (stage_interface_t){avg_queue, tx_queue};

xTaskCreate(TaskSample, "Sample", 4096, avg_queue, 1, NULL);
xTaskCreate(TaskAvg, "Avg", 4096, &avg_pair, 1, NULL);
xTaskCreate(TaskTransmit, "Transmit", 4096, tx_queue, 1, NULL);

```

Is it always a good idea to put running-average in a separate task?

Not always. A separate task is beneficial when: - The upstream (sample) task runs at a different rate than downstream (transmit). - You want to decouple them with a queue so neither blocks the other.

It adds overhead when: - The pipeline runs at a fixed, synchronous rate (one sample → one output). In that case a simple function call in-line with the sample task would save the task context-switch and queue overhead with no loss of concurrency. - RAM is tight: each FreeRTOS task requires a stack (4096 bytes here) plus the queue storage.

Bonus: consuming the first AVG_WINDOW_SIZE - 1 messages

```

// Replace the TODO comment with:
for (int i = 0; i < AVG_WINDOW_SIZE - 1; i++) {
    int val;
    while (xQueueReceive(input_queue, &val, portMAX_DELAY) != pdPASS);
    buffer[index] = val;
    sum += val;
    index = (index + 1) % AVG_WINDOW_SIZE;
}

```

Why this is attractive: without the warm-up phase, the first $AVG_WINDOW_SIZE - 1$ outputs are computed from a partially-filled buffer (the rest is zero). This produces a systematic downward bias in the initial outputs. Pre-filling the buffer with real samples means the first published output already represents a valid window, eliminating the transient start-up artefact.

Quest 2: Thresholding

Complete TaskThreshold

```
#define THRESHOLD 100

void TaskThreshold(void* pvParameters)
{
    stage_interface_t* pair = (stage_interface_t*) pvParameters;
    QueueHandle_t input_queue = pair->input;
    QueueHandle_t output_queue = pair->output;

    int last = 1 << 16; // initialise above any valid ADC reading

    while (1) {
        int current;
        while (xQueueReceive(input_queue, &current, 10) != pdPASS);

        if (abs(current - last) > THRESHOLD) {
            while (xQueueSendToBack(output_queue, &current, 10) != pdTRUE);
            last = current;
        }
    }
}
```

Pipeline wiring additions in app_main:

```
QueueHandle_t thres_queue;
stage_interface_t thres_pair;

thres_queue = xQueueCreate(THRES_BUFFER_SIZE, sizeof(int));
avg_pair = (stage_interface_t){avg_queue, thres_queue};
thres_pair = (stage_interface_t){thres_queue, tx_queue};

xTaskCreate(TaskThreshold, "Threshold", 4096, &thres_pair, 0, NULL);
```

Why the first test may fail — the watchdog timer

With all tasks at priority 1, the **FreeRTOS idle task** (priority 0) never gets CPU time if any task is always ready to run. The ESP32's **Task Watchdog Timer (TWDT)** monitors whether the idle task has run within a timeout period (default 5 seconds). If the idle task starves, the TWDT fires and resets the chip with a "Task watchdog got triggered" message.

In the threshold pipeline, TaskSample and TaskAvg both spin in tight loops with short queue timeouts (10 ticks). If the sample rate is fast enough they never yield long enough for the idle task to run.

Fix: set all task priorities to 0

When all tasks run at priority 0 they share the same priority level as the idle task. FreeRTOS round-robin schedules them equally, ensuring the idle task runs regularly and feeds the

TWDT.

NOTE: In a real application you would instead call `esp_task_wdt_reset()` from the idle hook, or increase the TWDT timeout, rather than running all tasks at the lowest priority. Priority 0 prevents you from using priority-based preemption to give time-critical tasks more CPU.

How thresholding helps with radio budget

Without thresholding, **every sample** is transmitted — one TCP/MQTT packet per sample period. With thresholding, a packet is only sent when the signal changes by more than THRESHOLD ADC counts. In a stable environment (e.g. temperature changing slowly) this can reduce transmissions by **90% or more**. Fewer transmissions directly mean:

- Less radio-on time → lower average power consumption.
- Fewer packets → lower data volume → lower cellular/LoRaWAN airtime costs.
- Longer battery life for the radio module.

The trade-off is that the receiver only learns about changes, not the absolute value at every instant. Applications must handle sparse, event-driven updates rather than a regular stream.

Solutions — Exercises 12: Annotating Identity, Time and Space

Q1: The Three Annotation Axes

a) Definitions

- **Identity:** which device or entity produced this measurement.
- **Time:** when the measurement was taken (or the event occurred).
- **Space:** where the device was located when the measurement was taken.

b) What is missing from a bare value

A temperature of 21.3 C without any annotation tells you nothing actionable: you do not know which sensor in which location recorded it, when it was recorded, or where that sensor is. You cannot tell whether it is an indoor reading or outdoor, whether it is current or from a month ago, or whether two readings come from the same or different devices.

c) Most critical axis per data point

- **Fire alarm trigger:** *Time* is most critical. You need to know exactly when the alarm fired to correlate with other events, trigger evacuation procedures, and meet regulatory logging requirements. A few seconds of timestamp error could affect insurance or legal outcomes.
 - **GPS breadcrumb trail:** *Space* is most critical. The entire value of a breadcrumb trail is the sequence of positions — without spatial annotation there is nothing left.
 - **Smart meter energy reading billed per hour:** *Time* is most critical. Billing depends on knowing which tariff period applies; a wrong timestamp could misallocate cost between peak and off-peak rates.
-

Q2: Identity Sources

Source	Example	When it breaks down
Unique device number (MAC, serial)	ESP32 MAC address AA:BB:CC:DD:EE:FF	MAC spoofing; factory provisioning errors producing duplicate serials; chip replacement without re-registration
User-provided name	“living_room_sensor”	User renames the device; two users give different devices the same name; typos
Location-derived	“sensor at shelf B3”	Sensor is physically moved without updating the location record
Network-derived (IP address)	DHCP-assigned 192.168.1.42	IP address changes on reconnect (DHCP lease expiry); NAT hides the original address; multiple devices behind one NAT
Context-derived	“the sensor that last reported 18C when all others read 22C”	Context changes; another sensor fails and takes on the anomalous role; ambiguous under normal variation

Q3: Identity Assignment Points

a) Three main assignment points

1. **Node (device itself):** identity is embedded in firmware or stored in non-volatile memory on the device; the device self-annotates every message.
2. **Private (gateway or on-premise server):** a local gateway or broker maps device identifiers (e.g. MAC address) to logical names before forwarding to the cloud.
3. **Cloud (application server):** the cloud tier maps incoming connection identifiers (IP, client ID) to logical device identities in a registry.

b) Problems with cloud-tier assignment in a 500-sensor building

- *Physical move:* the cloud registry maps sensor ID → room. If a sensor moves but the registry is not updated, all readings from that sensor are attributed to the wrong room. There is no automatic way for the cloud to detect the move.
- *Sensors swapped during maintenance:* readings from two rooms are silently swapped in the database. This is especially insidious because both sensors continue to report valid-looking data.
- *Network offline, buffered readings:* when connectivity is restored, the cloud server receives a burst of readings with device-generated timestamps. If the cloud assigns identity based on connection-time metadata (e.g. IP address, session token) rather than a persistent device identifier, buffered messages may be mis-attributed or rejected.

c) Advantage of node-level identity assignment

Each message carries its identity from the moment it is created. There is no risk of mis-attribution during transit, buffering, or when multiple messages arrive out of order. Identity is an immutable property of the data regardless of which gateway or cloud endpoint receives it.

Q4: Timestamping

a) Three timestamping points

1. **At the sensor (event time):** closest to the true event; requires a synchronised real-time clock on the device.
2. **At the gateway (transit time):** when the gateway receives the packet; adds network + queuing latency.
3. **At the application server (receipt time):** when the cloud stores the record; adds further network and processing latency.

Most accurate: sensor event time (closest to the physical event), provided the sensor clock is synchronised.

Most practical: server receipt time — no special hardware needed; every server has a clock. Acceptable for low-frequency, latency-tolerant applications.

b) Timestamp error from a 47-second WiFi delay

The sensor event occurred ~47 seconds before the server received the message. The stored timestamp (server receipt time) is therefore **47 seconds later than the true event time**. In a system sampling once per minute, this error is almost an entire sample period — the

reading could be attributed to the wrong minute bucket, invalidating time-series alignment with other sensors that did not experience the delay.

c) Why an unsynchronised internal clock drifts

Even a correctly set clock at provisioning accumulates **crystal oscillator drift** — typically 10–100 ppm (parts per million). At 50 ppm, a device running for 1 year drifts by:

$$50 \times 10^{-6} \times 365 \times 24 \times 3600 \approx 1575 \text{ seconds} \approx 26 \text{ minutes}$$

Without periodic resynchronisation (NTP, GPS, or a network-provided timestamp), the sensor's clock diverges from wall time continuously. Long-running deployments can accumulate hours of error, making time-series correlation across devices meaningless.

Q5: Time Sources

a) Ranking by accuracy (best first)

1. **GNSS (GPS)** — ~100 ns accuracy (UTC-traceable, atomic clock-derived)
2. **NTP (LAN)** — ~1 ms accuracy (low round-trip jitter on local network)
3. **NTP (internet)** — ~10–50 ms accuracy (variable internet RTT)
4. **LoRaWAN network server timestamp** — ~10–100 ms (gateway GPS-disciplined, but adds processing latency)
5. **Device RTC without sync** — seconds to minutes drift over weeks/months
6. **Application server receipt time** — worst; adds variable network and processing delay on top of RTC drift

b) Selective Availability (SA)

SA was the US Department of Defense's deliberate degradation of civilian GPS accuracy by adding controlled dithering to satellite clock signals and ephemeris data. Before SA was disabled in May 2000, civilian GPS horizontal accuracy was ~100 m; after, it dropped to ~10–20 m. For historical IoT deployments that used GPS timestamping before 2000, the timing accuracy was also degraded proportionally (clock dithering affects both position and time). Any archived data from GPS-timestamped devices before May 2000 should be treated as having coarser time resolution.

c) Recovering approximate absolute timestamps from a free-running counter

The gateway records the absolute arrival time of each packet (GPS-disciplined). The sensor transmits its internal free-running counter value (or sequence number) with each packet. After collecting data you can:

1. Map counter values to absolute timestamps using the gateway's arrival times as anchors.
2. The counter increments at approximately 1/hour; gateway arrival time gives the wall-clock time for each counter tick.
3. Interpolate between gateway-timestamped packets to assign absolute times to intermediate readings.

Accuracy depends on: the stability of the counter (crystal drift), the density of received packets (more packets = better calibration), and the jitter of the LoRaWAN network server

timestamp. In practice this gives absolute timestamps accurate to within a few seconds over hours, degrading with longer gaps between received packets.

Q6: Space — Position Sources

a) Direct vs. indirect position sources

Direct — the device measures its own position: - GPS/GNSS receiver on the device - Surveyed fixed installation (position entered manually at deployment)

Indirect — position is inferred from signals not intended primarily for positioning: - WiFi RSSI trilateration (signal strength from known APs) - Cell-tower-based localisation (Cell-ID, OTDOA)

b) Sensor fusion for indoor/outdoor continuity

- **Outdoors:** use GPS as primary position. Confidence is high when HDOP < 2 and ≥ 4 satellites are visible.
- **Transitioning indoors:** GPS signal degrades (multipath, signal loss). Switch to WiFi RSSI trilateration using the three known APs. Kalman filter fuses WiFi position estimates with IMU dead reckoning.
- **Deep indoors:** GPS unavailable. Primary = WiFi trilateration; secondary = IMU dead reckoning from last known position. The floor plan constrains movement (a robot cannot walk through walls) — use a particle filter with the floor plan as a constraint map.
- **IMU drift:** accelerometer and gyroscope drift over time; periodically correct using WiFi position fix.

c) Trilateration vs. triangulation

Trilateration determines position from *distances* to known reference points. Each reference gives a circle (sphere in 3D) of possible positions; the intersection of three circles gives a unique 2D point. GPS and WiFi RSSI-based localisation use trilateration.

Triangulation determines position from *angles* (bearings) to known reference points. Each reference gives a line of bearing; two lines intersect to give position. Classic surveying and some direction-finding radio systems use triangulation.

The key difference: trilateration needs *range* measurements; triangulation needs *angle* measurements.

Q7: Precision vs. Accuracy in Space

a) Decimal places $\neq$ accuracy

Precision refers to the *resolution* of a measurement (how many significant figures are reported). Accuracy refers to *how close the value is to the true value*. A GPS reporting 7 decimal places (≈ 1 cm resolution) can still be *inaccurate* by 5–10 metres due to satellite geometry, atmospheric delay, multipath, and receiver noise. The extra decimal places are false precision — they do not reflect true positional accuracy. Consumer GPS accuracy is typically 3–10 m under open sky regardless of the number of decimal places reported.

b) Cell-ID localisation acceptability

Acceptable for: - Fleet management (is this truck in city A or city B?) - Coarse asset tracking (which warehouse is this pallet in?) - Presence detection (is this device on-site or off-site?) - Statistical analysis (which region did most trips originate from?)

Not acceptable for: - Indoor navigation (room-level or aisle-level location) - Mapping or geofencing with boundaries < 1 km - Delivery confirmation to a specific address - Any application requiring sub-100 m resolution

c) HDOP

HDOP (Horizontal Dilution of Precision) is a dimensionless multiplier that captures the effect of satellite geometry on position accuracy. It is computed from the geometry of the visible satellites relative to the receiver. When satellites are spread across the sky the geometry is good (HDOP ≈ 1); when clustered near the horizon or in one direction the geometry is poor (HDOP $\gg 1$).

$$\sigma_{position} = HDOP \times \sigma_{ranging}$$

A receiver with 3 m ranging error and HDOP = 1 achieves 3 m position accuracy; the same receiver with HDOP = 5 achieves only 15 m. Low HDOP (< 2) indicates good geometry; HDOP > 6 means position fixes are unreliable.

Q8: Full Annotation Pipeline — Agricultural Soil Moisture System

Identity

Where/how assigned: at the node. Each sensor is pre-programmed with a unique device ID (e.g. a serial number or LoRaWAN DevEUI) stored in firmware flash. Every uplink payload includes this ID.

Source: factory-provisioned DevEUI or a custom 32-bit serial written during firmware flashing.

Failure modes: - Duplicate IDs if the provisioning tool has a bug. - ID stored in unprotected flash could be overwritten by a firmware bug. - A sensor is replaced with a new unit that has a different DevEUI — the cloud dashboard must re-associate the new ID with the same field location.

Detection/recovery: maintain a registry mapping DevEUI $\rightarrow$ field location. Alert when a new DevEUI appears that is not in the registry. Cross-check expected sensor count per gateway vs. observed count.

Time

Where/how assigned: at the LoRaWAN network server. Each uplink is timestamped when the gateway receives it. Optionally, the device embeds a sequence number in the payload for ordering.

Source: gateway's GPS-disciplined clock (nanosecond-accurate gateway timestamps, millisecond-accurate at the network server after processing).

Failure modes: - Gateway clock loses GPS lock → falls back to internal RTC which drifts.
- Network server clock skew if NTP sync is interrupted. - Buffered packets (device out of range for several hours) arrive with reception timestamps much later than the true measurement time.

Detection/recovery: monitor gateway GPS lock status. Compare sequence numbers to detect gaps (buffered messages). Flag readings where arrival timestamp is implausibly far from neighbouring readings. Use sequence numbers to reconstruct correct ordering even when absolute timestamps are uncertain.

Space

Where/how assigned: at deployment time, a surveyor records the GPS coordinates of each sensor and stores them in the cloud registry keyed by DevEUI. The dashboard looks up coordinates from the registry when rendering the map.

Source: direct GPS measurement at installation (survey-grade or consumer GPS, depending on required grid resolution).

Failure modes: - Sensor physically moved (e.g. by farm machinery) without updating the registry — readings shown at wrong map location. - GPS survey error at installation (consumer GPS 5 m, which may be acceptable for a 10 km field at ~50 m sensor spacing).
- Registry entry for the wrong DevEUI (typo during provisioning).

Detection/recovery: periodically audit field sensor positions (visual inspection or rover GPS survey). Anomaly detection: a sensor showing systematically different moisture than all its spatial neighbours may have moved. Require a sign-off confirmation when any sensor position record is updated in the registry.

Re-exam 2025 — Model Answers

NOTE: Exam guidance Short answers: ~4 lines, max 10. Detailed answers: 10–15 lines, max 30. Total max 10 000 characters / 3 pages. Avoid padding — irrelevant content costs marks.

Question 1 (short) — What is a C “union”?

A union is a data type in C where all member fields share the **same block of memory**. The total size of the union equals the size of its **largest member**. Reading a member that was not the last one written yields a reinterpretation of those same bytes under a different type. This is used when you want to treat a piece of memory as different types at different times — for example, reading a 32-bit integer either as a float or as four `uint8_t` bytes.

```
typedef union {  
    float    reading;    // 4 bytes  
    uint8_t  bytes[4];   // 4 bytes — same memory  
} sensor_t;
```

WARNING: Writing to one member corrupts the meaningful content of all other members.

Question 2 (short) — FreeRTOS task interface

Every FreeRTOS task function has the signature:

```
void TaskName(void* pvParameters);
```

It takes a single `void*` and returns `void`. The `void*` exists because FreeRTOS needs **one universal interface** for all tasks regardless of what data they need — the scheduler stores and passes the pointer without knowing its type. At the start of the task body, the programmer casts it to the expected type (e.g. a queue handle, a struct pointer). The function **never returns**; it runs in an infinite loop or calls `vTaskDelete(NULL)` to terminate itself.

Question 3 (short) — What goes into a C header file?

A header file contains **declarations**, not definitions:

- **Function prototypes** — the signature of functions implemented in the corresponding `.c` file.
- **Type definitions** — `typedef`, `struct`, `enum`, `union` declarations.
- **Macro constants** — `#define` values and inline macros.
- **Include guards** — `#ifndef MY_H / #define MY_H / ... / #endif` to prevent double-inclusion.
- **External variable declarations** — extern declarations of globals defined elsewhere.

It does **not** contain function bodies or variable definitions (those go in `.c` files).

Question 4 (short) — Adding a .c/.h file combination to a FreeRTOS project

Two steps:

1. **Register the source file** in main/CMakeLists.txt:

```
idf_component_register(SRCS "main.c"
                        "hl_wifi.c"
                        INCLUDE_DIRS ".")
```

Adding "hl_wifi.c" to SRCS tells the build system to compile it.

2. **Include the header** in any .c file that uses its functions:

```
#include "hl_wifi.h"
```

The header must have an include guard so it is safe to include from multiple translation units.

Question 5 (detailed) — Elixir actor communication: call vs. cast

Elixir actors (GenServer processes) communicate via two mechanisms:

GenServer.call/2 — synchronous: The caller sends a message and **blocks** until the server replies. The server handles it in handle_call/3 and returns {:reply, value, new_state}. The caller receives the return value.

GenServer.cast/2 — asynchronous: The caller sends a message and **returns immediately** with :ok. The server handles it in handle_cast/2 and returns {:noreply, new_state}. The caller gets no result.

When to choose call: - You need the return value. - You need to know the operation completed successfully before proceeding. - You want natural **back-pressure**: if the server is slow, the caller slows down automatically (because it blocks waiting for the reply).

When to choose cast: - You do not need a result. - The caller must not be delayed — fire-and-forget semantics are acceptable. - You are publishing an event where many receivers might listen.

Key trade-off: call gives synchronisation and back-pressure at the cost of caller latency. cast decouples caller and callee but the caller cannot detect if the server crashed or ignored the message. In practice, call is safer for anything where correctness matters; cast is appropriate for notifications and metrics where occasional loss is acceptable.

Question 6 (short) — ADC_ATTEN_DB_11 practical implications

ADC_ATTEN_DB_11 sets the input attenuation to 11 dB, expanding the ESP32 ADC's measurable input range from the default ~0-1.1 V up to approximately **0-3.9 V**. The practical implication is that sensors (such as the LMT86 thermistor circuit) whose output spans close to the full 3.3 V supply rail can be read without clipping. Without this attenuation, any input voltage above ~1.1 V would saturate the ADC and read as maximum, making

measurements above that level meaningless. The trade-off is slightly reduced accuracy at very low voltages.

Question 7 (detailed) — Noise, adaptive sampling, and radio budget

The problem: Adaptive sampling (thresholding) transmits only when the new reading differs from the last transmitted value by more than a threshold Δ . This is designed to save radio budget by suppressing transmissions when the physical signal is stable.

ADC noise introduces random fluctuations around the true signal value on every sample. If the noise amplitude σ_{noise} is comparable to or larger than the threshold Δ , then noise alone will cause the threshold to be crossed repeatedly — triggering transmissions even though the underlying physical quantity has not changed. In the worst case the device transmits at the sample rate rather than the signal-change rate, consuming the radio budget as if no thresholding were in place.

Mitigation — moving average (low-pass filter): Insert a running-average stage *before* the threshold comparison. Averaging N samples reduces the noise amplitude by $\sqrt{N}$ (SNR improves by $\sqrt{N}$). If the threshold is set above the smoothed noise floor, only genuine signal changes trigger transmissions.

This is exactly the pipeline from Exercise 11: Sample $\rightarrow$ Running Average $\rightarrow$ Threshold $\rightarrow$ Transmit. The running average acts as a low-pass filter, attenuating high-frequency noise while preserving slow-moving temperature trends.

Secondary mitigation — raise the threshold: Set $\Delta >$ noise amplitude. Trade-off: small but genuine signal changes below Δ are silently dropped, reducing resolution. The right choice depends on the application's accuracy requirements vs. radio budget constraint.

Question 8 (short) — Benefit of persistent sessions in MQTT

With a persistent session, the broker **buffers QoS 1 and 2 messages** addressed to a client while that client is disconnected (e.g. due to power cycling, poor WiFi, or sleep mode). When the client reconnects it receives all buffered messages immediately. This ensures no published data is lost during transient disconnections without requiring publishers to retransmit or the client to poll. It is particularly valuable for battery-powered IoT devices that disconnect frequently to save energy.

Question 9 (detailed) — Beamforming: benefits and mechanism

How it works: Beamforming uses an array of multiple antennas transmitting the same signal simultaneously, each with a controlled **phase offset** (and optionally amplitude weighting). The phase offset for antenna i is:

$$\phi = 2\pi d_i \sin(\theta) / \lambda$$

where d is the antenna spacing, λ is the wavelength, and θ is the target direction. At the target direction, the signals from all antennas arrive **in phase** and add constructively

(amplitude multiplied by N for N antennas). In other directions they arrive with varying phase relationships and tend to cancel destructively.

Benefits: 1. **Increased effective range** — the focused beam delivers more power to the target receiver, improving SNR at that location without increasing total transmit power. 2. **Reduced interference to other devices** — energy is not wasted radiating in all directions, reducing co-channel interference. 3. **Spatial multiplexing (MU-MIMO)** — multiple independent beams can serve different devices simultaneously on the same frequency channel, increasing total network throughput. 4. **Improved security** — signals are only strong in the intended direction, making eavesdropping from other angles harder.

Beamforming is used in 802.11ac/ax (WiFi 5/6) and 5G massive MIMO. Adaptive beamforming tracks moving clients by continuously updating the phase offsets based on channel feedback.

Question 10 (short) — Why benchmarking in public cloud is hard

Public cloud hardware is **shared** with other tenants (noisy neighbours) whose workloads compete for the same CPU cores, memory bandwidth, and network switches. The hypervisor can preempt your VM at any moment. Physical machine generation and configuration may vary between runs. All of these introduce **non-deterministic variance** in measured latency and throughput that is outside your control. Results are therefore not reproducible in the strict sense required for scientific benchmarking — two identical runs on “the same” cloud instance may produce significantly different results.

Question 11 (short) — Routing table for multihop routing

A routing table maps each **known destination** to a **(next-hop neighbour, cost/metric)** pair. To reach destination D , forward the packet to next-hop N ; the cost represents the total path metric (e.g. hop count, ETX — expected transmissions).

Maintenance: - **Distance Vector (RIP, RPL):** Each node periodically broadcasts its routing table to direct neighbours. Neighbours run Bellman-Ford to update their own table. Converges eventually but vulnerable to counting-to-infinity on link failure (mitigated by split horizon / poison reverse). - **Link State (OSPF):** Each node floods its local link costs to the entire network. Every node independently runs Dijkstra on the full topology. Faster convergence but higher overhead.

Question 12 (short) — When to choose LoRa over LoRaWAN?

Choose raw **LoRa** (physical layer only) when you want **full control** over the MAC layer and do not need LoRaWAN’s managed features:

- **Point-to-point or private star** networks where you implement your own addressing and duty-cycle management.
- **Custom MAC protocols** — e.g. TDMA slots, custom ACK schemes, or non-standard packet formats.

- **Direct device-to-device** communication without a network server intermediary.
- Situations where you **cannot use the LoRaWAN network server** (offline, airgapped, or cost-sensitive deployment).

If you need OTAA/ABP activation, ADR, multi-gateway redundancy, standardised AES security, or downlink management, use LoRaWAN instead.

Question 13 (short) — External ADC

An **external ADC** is a dedicated analogue-to-digital converter chip separate from the microcontroller. It is used when the MCU's on-chip ADC is insufficient — e.g. when higher resolution (16–24 bit), better noise performance, faster sampling rates, or more input channels are required.

Communication: Via a serial bus, typically: - **SPI** — for high-speed ADCs (MHz sample rates); full-duplex, chip-select per device. - **IC** — for slower, simpler setups; shared bus with address-based device selection.

The MCU initiates a conversion (or reads from a continuously converting device), then reads the digital result over the bus. The external ADC offloads analogue frontend quality concerns from the MCU silicon.

Question 14 — Clever Cup System Design

14.1 (short) — Sampling frequency for fluid temperature

Beverage temperature changes slowly — governed by Newton's Law of Cooling with a time constant of several minutes. The Nyquist theorem requires sampling at more than twice the highest frequency of interest. Since meaningful temperature variation occurs over ~minutes, the signal's highest relevant frequency is well below 0.01 Hz.

Recommendation: 0.1-1 Hz (once every 1-10 seconds). This far exceeds Nyquist for temperature dynamics, captures comfort-range violations promptly, and keeps processing and radio load low. For sip detection (via IMU/accelerometer), a higher rate of ~10-50 Hz is needed to detect the tilt gesture, but that is a separate sensor and duty-cycled burst.

14.2 (detailed) — Network topology and technologies

Constraints: The cup is battery-powered, mobile across a campus (~1 km), must support locate (LED trigger = downlink), comfort-range alerts (near-real-time), and optional long-term data upload.

Recommended topology: BLE to smartphone, smartphone to cloud.

```
[Clever Cup] -BLE- [User's Smartphone] -WiFi/LTE- [Cloud Backend]
                |
                Local alerts,
                LED locate command
```


Why BLE (PHY/Data Link): - Very low power on the cup (A in advertisement mode, mA during connection). - The user always has their phone nearby — range is not a problem. - BLE supports both uplink (temperature readings) and downlink (locate LED command, comfort-range config) in connected mode. - GATT profiles cleanly model temperature (Notify characteristic) and LED control (Write characteristic).

Why smartphone as gateway (fog tier): - The phone has WiFi/LTE with plentiful energy — radio budget constraint only applies to the cup. - Local comfort-range checking and push notifications can run on the phone without cloud round-trips, minimising latency for alarms. - The phone aggregates and timestamps readings before forwarding, solving the device timestamping problem (cup has no real-time clock).

Cloud backend: - MQTT broker (e.g. HiveMQ or self-hosted Mosquitto) receives timestamped temperature readings published by the phone. - Elixir/BEAM stream processing pipeline for long-term storage, visualisation, and pattern detection. - Topic structure: cup/<device_id>/temperature, cup/<device_id>/motion, cup/<device_id>/locate.

Alternative considered — LoRaWAN direct to campus gateway: Pros: no phone dependency. Cons: LoRaWAN Class A limits downlink (locate LED) to brief windows after an uplink; campus gateway infrastructure must exist; harder to achieve sub-second locate response.

14.3 (short) — Where should processing happen?

Location	Processing
On the cup (device/mist)	Comfort-range threshold check (avoid waking phone for every sample); sip detection from IMU; moving average to denoise temperature before threshold comparison.
On the smartphone (fog)	BLE gateway; timestamping; local push notification for comfort alarms; LED locate command relay.
In the cloud	Long-term time-series storage; behavioural analytics (drinking patterns, sip counting); deviation detection; visualisation dashboard.

The general principle: push filtering and urgent alerting **to the device** to save radio budget and reduce latency; push analytics and storage **to the cloud** where energy and compute are unconstrained.